Friedrich-Alexander-Universität

Pattern Recognition Lab

Continual Learning for Promptable Universal 3D Medical Image Segmentation in Few-Shot Settings

Master's Thesis in Computer Science

submitted
by

Shashank Shashank

born 25.03.1998 in Pehowa, India

Written at

Lehrstuhl für Mustererkennung (Informatik 5)
Department Informatik
Friedrich-Alexander-Universität Erlangen-Nürnberg.

Advisor: Prof. Dr.-Ing. habil. Andreas Maier, Dr.-Ing. Tomás Arias Vergara

Started: 01.06.2026

Finished: 30.11.2026

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Die Richtlinien des Lehrstuhls für Examensarbeiten habe ich gelesen und anerkannt, insbesondere die Regelung des Nutzungsrechts in Punkt 2.5

Erlangen, den ..... Unterschrift

Übersicht

VoxTell, ein universelles Modell für die durch Freitext-Prompts gesteuerte Segmentierung, segmentiert die Knorpel und Menisken des Knies nur unzureichend. Wird es nacheinander an solche Strukturen angepasst, droht es frühere Strukturen und vortrainierte Fähigkeiten zu vergessen. Wir haben VoxTell an sechs Kniestrukturen des Datensatzes SKM-TEA angepasst und sequenzielles Fine-Tuning mit fünf Strategien des kontinuierlichen Lernens verglichen, davon drei auf Basis der Low-Rank Adaptation (LoRA), mit 108 annotierten Volumen pro Struktur. Sequenzielles Fine-Tuning und zwei der LoRA-Strategien wurden zusätzlich mit nur 5, 10 und 20 Volumen pro Struktur trainiert. Eine zurückgehaltene Kontrollaufgabe, die Segmentierung der Lunge im CT, maß den Verlust vortrainierter Fähigkeiten getrennt vom Vergessen früherer Strukturen. Sequenzielles Fine-Tuning passte sich am besten an, vergaß aber die meisten früheren Strukturen und verlor die Kontrollaufgabe, während die LoRA-Strategien deutlich mehr behielten und sich deutlich weniger anpassten. Keine Strategie verband die Anpassung des sequenziellen Fine-Tunings mit dem Erhalt der LoRA-Strategien. Mit wenigen Volumen und zehn Epochen pro Struktur lernte nur sequenzielles Fine-Tuning die Strukturen. Ob die Lücke zwischen Anpassung und Erhalt auf die Strategien oder auf die gemeinsame Trainingskonfiguration zurückgeht, ist noch zu prüfen.

Abstract

VoxTell, a universal model for free-text promptable segmentation, segments the cartilages and menisci of the knee poorly. Adapting it to such structures one after another risks forgetting earlier structures and pretrained abilities. We adapted VoxTell to six knee structures of the SKM-TEA dataset and compared sequential fine-tuning with five continual-learning strategies, three of them based on low-rank adaptation (LoRA), using 108 labelled volumes per structure. Sequential fine-tuning and two of the LoRA strategies were also trained with only 5, 10, and 20 volumes per structure. A held-out control task, lung segmentation in CT, measured the loss of pretrained ability separately from the forgetting of earlier structures. Sequential fine-tuning adapted best but forgot most earlier structures and lost the control task, whereas the LoRA strategies retained much more and adapted much less. No strategy combined the adaptation of sequential fine-tuning with the retention of the LoRA strategies. With few volumes and ten epochs per structure, only sequential fine-tuning learned the structures. Whether the gap between adaptation and retention stems from the strategies or from the shared training configuration remains to be tested.

Contents

Notation **xi**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Question and Objectives . . . 4
1.4 Approach and Scope . . . 5
1.5 Contributions . . . 6
1.6 Thesis Outline . . . 7

2 State of the Art **9**
2.1 Text-Promptable Medical Image Segmentation . . . 9
2.2 Few-Shot Adaptation of Segmentation Models . . . 10
2.3 Continual Learning for Pretrained Vision–Language Models . . . 12
2.4 Continual Learning for Medical Image Segmentation . . . 14
2.4.1 Networks Without a Foundation Model . . . 15
2.4.2 Foundation Models Without Text Prompts . . . 16
2.4.3 Text-Promptable Foundation Models . . . 17
2.5 Summary and Research Gap . . . 18

3 Theoretical Background **23**
3.1 Free-Text Promptable Volumetric Segmentation . . . 23
3.1.1 Encoder–Decoder Networks for Volumetric Segmentation . . . 24
3.1.2 Attention . . . 24
3.1.3 VoxTell . . . 25
3.2 Continual Learning . . . 27
3.2.1 Problem Setting and Scenarios . . . 27

3.2.2 Low-Rank Adaptation . . . 28
3.2.3 Knowledge Distillation and ZSCL . . . 28
3.2.4 Prompt Learning and CPE-CLIP . . . 29
3.3 Evaluation Metrics . . . 30

4 Data Description **35**
4.1 SKM-TEA . . . 35
4.2 AeroPath . . . 37
4.3 Data Splits and Few-Shot Subsets . . . 38
4.4 Preprocessing . . . 38

5 Method **41**
5.1 Evaluation Protocol . . . 41
5.2 Adapting VoxTell to One Task . . . 44
5.3 Strategies . . . 47
5.3.1 Sequential Fine-Tuning . . . 49
5.3.2 Task-Specific and Shared LoRA . . . 49
5.3.3 Shared LoRA with Replay . . . 50
5.3.4 Prompt-Tuning Strategy . . . 51
5.3.5 Distillation-Based Strategy . . . 52
5.4 Computational Complexity . . . 53

6 Experiments and Results **55**
6.1 Experimental Setup . . . 55
6.1.1 Task Orders and Few-Shot Settings . . . 55
6.1.2 Training and Evaluation Configuration . . . 56
6.1.3 Metrics and Statistical Analysis . . . 57
6.2 Comparison of the Strategies . . . 58
6.3 Diagnosis of the Patellar Failure . . . 63
6.4 Amount of Labelled Data . . . 64
6.5 LoRA Rank . . . 67
6.6 Task Order . . . 69
6.7 Computational Cost . . . 72
6.8 Limitations and Generalisation . . . 73

7 Summary **77**

A Additional Experimental Details and Results **81**
A.1 Training and Test Volumes 81
A.2 Training and Strategy Parameters 82
A.3 Additional Evaluations 85
A.4 Weight Ensemble and Gradient Scaling 85
A.5 Evaluation Matrices of All Runs 87
A.6 Training Loss of Shared LoRA 101

List of Abbreviations **103**

List of Figures **105**

List of Tables **107**

Bibliography **109**

Notation

Scalars are written in italic, vectors in bold lowercase and matrices in bold uppercase letters. The section in the right column is where the symbol is first defined.

Symbol	Meaning	Sec.
Tasks and data		
T	number of tasks in the task sequence ($T = 6$)	3.2
t; j	task index; stage index ($j = 0$: pretrained model, $j = t$: after training on task t)	3.2
k	number of labelled training volumes per task in a few-shot setting	1.2
$\mathcal{D}_t^{\text{tr}}, \mathcal{D}_t^{\text{val}}, \mathcal{D}_t^{\text{te}}$	training, validation and test set of task t	3.2, 5.1
$\mathcal{D}_c^{\text{te}}$	test set of the control task	5.1
p	text prompt	3.1
p_t; p_c	text prompt of task t; text prompt of the control task	3.2, 5.1
$\boldsymbol{X}$	image volume or patch	3.1
$\boldsymbol{Y}$; $\hat{\boldsymbol{Y}}$	binary label mask; binary prediction	3.1
Ω; v	voxel set of a volume or patch; voxel	3.1
y_v; z_v; $\hat{y}_v$	label, logit and predicted foreground probability of voxel v	3.1
Model		
f_θ; θ	segmentation model; its parameters (for VoxTell all except those of the frozen text encoder)	3.1, 3.1.3
$\boldsymbol{Z}$	output logits of the model for a volume or patch	3.1
σ	logistic function	3.1
θ_0	pretrained parameters	3.2
θ_t	parameters after stage t, carried forward to the next task	3.2

Symbol	Meaning	Sec.
s	resolution stage ($s = 0$: full resolution, $s = 5$: bottleneck)	3.1.1
$\boldsymbol{F}_s$; C_s	encoder features of stage s; their number of channels	3.1.1
$\boldsymbol{U}$; $\boldsymbol{S}$	query vectors and source vectors of an attention block (rows)	3.1.2
n_q; n_s; n_h	number of queries, of sources and of attention heads	3.1.2
d; d_h	model dimension; dimension per attention head	3.1.2
$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_o$	query, key, value and output projections of an attention block	3.1.2
MHA; head_i	multi-head attention; output of attention head i	3.1.2
$\boldsymbol{x}$	input of a linear projection $\boldsymbol{x} \mapsto \boldsymbol{W}\boldsymbol{x}$	3.1.2
f_{text}; $\boldsymbol{q}$	frozen text encoder; text query $\boldsymbol{q} = f_{\text{text}}(p)$	3.1.3
L_{dec}	number of prompt-decoder layers ($L_{\text{dec}} = 6$)	3.1.3
G; $\boldsymbol{E}_s$	number of guidance vectors ($G = 32$); text-guidance matrix of stage s	3.1.3
$\boldsymbol{H}_s$; $\boldsymbol{H}'_s$	decoder features of stage s after and before the fusion with the text guidance	3.1.3
Up; ConvBlock	upsampling from stage $s+1$ to the resolution of stage s; convolutional layers of a decoder stage	3.1.3
Training		
$\mathcal{S}$	strategy	5.1
ϕ	parameters trained by a strategy	5.2
$\mathcal{L}_{\text{seg}}$	segmentation loss	3.1
E; N_t; U	epochs per task; training samples per epoch of task t; updates (iterations) per task, $U = E \cdot N_t$	5.2
$\mathcal{D}$; u; $\mathcal{L}$	training data of a task including the samples added by the strategy; update counter; total training loss	5.2
η_0; μ; ω	initial learning rate; momentum; weight decay	5.2
$\xi_{\mathcal{S},t}$	factor on the gradient of strategy $\mathcal{S}$ in task t (one except for the prompt-tuning strategy)	5.2
π_{fg}	probability that a training patch is centred on a voxel of the target structure	5.2
ℓ; ℓ^*; θ^{best}	validation loss of an epoch; lowest validation loss so far; parameters with the lowest validation loss	5.2

Symbol	Meaning	Sec.
Strategies		
$\boldsymbol{W}_0$	frozen pretrained weight adapted by LoRA	3.2.2
$\boldsymbol{A}$, $\boldsymbol{B}$; r	LoRA factors; rank	3.2.2
d_{in}; d_{out}	input and output dimension of an adapted weight	3.2.2
α; γ_r	LoRA scaling hyperparameter; scaling factor $\gamma_r = \alpha/\sqrt{r}$	3.2.2
$\mathcal{M}_t$; ρ; $M_{\max}$	replay memory after task t; replay fraction; capacity of the memory	5.3.3
β_t; $\mathcal{C}_t$	gradient scale of CPE-CLIP in session t; set of classes of session t in CPE-CLIP	3.2.4
$\boldsymbol{V}_l$; m; L_p	learned prompt tokens of prompt-decoder layer l; tokens per layer; number of prompted layers	5.3.4
h_ψ; ψ	meta-net; its parameters	5.3.4
$\boldsymbol{q}_l$	query entering prompt-decoder layer l, offset by the meta-net	5.3.4
$\bar{\boldsymbol{f}}$	mean of the projected stage-4 image features, input of the meta-net	5.3.4
λ	weight of the distillation loss	3.2.3
$\mathcal{L}_{\text{KD}}$	distillation loss	5.3.5
$\hat{\theta}_\nu$; ν; I_{we}	weight ensemble after ν updates; number of ensemble updates; interval between ensemble updates in training updates	3.2.3
$\boldsymbol{e}_g$	g-th guidance vector of $\boldsymbol{E}_4$	5.3.5
$\boldsymbol{f}_v$; Ω_4	stage-4 encoder feature at voxel v; voxels of stage 4	5.3.5
$w_{g,v}$; $\bar{w}_{g,v}$	distribution over Ω_4 for guidance vector g of the student; of the teacher	5.3.5
Evaluation		
$\text{DSC}(\hat{\boldsymbol{Y}}, \boldsymbol{Y})$	Dice similarity coefficient	3.3
$a_{i,j}$	entry of the evaluation matrix: mean DSC on task i after stage j	3.3
c_j	mean DSC on the control task after stage j	3.3
N_{boot}	number of bootstrap draws	6.1.3

Chapter 1

Introduction

1.1 Motivation

Medical image segmentation assigns every voxel of an image either to a structure, such as an organ or a tissue, or to the background. The voxels assigned to a structure form its mask. Masks can be drawn by hand. However, drawing them is laborious, since in a three-dimensional (3D) volume the structure has to be outlined on every slice in which it appears. Masks of the same structure also differ between readers [Des+21].

Automatic segmentation with deep learning has commonly meant training one model for each dataset and its target structures, for example with nnU-Net [Ise+21]. Universal segmentation models replace these per-dataset models with one network that segments many structures. For instance, the developers of the universal model SAT compared it with 72 nnU-Nets, one trained on each of their datasets [Zha+25d]. In such models, the structure to segment is specified by an additional input, the prompt [Zha+25d; Rok+26]. This prompt can be spatial, such as a point or a box marked in the image [Ma+24; Che+23], or text that names the target.

Text-promptable models differ in the text they accept. SAT is prompted with terms from a fixed vocabulary [Zha+25d], whereas VoxTell accepts both a single anatomical term and a full clinical sentence [Rok+26]. We call models of the latter kind *free-text promptable*. In both models, however, the structure to segment is not tied to an output channel of the network. Instead, the same weights serve every structure, and the text prompt alone selects the target [Zha+25d; Rok+26].

This thesis adapts VoxTell, which was trained on more than 62,000 computed tomography (CT), magnetic resonance imaging (MRI), and positron emission tomography (PET)

volumes [Rok+26]. On datasets it was not trained on, VoxTell reached a mean Dice similarity coefficient (DSC) of 70.85 % [Rok+26, Table 1], where the DSC measures the overlap between the predicted and the reference mask. On knee MRI from the SKM-TEA dataset [Des+21], however, its mean DSC was only 10.72 % (averaged over four cartilages and two menisci) [Rok+26, Suppl. Table 9]. The VoxTell authors attribute this low value to the small share of knee images in the training data and to structures missing from it. As future work, they suggest adapting the model with a few labelled examples [Rok+26, Sec. 6].

Automatic segmentation of the knee cartilages and menisci is useful because quantitative biomarkers for osteoarthritis are measured within these structures [Des+21]. One example is the T2 relaxation time, which reflects the degeneration of collagen-rich tissue [Des+21, Sec. 2]. These values are measured within the segmented structures. The SKM-TEA authors name manual outlining as one obstacle to the use of quantitative MRI [Des+21, Sec. 2].

With few manual outlines available, VoxTell would have to learn the knee structures from a few labelled volumes. New structures may also become relevant over time, and the model would then have to adapt to each of them in turn. Retraining VoxTell from scratch on its training data together with each new structure would be expected to preserve most of what it could segment before. However, training the published model took about six days on 64 NVIDIA A100 GPUs [Rok+26, Sec. 5].

1.2 Problem Statement

Instead of being retrained from scratch, the model can be adapted to one new structure at a time. We call the learning of one new structure, named by one text prompt, a *task*. In the task-incremental setting of continual learning, a model learns from a sequence of tasks, using the data of the current task and limited or no data of earlier tasks [DeL+22, Sec. 2]. After each task, the model has to segment the new structure while keeping the structures learned earlier in the sequence. It also has to keep what it could segment before the first task, which we call its pretrained ability (Figure 1.1).

The simplest way to learn a task is to fine-tune the model on its data. However, training on later tasks can lower the performance on earlier ones, an effect known as catastrophic forgetting [DeL+22, Sec. 1]. For pretrained vision–language models, continual fine-tuning can also lower the zero-shot accuracy on data the model was not trained on [Zhe+23]. In

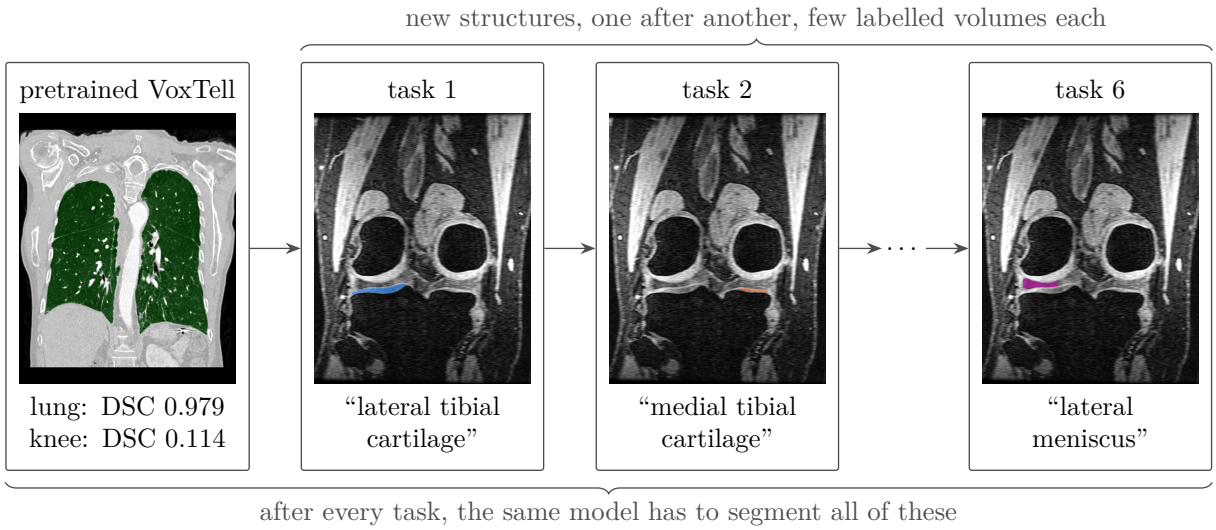


Figure 1.1: Setting of this thesis. The pretrained VoxTell segments the lung well but the knee structures poorly (DSC from our evaluation; for the knee, the mean over its six structures). It is then adapted to the six knee structures one after another, each named by a text prompt and learned from a few labelled volumes. Tasks 3 to 5 are not shown. After every task, the same model has to segment all of these structures, with the lung standing for its pretrained ability. Images: example volumes of Figure 4.1 with their reference masks.

our setting, such a lowering corresponds to a loss of pretrained ability. The more the model is allowed to change, the better it can learn the new structure, but the more it may lose of what it knew. De Lange et al. call this conflict the stability–plasticity dilemma, with plasticity “referring to the ability of integrating new knowledge” and stability “retaining previous knowledge while encoding it” [DeL+22, Sec. 1].

With few labelled volumes per task, it is not clear in advance how the balance between stability and plasticity shifts. When the number of training epochs is fixed, fewer volumes mean fewer updates. On the one hand, fewer updates give the model less opportunity to learn the new structure. On the other hand, they may change less of what it knew. In this thesis, *few-shot* means training the model on k labelled volumes per task, where one shot is one labelled training volume.

The continual-learning results we found do not show how few labelled volumes shift the balance between stability and plasticity, because they were obtained in other settings. The methods for pretrained vision–language models reviewed in Section 2.3 were developed and evaluated on classification or on tasks with text output. In medical image segmentation, many continual-learning methods give each new task or class components of its own, such as a decoder or an output head [Ji+23; Guo+25; Zha+23; Ye+25]. In contrast, VoxTell needs

no such components, since a new structure is named by a new text prompt and segmented with the same weights.

In our literature search, we found no work that adapts a pretrained free-text promptable 3D segmentation model to a sequence of new structures with few labelled volumes per task (Section 2.5). The closest works use either a two-dimensional (2D) model with full training sets [Gao26; Gao+26b; Gao+26a] or a 3D model without text prompts [Che+26b]. None of them measures how much of the pretrained ability is kept. Other works measure it for CLIP [Zha+24] and for segmentation models prompted with points or boxes [Wan+26b; Wan+24], and our evaluation builds on this idea (Section 1.4).

1.3 Research Question and Objectives

This thesis addresses the following research question:

> **How do different continual-learning strategies trade off adaptation performance and catastrophic forgetting when adapting free-text promptable universal 3D medical image segmentation models under few-shot conditions?**

By *adaptation performance*, we mean how well the model segments a new structure right after it has been trained on it. In the research question, *forgetting* covers both the loss on earlier tasks of the sequence and the loss of pretrained ability. As few-shot conditions, we use $k = 5$, 10, and 20 labelled volumes per task. We read them against the full training split of 108 volumes per task, which serves as the reference setting and is the only one in which all strategies are compared.

We pursue four objectives:

(1) Develop an evaluation protocol for the setting of the research question that measures adaptation, the forgetting of earlier tasks, and the loss of pretrained ability separately.

(2) Compare continual-learning strategies with each other and with fine-tuning on each task in turn, and determine how they trade adaptation against forgetting.

(3) Investigate how the number of labelled volumes per task influences adaptation and forgetting, separating the effect of fewer volumes from that of fewer updates.

(4) Analyse how the order of the tasks and the rank of the LoRA adapters influence adaptation and forgetting.

1.4 Approach and Scope

We study the research question with VoxTell [Rok+26] as the pretrained model and six structures of the SKM-TEA knee MRI dataset [Des+21] as tasks. These are four cartilages (patellar, femoral, medial tibial, and lateral tibial) and two menisci (medial and lateral). To see whether the results depend on the order of the tasks, we train them in two task orders, an original order chosen at random and its reverse. The lung in the chest CT volumes of AeroPath [Stø+24] serves as a *control task*. It is not trained on during the adaptation and is used only to measure how much of the pretrained ability is kept. After each task, the model is tested on all six tasks and on the control task (Section 5.1).

We compare sequential fine-tuning, the baseline, with five continual-learning strategies. Sequential fine-tuning trains all parameters on each task and adds nothing to reduce forgetting. Three strategies are based on low-rank adaptation (LoRA) [Hu+22], which keeps the pretrained weights frozen and trains only a low-rank update of them. We call this update the adapter. The number of parameters of the adapter grows with its rank r. Task-specific LoRA trains one adapter of rank $r = 4$ per task and selects it by the task identity at inference. In contrast, shared LoRA trains one adapter of rank $r = 1024$ for all tasks, and shared LoRA with replay adds a memory of training samples from earlier tasks. The prompt-tuning strategy (adapted from CPE-CLIP [DAl+23]) also freezes the pretrained weights and trains learned prompt tokens together with a meta-net, a small network that shifts the encoded text prompt. The meta-net is our addition, inspired by CoCoOp [Zho+22a]. The distillation-based strategy (adapted from ZSCL [Zhe+23]) trains all parameters and distils the features of the frozen pretrained model into the adapted one. Together, the five continual-learning strategies cover multimodal replay, cross-modal regularisation, and parameter-efficient adaptation, which are the three families of continual-learning methods for vision–language models [Liu+26b, Sec. 4]. All strategies except task-specific LoRA use one parameter set for all tasks.

All strategies are trained with the full training split. Sequential fine-tuning, task-specific LoRA, and shared LoRA are also trained with $k = 5$, 10, and 20 volumes per task. Further runs keep the number of updates of the full training split and so separate the effect of fewer volumes from that of fewer updates (Section 6.1.1).

From the test results, we compute three primary metrics (Section 3.3): Adaptation, the DSC on each task right after training on it; Final Forgetting [Cha+18], the loss on the earlier tasks by the end of the sequence; and the control-task drop, the loss on the control task. Together, Final Forgetting and the control-task drop measure retention, the performance

kept on earlier tasks and on the control task. Final Performance and Transfer [Zhe+23] are secondary. The pretrained model, evaluated in the same way, and joint fine-tuning, which trains the model on all six tasks at once, serve as references.

The study uses one model, one dataset for the tasks, one control task, one fixed text prompt per structure, and one draw of training volumes per k. Since each configuration was trained once, the only variability we can estimate is that over the test volumes (Section 6.1.3). Therefore, the conclusions are limited to this setting (Section 6.8). Clinical deployment and studies with clinical users are outside the scope of this thesis, but we report the training cost of the strategies (Section 6.7).

1.5 Contributions

The contributions of this thesis are:

(1) An evaluation protocol that measures the loss of pretrained ability separately from the forgetting of earlier tasks, through a held-out control task, with the pretrained model and joint fine-tuning as reference points (Section 5.1).

(2) A comparison of sequential fine-tuning and five continual-learning strategies adapted to VoxTell under one shared training configuration, together with a diagnosis of the failure of task-specific LoRA on patellar cartilage (Sections 5.3, 6.2 and 6.3).

(3) An analysis of how the amount of labelled data affects adaptation and forgetting, with three of the strategies and joint fine-tuning. It separates fewer volumes from fewer updates by training on five volumes with as many updates as on the full training split (Section 6.4).

(4) An analysis of how the task order and the rank of the LoRA adapters affect adaptation and forgetting, with two task orders and ranks from $r = 4$ to $r = 2048$ (Sections 6.5 and 6.6).

Sequential fine-tuning, LoRA, replay [DeL+22, Sec. 3.1], ZSCL, and CPE-CLIP are existing methods, and the metrics and the idea of a held-out control set come from prior work (Section 3.3). Our work consists of the application of these methods to VoxTell with the changes described in Section 5.3, the evaluation protocol that combines these metrics with a control task and a pretrained-model reference, and all experiments. The works reviewed in Section 2.5 do not study the setting of Section 1.2.

1.6 Thesis Outline

Chapter 2 reviews text-promptable medical image segmentation and few-shot adaptation of segmentation models, as well as continual learning for pretrained vision–language models and for medical image segmentation. It ends with the gap that this thesis addresses. Chapter 3 introduces VoxTell, states the continual-learning problem in formal terms, describes the published methods on which the strategies are based, and defines the evaluation metrics. Chapter 4 describes the SKM-TEA and AeroPath data, the tasks, and the data splits. Chapter 5 defines the evaluation protocol, the adaptation of VoxTell to one task, and the strategies with their complexity. Chapter 6 gives the experimental setup and the results. It compares the strategies, diagnoses the failure of task-specific LoRA on patellar cartilage, and reports the experiments on the amount of labelled data, the LoRA rank, and the task order, as well as the training cost. The chapter ends with the limitations of the experiments and the results that may generalise beyond them. Chapter 7 summarises the results, answers the research question, and outlines future work. The appendix lists the training and test volumes and the parameters of all runs (Sections A.1 and A.2) and reports the additional evaluations, including two components adopted from ZSCL and CPE-CLIP (Sections A.3 and A.4). It also gives the evaluation matrices of all runs and the training loss of shared LoRA (Sections A.5 and A.6).

Chapter 2

State of the Art

This thesis adapts a pretrained free-text promptable 3D segmentation model to a sequence of new target structures, each with few labelled volumes. The related work therefore comes from four areas: text-promptable medical image segmentation, few-shot adaptation of segmentation models, continual learning for pretrained vision–language models, and continual learning for medical image segmentation. The chapter reviews them in this order and closes with a summary of all reviewed works and the gap that this thesis addresses.

2.1 Text-Promptable Medical Image Segmentation

Promptable segmentation models differ in how the target is selected: either by a spatial prompt marked in the image or by a text prompt that names the target. MedSAM and SAM-Med2D are 2D adaptations of the Segment Anything Model (SAM) [Kir+23] to medical images that take spatial prompts. MedSAM is prompted with bounding boxes [Ma+24], and SAM-Med2D with boxes, points, and masks [Che+23]. SegVol, a 3D model for CT, accepts both kinds of prompt [Du+24]. Besides points and boxes, it takes category names, which it inserts into a fixed template and embeds with a frozen CLIP text encoder [Rad+21]. It was first pretrained on about 96,000 CT volumes without using labels and then fine-tuned on about 6,000 labelled volumes covering more than 200 categories. Other work keeps the target class fixed and uses text as additional input when labels are scarce, as LViT does with radiologist-verified descriptions of lung infections, which it needs at inference as well [Li+24].

Several text-promptable models select their target from a fixed vocabulary. The CLIP-Driven Universal Model feeds the CLIP embedding of a template such as “A computerized

tomography of a [CLS]" to a controller that generates the output convolutions of each class [Liu+23a]. It was trained on 3,410 CT scans from 14 public datasets with 25 organs and 6 tumour types. SAT extends text prompting to CT, MRI, and PET volumes. Its text encoder is trained on medical knowledge and embeds terms from a vocabulary of 497 classes, and a query decoder matches them with the features of a 3D U-Net [Zha+25d]. In the authors' demonstrations, free-form input such as a radiology report is first converted into terms of this vocabulary by a separate language model. On the test splits of 72 datasets, which hold 22,186 scans in total, SAT's larger variant, SAT-Pro, reached an average DSC of 78.81 % over all classes, which the authors report as comparable to the 80.54 % of 72 dataset-specific nnU-Nets. BiomedParse [Zha+25c] and MedSAM-3 [Liu+26a] also take text prompts, but they segment 2D images. On 3D data, SAT-Pro was better than BiomedParse on 29 of 30 categories, and the SAT authors attribute BiomedParse's false positives to its slice-wise design [Zha+25d].

VoxTell, the model adapted in this thesis, accepts a text prompt that may be a single anatomical term or a full clinical sentence [Rok+26]. It was trained on more than 62,000 CT, MRI, and PET volumes from 158 public datasets with 1,087 anatomical and pathological concepts. Unlike SAT, it fuses the text with the image features at every decoder resolution. On eleven datasets that were not used for training, its mean zero-shot DSC was 70.85 %, against 51.23 % for SAT [Rok+26, Table 1], but on the knee MRI of SKM-TEA [Des+21] it reached only 10.72 % [Rok+26, Suppl. Table 9].

The VoxTell authors report robustness to synonyms, rephrasings, and typos [Rok+26]. SEER measured how five text-promptable models respond to free-text requests that a language model generated in eight clinical styles [Zha+26]. Among these models, VoxTell kept the highest DSC under free-text requests, on average and in the worst case, whereas the DSC of SAT on the pelvic-bone dataset PENGWIN fell from 96.05 % with the class-name prompt to 0.01 %. Even for VoxTell, the worst rephrasing lowered the DSC on PENGWIN from 97.59 % with the class-name prompt to 79.34 %.

2.2 Few-Shot Adaptation of Segmentation Models

Each task in this thesis comes with only a few labelled training volumes, so this section reviews how pretrained models have been adapted to a single target with few labels. In the review by Dissanayake et al., a few-shot model has to segment a new class from one or a few labelled examples, and it can do so in two ways [Dis+25, Sec. 2.2]. In the first, the

model receives the labelled examples as input together with the image to segment, so a new class often needs no change to its weights. UniverSeg is an example of this approach for 2D medical images [But+23]. In the second, the weights of a trained model are updated on the labelled examples, for instance by fine-tuning [Dis+25, Sec. 2.2]. This thesis follows the second approach: a pretrained model is fine-tuned on a few labelled volumes of each task, and one labelled training volume counts as one shot.

Silva-Rodríguez et al. fine-tuned a Swin UNETR, pretrained on partially labelled CT scans, to one organ of TotalSegmentator at a time with 1, 5, or 10 labelled volumes [Sil+25]. This range overlaps with the 5, 10, and 20 labelled volumes per task used in this thesis. On nine organs that were already among the pretraining classes, LoRA [Hu+22] in the encoder and full fine-tuning differed by less than one percentage point of mean DSC at 5 and 10 shots, and of the two, only LoRA improved from 5 to 10 shots [Sil+25]. On 16 structures that were new to the model, parameter-efficient methods in the encoder alone reached at most 40.46 % at 10 shots, below full fine-tuning (52.38 %) and fine-tuning the decoder together with LoRA in the encoder (57.39 %). SEA-PEFT uses the same pretrained model and organs and selects the type, position, and rank of the adapters automatically during training [Ly+26].

For vision–language models, CLIP-LoRA adds rank-2 LoRA modules to the query, key, and value projections of every layer of both CLIP encoders [Zan+24]. It uses one set of hyperparameters for eleven 2D image classification datasets with 1 to 16 shots per class. With a ViT-B/16 image encoder, its average top-1 accuracy was the best of the compared methods at every number of shots, for example 72.5 % against 70.9 % at one shot. In the authors' ablation, the value or output projection was the best single matrix to adapt, and adapting more matrices or using a higher rank could lower the accuracy. The authors note that choosing the adapted matrices and the rank is not trivial. We examine the rank in our setting in Section 6.5. Like CLIP-LoRA, we apply LoRA to attention projections, but only to the query and value projections of the cross-attention in the prompt decoder of VoxTell (Section 5.3.2). CoOp and CoCoOp adapt a frozen CLIP in another way, by training context vectors in front of the class name [Zho+22b; Zho+22a]. The meta-net of our prompt-tuning strategy is inspired by CoCoOp.

The works above adapt a model to one target or dataset at a time, each time starting from the same pretrained weights. The two 3D studies evaluate the adapted model only on that target, so they do not show how much of its earlier ability the model loses through the adaptation [Sil+25; Ly+26]. In this thesis, the same model is trained on the tasks one after

another, so each adaptation can also affect the tasks trained before it. Among the works found in our search, only Chen et al. adapt a pretrained 3D medical model to a sequence of tasks with few labelled examples each, one of them a segmentation task [Che+26b].

2.3 Continual Learning for Pretrained Vision–Language Models

VoxTell combines a frozen text encoder with an image encoder and lets the encoded text prompt attend to the image features, which makes it a vision–language model [Rok+26, Sec. 3]. Methods that add tasks to such a model while keeping its text-conditioned abilities have been developed for pretrained vision–language models such as CLIP [Rad+21], and this section reviews them. They were developed for 2D images and evaluated by their authors on classification or on tasks with text output.

Continual-learning scenarios differ in whether the task identity of an input is given at inference [vdVen+25]. Independently of the scenario, De Lange et al. divide continual-learning methods into three families according to how they preserve information about earlier tasks [DeL+22, Sec. 3]. Replay methods reuse stored or generated samples of earlier tasks while learning a new one. Regularisation methods add a loss term, either distillation from an earlier model or a penalty on changes to important parameters. Parameter-isolation methods assign different parameters to different tasks, which typically requires the task identity at inference. For vision–language models, Liu et al. group methods into multimodal replay, cross-modal regularisation, and parameter-efficient adaptation, and note that many methods combine these families [Liu+26b, Sec. 4].

The multimodal replay method reviewed here avoids storing real images by generating the replayed data. GIFT fine-tunes all parameters of CLIP on each task together with 1,000 images that Stable Diffusion generates from class names of earlier tasks and of ImageNet [Wu+25]. On these images, it distils the image–text similarities of the model from before the task. Replay of stored real samples, which shared LoRA with replay uses, is reviewed for medical image segmentation in Section 2.4.1, where Bera et al. also compare how the stored data are selected.

ZSCL, a cross-modal regularisation method, showed that continual fine-tuning of CLIP also lowers its zero-shot accuracy on datasets that it has not been trained on yet [Zhe+23]. Its benchmark trains CLIP on eleven classification datasets as tasks in two task orders, Order I and Order II, and reports Transfer, the mean accuracy on each task before the

task is trained, and Last accuracy, the mean accuracy on all tasks after the last one. To keep the zero-shot accuracy, ZSCL fine-tunes all parameters and distils the image–text similarity distributions of the frozen initial model on a reference dataset, by default 100,000 ImageNet images, and additionally constrains the weights during training. On Order I, this raised Transfer from 44.6 % for sequential fine-tuning to 68.1 %, close to the 69.4 % of the zero-shot model, and Last accuracy from 77.3 % to 83.6 %. In the authors' ablation, distilling on the images of the current task instead of a reference dataset kept Transfer at the same level (56.7 % compared with 56.8 %) but lowered Last accuracy from 83.0 % to 80.2 % [Zhe+23, Table 1b]. The authors name the need for a reference dataset as the limitation of ZSCL. The distillation-based strategy of this thesis is adapted from ZSCL (Section 5.3.5). SPU measures the loss of zero-shot accuracy outside the task sequence on a control set, the ImageNet validation set, where it compares the adapted model with the frozen pretrained model [Zha+24]. Our control task follows this idea.

PRIMED transfers ZSCL's distillation on reference data to medical 2D image classification with BiomedCLIP [Che+26c]. It was evaluated on two task sequences that include X-ray, pathology, fundus, skin, cell, and endoscopy images, with the task identity given at inference. Instead of a fixed reference dataset, it retrieves image–caption pairs related to the tasks from 18 million PubMed figures and distils on them. Unlike ZSCL, PRIMED distils from the model of the previous task, and in its ablation this teacher gave a higher Last accuracy than the initial model (82.1 % compared with 78.2 %) [Che+26c, Table 4a]. The ablation of ZSCL had found the opposite, a Last accuracy of 83.0 % with the initial model and 80.7 % with the previous one [Zhe+23, Table 1d].

Parameter-efficient methods freeze most of the pretrained model and train a small set of added parameters [Liu+26b, Sec. 4.3]. Liu et al. note that tasks sharing the low-rank space of one LoRA can still interfere. Some methods therefore keep the updates of new tasks orthogonal to the subspaces of earlier tasks. LiLoRA, for continual instruction tuning of the multimodal language model LLaVA, shares one LoRA factor across tasks and splits the other into a shared and a task-specific part [Che+26a]. In its Single-type setting, one LoRA trained sequentially on all tasks reached an average accuracy of only 46.21 %, against 88.13 % for a separate LoRA per task. LiLoRA came close to the separate LoRAs with less than half their added parameters. MSLoRA-CR adds a LoRA branch per task to the frozen medical model LLaVA-Med and sums all branches learned so far [Zha+25a]. It was evaluated on 2D question answering, classification, and report generation, with the task identity given at inference, and reached a higher sum of all task metrics than a separate

LoRA per task. MoE-Adapters adds a mixture of LoRA experts with one router per task to a frozen CLIP [Yu+24]. At inference, one autoencoder per task selects the router, and the authors observed that this selection treated some samples of one task as out-of-distribution and passed them to the frozen CLIP. A separate LoRA per task and one LoRA shared by all tasks are the two LoRA strategies compared in this thesis.

Other methods train learned prompt tokens, which are vectors added to the input of a frozen model. S-Prompts learns separate prompt tokens for each domain and selects them at inference through the nearest of the image-feature centroids stored per domain [Wan+22]. CoLeCLIP learns one prompt token per task in the frozen image encoder and keeps a single vocabulary of class text embeddings for all tasks [Li+25]. CPE-CLIP targets few-shot class-incremental learning with five examples per new class [DAl+23]. It learns prompt tokens in both CLIP encoders, couples them through a learned projection, and scales their gradients down as more classes have been seen. It trains about 400,000 parameters. In its ablation on CUB200, a fine-grained bird classification dataset, removing the gradient scaling lowered the accuracy more than removing any other component [DAl+23, Sec. 5.5, Fig. 2]. The prompt-tuning strategy of this thesis is adapted from CPE-CLIP (Section 5.3.4).

The distillation and prompt-learning methods do not carry over directly to VoxTell. ZSCL, GIFT, and PRIMED distil distributions over the similarities between an image and a set of texts [Zhe+23; Wu+25; Che+26c]. VoxTell predicts one mask per text prompt and does not compute such distributions. CPE-CLIP places prompt tokens in transformer encoders [DAl+23], whereas the image encoder of VoxTell is convolutional [Rok+26, Sec. 3]. Section 5.3 describes how our strategies adapt these methods.

2.4 Continual Learning for Medical Image Segmentation

Unlike the methods for vision–language models, which classify images or generate text, the methods reviewed in this section segment medical images. In continual segmentation, a structure that is foreground in the masks of one task can be labelled as background in the masks of another, which is known as background shift [Cer+20]. In the multi-organ CT sequences of P. Liu et al., for example, the liver is foreground in the first dataset and background in the later ones [Liu+22]. Following Bommasani et al. [Bom+21, p. 3], we call a model a foundation model if it is pretrained on broad data and can then be adapted to many

downstream tasks. Section 2.4.1 reviews networks that do not build on a foundation model, and Sections 2.4.2 and 2.4.3 review foundation models without and with text prompts.

2.4.1 Networks Without a Foundation Model

Early work on continual learning for medical image segmentation applied regularisation and distillation. Baweja et al. used elastic weight consolidation (EWC) [Kir+17] for brain MRI [Baw+18], and Ozdemir et al. combined distillation from the previous model with a small set of stored examples for shoulder MRI [Ozd+18]. ILT, a method from natural-image segmentation, distils both the output and the encoder features of the previous model [Mic+19]. Several of the works reviewed below use it as a baseline [Ji+23; Liu+24].

In natural-image segmentation, MiB was the first method to explicitly address background shift, by treating old classes as possibly present in the background of a new task [Cer+20]. Like ILT, it is a baseline in several of the medical works reviewed below [Ji+23; Gon+23]. In medical image segmentation, P. Liu et al. used a marginal loss for partially labelled multi-organ CT [Liu+22], and X. Liu et al. used complementary pseudo-labels for structures added one after another in 2D brain-tumour MRI [Liu+23b]. Our masks also label the structures of the other tasks as background, but each mask is the target for one text prompt only, so no voxel receives different labels for the same text prompt. We therefore do not adopt these methods.

Replay methods for segmentation either store selected parts of earlier data or generate the replayed data. Bera et al. keep crops around the target instead of full volumes and select them by the fraction of target voxels and by training difficulty [Ber+23]. On a sequence of prostate, spleen, and hippocampus segmentation, twelve such crops gave an average DSC of $83.3 \pm 1.3\,\%$, against $78.3 \pm 1.3\,\%$ for three randomly chosen full volumes. C^2GR trains a universal segmentation network from scratch on 20 tasks from eight modalities, all as 2D images or slices, and replays image–mask pairs generated by a diffusion model [Li+26]. Each task is specified by a fixed text template that names the modality, the region, and the target. After the last task, the mean DSC over all tasks was $83.62\,\%$, against $86.06\,\%$ for joint training and $25.23\,\%$ for sequential fine-tuning.

Parameter-isolation methods keep separate parts of the model for different tasks or structures. Continual Segment freezes an encoder trained on TotalSegmentator and adds one pruned decoder per later dataset [Ji+23], and its extension CL-Net adds decoders for new anatomies [Guo+25]. In an extension of the CLIP-Driven Universal Model, Zhang et al. share the encoder and decoder across all classes, keep only the output heads class-specific,

and train the earlier classes on pseudo-labels from the previous model [Zha+23; Liu+24]. The weights of each head are generated by a separate multilayer perceptron (MLP) from the pooled image features and the CLIP embedding of the class template. After 13 BTCV organs followed by the LiTS liver tumour, the mean DSC was 0.817 on the organs and 0.466 on the tumour, against 0.770 and 0.456 with pseudo-labels alone. Since each class has to be named in advance and adds a head, the model grows with every class. In our setting, the text prompt names the target structure, and all strategies except task-specific LoRA use one parameter set for all tasks.

Some studies compare existing methods instead of proposing a new one. Lifelong nnU-Net trains an nnU-Net through sequences of hippocampus, prostate, and cardiac MRI datasets [Gon+23]. Replay, which the authors call rehearsal, showed the least forgetting, whereas EWC and MiB preserved the performance on earlier tasks at the expense of the new ones. On the last hippocampus dataset, for example, EWC and MiB reached a DSC of 31.93 % and 20.75 %, compared with 90.92 % for sequential training. The authors also observed that the task order affected the results and recommend comparing methods over several orders. Ranem et al. found the difference in forgetting between nnU-Net and a vision-transformer variant to depend on the method used [Ran+22]. The Beyond Forgetting benchmark compares 15 continual-learning methods on 2D slices for new domains, new structures in the same images, and new organs in new images, and reports plasticity and forward transfer besides forgetting [Wan+26a]. It names no single best method, but replay gave the best trade-off between retaining earlier tasks and learning new ones, and parameter isolation avoided forgetting at the cost of a growing model size. For the new-domain sequences, the task order affected the results.

2.4.2 Foundation Models Without Text Prompts

SAM-Med3D-MoE and CA-SAM add task-specific parts to a pretrained SAM-based model, which is prompted with points or boxes, and keep the pretrained model unchanged [Wan+24; Wan+26b]. SAM-Med3D-MoE fine-tunes one copy of the mask decoder of SAM-Med3D for each downstream task or poorly segmented class, separately rather than in a sequence. A gate selects one of these experts or falls back to the original decoder. With box prompts, the mean DSC on 15 poorly segmented classes rose from 0.532 to 0.564, whereas it fell from 0.541 to 0.400 on the classes that no expert was trained for [Wan+24, Table 1]. CA-SAM inserts one small alignment network per dataset between the frozen image encoder and the mask decoder of SAM, and one variational autoencoder per task selects the network or

passes unfamiliar inputs to the frozen SAM. On nine 2D datasets learned in sequence, its final mean intersection over union (IoU) was 76.12 %, against 78.43 % for joint training and 53.81 % for zero-shot SAM. On five datasets outside the sequence, its IoU after the last task stayed close to that of the frozen SAM, whereas Learning without Forgetting [Li+18] on the same alignment network fell far below it [Wan+26b, Table 2]. Both evaluations measure how much pretrained ability is kept outside the adapted tasks, as our control task does.

Two works adapt pretrained 3D models without spatial prompts. MedSeqFT fine-tunes a self-supervised CT model through five segmentation tasks in sequence, with a new head per task [Ye+25]. It adds distillation on stored samples, which are chosen by their pretraining loss, and a LoRA-based refinement that is merged into the weights. Its mean DSC over the five tasks was 68.3 %, against 65.3 % for fine-tuning a separate copy of the pretrained model per task. The paper reports no results on earlier tasks after later ones and no measure of forgetting. Chen et al. freeze a 3D brain-MRI foundation model and train one LoRA module and one head per task, selected by the task identity at inference [Che+26b]. Its two tasks are tumour segmentation and brain-age regression, run with 16, 32, or 64 labelled samples per task. With 32 samples, the tumour DSC of this design was 0.60 ± 0.08 and unaffected by the second task, whereas sequential fine-tuning reached 0.80 ± 0.02 and fell to 0.16 ± 0.19 after the second task [Che+26b, Table 1].

2.4.3 Text-Promptable Foundation Models

Three recent works by Gao and colleagues are the closest to this thesis. They train a text-promptable segmentation model on a sequence of medical segmentation tasks [Gao26; Gao+26b; Gao+26a]. All three use the 2D model CLIPSeg, treat each dataset or dataset subset as a task, mix endoscopy, dermoscopy, ultrasound, and X-ray images, and train on the full training sets. The text prompts of MedCRP-CL and MedPEFT-CL include descriptive sentences per image. PA-EWC uses templated phrases that range from class names to combinations of visual, spatial, and medical descriptions.

MedCRP-CL clusters the tasks by the CLIP embeddings of their text prompts and trains one LoRA module per cluster, with EWC within each cluster [Gao26]. On 16 tasks, its mean DSC after the sequence was 73.3 % with 4.1 % forgetting, compared with 65.3 % and 7.1 % for MoE-Adapters and 48.0 % and 28.3 % for sequential fine-tuning [Gao26, Table 2]. PA-EWC weights the EWC penalty of different parameter groups by the complexity of the text prompts [Gao+26b]. On five tasks, it reports a higher mean DSC and less forgetting than ZSCL and MoE-Adapters, although the averages in its summary table do not match

its per-task results [Gao+26b, Tables 3 and 5]. MedPEFT-CL adds LoRA to both CLIPSeg encoders, reuses the adapter of an earlier task when the text embeddings of the two tasks are similar, and replays the hardest 15 % of each task's training samples [Gao+26a]. On a sequence of ten tasks, its mean DSC was 78.19 %, against 74.15 % for MoE-Adapters and 81.02 % for one model per task. For MedCRP-CL and MedPEFT-CL, the papers do not state which adapter is used at test time, so it is unclear whether the adapter is selected by the task identity or by the text prompt.

2.5 Summary and Research Gap

Text-promptable 3D models such as SAT and VoxTell are trained once on many structures, and VoxTell's zero-shot results on the knee MRI of SKM-TEA show that its zero-shot generalisation does not extend to the knee structures. In the reviewed 3D studies, few-shot fine-tuning adapts 3D segmentation models to one new target with few labelled volumes and evaluates only that target. Continual-learning methods for pretrained vision–language models protect the pretrained representation while tasks are added and were evaluated by their authors on classification or on tasks with text output rather than on segmentation. Among the continual-learning works for medical image segmentation, only Chen et al. adapt with few labelled samples, and those conditioned on text are either 2D or limited to a fixed set of class names.

In our search[1], we found no work that adapts a pretrained free-text promptable 3D segmentation model to a sequence of new structures with few labelled volumes per task. The closest works each lack part of this setting: MedCRP-CL, PA-EWC, and MedPEFT-CL adapt the 2D model CLIPSeg with full training sets, and Chen et al. adapt a 3D model with few samples but without text prompts and with the task identity given at inference. Table 2.1 contrasts our setting with these works. None of these four works measures how much of the pretrained model's ability outside the task sequence is kept at the end of the sequence. SPU measures this for CLIP, and CA-SAM and SAM-Med3D-MoE for models prompted with points or boxes.

Unlike the closest works, which each propose a new method, this thesis adapts existing methods to VoxTell and compares the resulting strategies, as Lifelong nnU-Net and the Beyond Forgetting benchmark compare existing methods for networks without a foundation

[1] Keyword queries in OpenAlex, PubMed, and arXiv and a search of the works citing VoxTell and SAT, run on 23 October 2026; the 461 hits contained 332 unique titles, of which 43 passed a keyword filter on title or abstract; these 43 and the 23 works citing VoxTell were screened by title and abstract.

Table 2.1: Setting of this thesis compared with the closest works. Pretrained model: whether the task sequence starts from a foundation model (Section 2.4). Few-shot: labelled samples per task (no: full training sets). Control task: a held-out task evaluated after the sequence to measure the loss of pretrained ability (Section 1.4). [a]Only for sequential fine-tuning, task-specific LoRA and shared LoRA.

Work	**Text prompt**	**3D**	**Pretrained model**	**Few-shot**	**Control task**
Zhang et al., Liu et al. [Zha+23; Liu+24]	Fixed CLIP class names	yes	no (Swin UNETR)	no	no
MedSeqFT [Ye+25]	none	yes	yes (VoCo, UniMiSS+)	no	no
Chen et al. [Che+26b]	none	yes	yes	16, 32, 64	no
MedCRP-CL [Gao26]	Descriptive sentences	no	yes (CLIPSeg)	no	no
PA-EWC [Gao+26b]	Templated phrases	no	yes (CLIPSeg)	no	no
MedPEFT-CL [Gao+26a]	Descriptive sentences	no	yes (CLIPSeg)	no	no
This thesis	Free text, one fixed prompt per task	yes	yes (VoxTell)	5, 10, 20[a]	yes (AeroPath)

model. Table 2.2 summarises the data, approach, and best reported result of all works reviewed in this chapter.

Table 2.2: Works reviewed in this chapter, grouped by section. Works cited only for a definition, a method or a single finding are not listed. Results are given as reported by the authors, in the paper's metric, with the main comparison in parentheses; settings are named in the paper's own terms. FT: fine-tuning; seq.: sequential; TIL, DIL, CIL: task-, domain- and class-incremental learning (Section 3.2); MTIL: the eleven-dataset benchmark of ZSCL; FSCIL: few-shot class-incremental learning.

Work	Data	Approach and setting	Metric: best reported result
Text-promptable medical image segmentation (Section 2.1)			
SegVol [Du+24]	25 CT datasets; 3D	Points, boxes, class names in a template; not continual	Median DSC, AMOS22: 0.8593, box + text (SAM-Med2D 0.6668)
CLIP-Driven Universal Model [Liu+23a]	14 CT datasets; 3D	Fixed class names in a CLIP template; not continual	DSC, BTCV: 86.13 % (Swin UNETR 82.06 %)
SAT [Zha+25d]	72 CT, MRI, PET datasets; 3D	Terms from a fixed vocabulary; not continual	Mean DSC, all classes: 78.81 % (72 nnU-Nets 80.54 %)
VoxTell [Rok+26]	>62,000 CT, MRI, PET volumes; 3D	Free text; not continual	Mean zero-shot DSC, 11 unseen datasets: 70.85 % (SAT 51.23 %); SKM-TEA 10.72 %
Few-shot adaptation of segmentation models (Section 2.2)			
Silva-Rodríguez et al. [Sil+25]	TotalSegmentator, FLARE'22; 3D	Parameter-efficient vs. full FT; one target at a time; 1, 5, 10 (30) volumes	Mean DSC, 9 organs, 10 shots: 77.79 %, LoRA (full FT 76.93 %)
CLIP-LoRA [Zan+24]	11 classification datasets; 2D	Rank-2 LoRA in both CLIP encoders; 1–16 shots per class; not continual	Top-1 accuracy, mean: 72.5 % (1 shot), 83.0 % (16 shots)
Continual learning for pretrained vision–language models (Section 2.3)			
GIFT [Wu+25]	MTIL; 2D	Replay of generated images, full FT; MTIL, CIL	Transfer / Last, Order I: 69.3 / 86.0 % (ZSCL 68.1 / 83.6 %)
ZSCL [Zhe+23]	MTIL; 2D	Distillation on reference data, weight averaging, full FT; MTIL	Transfer / Last, Order I: 68.1 / 83.6 % (seq. FT 44.6 / 77.3 %)
PRIMED [Che+26c]	HieraMedTransfer, MedXtreme; 2D	Distillation on retrieved image–caption pairs; TIL, DIL	Transfer / Last, Order I: 58.3 / 82.1 % (ZSCL 57.7 / 77.6 %)
LiLoRA [Che+26a]	CVIT benchmark; 2D	Shared and task-specific LoRA parts; continual instruction tuning	Average accuracy, Single-type: 84.88 % (separate LoRAs 88.13 %, seq. LoRA 46.21 %)

continued on next page

Table 2.2 (continued)

Work	Data	Approach and setting	Metric: best reported result
MSLoRA-CR [Zha+25a]	Nine medical datasets; 2D	One LoRA per task, branches summed; TIL	Sum of task metrics: 995.21 (separate LoRAs 970.29)
CPE-CLIP [DAl+23]	CIFAR100, miniImageNet, CUB200; 2D	Prompt tokens in both encoders, gradient scaling; FSCIL	Average accuracy, CIFAR100: 83.42 % (zero-shot 70.58 %)
Continual learning for medical image segmentation (Section 2.4)			
Bera et al. [Ber+23]	Prostate, spleen, hippocampus datasets; 3D	Replay of selected crops; DIL, "task incremental"	Average DSC, three organs: 83.3 ± 1.3 % (random replay 78.3 ± 1.3 %)
C^2GR [Li+26]	20 tasks, 8 modalities; 2D	Diffusion replay, fixed task template, trained from scratch; TIL	Mean DSC, final: 83.62 % (joint 86.06 %)
Zhang et al., Liu et al. [Zha+23; Liu+24]	Private CT, BTCV, LiTS; 3D	Fixed CLIP class names, head per class, pseudo-labels; class-incremental	DSC, BTCV organs / LiTS tumour: 0.817 / 0.466 (PLOP 0.799 / 0.362)
Lifelong nnU-Net [Gon+23]	Hippocampus, prostate, cardiac MRI; 3D	Five methods compared; DIL	Rehearsal forgets least (DSC)
Beyond Forgetting [Wan+26a]	Prostate MRI, MMWHS, four organs; 2D	15 methods; Domain-, Class-, Organ-CL	Mean final DSC, Domain-CL: 0.789, PNN (joint 0.830)
SAM-Med3D-MoE [Wan+24]	SPPIN2023, poorly segmented classes; 3D	Points or boxes; decoder experts with a gate; not sequential	DSC, box prompts: poorly segmented 0.564 (0.532); without expert 0.400 (0.541)
CA-SAM [Wan+26b]	Nine datasets; 2D	Boxes; alignment layer per task, autoencoder router	Final IoU: 76.12 % (joint 78.43 %)
MedSeqFT [Ye+25]	Five CT, five MRI tasks; 3D	Seq. FT with distillation and LoRA refinement	Mean DSC, CT tasks: 68.3 % (parallel FT 65.3 %)
Chen et al. [Che+26b]	BraTS, IXI; 3D	LoRA and head per task, task identity given; 16, 32, 64 samples per task; task-incremental	DSC, tumour, 32 samples: 0.60 ± 0.08 (seq. FT 0.16 ± 0.19 after task 2)
MedCRP-CL [Gao26]	16 tasks, four modalities; 2D	CLIPSeg; LoRA per prompt cluster, EWC	Mean DSC / forgetting: 73.3 / 4.1 % (MoE-Adapters 65.3 / 7.1 %)

continued on next page

Table 2.2 (continued)

Work	Data	Approach and setting	Metric: best reported result
PA-EWC [Gao+26b]	Kvasir, ISIC, CheXlocalize, BUSI, CAMUS; 2D	CLIPSeg; EWC weighted by prompt complexity	Mean DSC: 75.34 % (ZSCL 73.02 %)[a]
MedPEFT-CL [Gao+26a]	Ten datasets; 2D	CLIPSeg; LoRA reused by prompt similarity, replay, EWC	Mean DSC: 78.19 % (MoE-Adapters 74.15 %)

[a] Summary-table value; it does not match the paper's own per-task results.

Chapter 3

Theoretical Background

This chapter introduces the concepts on which Chapter 5 builds. Section 3.1 describes volumetric segmentation networks, attention, and the free-text promptable model VoxTell. Section 3.2 states the continual-learning problem in the notation used throughout the thesis and describes the published methods from which our strategies are adapted. Section 3.3 defines the evaluation matrix and the metrics with which we measure adaptation and forgetting.

3.1 Free-Text Promptable Volumetric Segmentation

We consider the segmentation of one target structure in a 3D image volume. A single-channel volume $\boldsymbol{X}$ assigns an intensity to every voxel v of its voxel set Ω, and the label mask $\boldsymbol{Y}$ assigns $y_v = 1$ to the voxels of the structure and $y_v = 0$ to all others. For this structure, a segmentation network predicts one logit z_v per voxel. The logistic function σ turns each logit into a foreground probability $\hat{y}_v = \sigma(z_v)$, and thresholding these probabilities gives the binary prediction $\hat{\boldsymbol{Y}}$. In a free-text promptable model, a text prompt p names the target structure, so the structure is not tied to a fixed output channel of the network. One network with parameters θ then serves all target structures and computes the logit map $\boldsymbol{Z} = f_\theta(\boldsymbol{X}, p)$ with entries z_v. We write $\mathcal{L}_{\text{seg}}$ for the segmentation loss that compares $\boldsymbol{Z}$ with $\boldsymbol{Y}$.

VoxTell implements f_θ with a volumetric encoder–decoder network and attention between the text prompt and the image features.

3.1.1 Encoder–Decoder Networks for Volumetric Segmentation

The 3D U-Net [Çiç+16] is an encoder–decoder network for volumetric segmentation. Its encoder computes features at successively lower resolutions in a sequence of stages, and its decoder upsamples them back to the input resolution. Skip connections pass the encoder features of every stage to the decoder stage of the same resolution, so the decoder can use spatial detail that was lost by downsampling. We write $\boldsymbol{F}_s$ for the encoder features of stage s and C_s for their number of channels, with $s = 0$ at the full resolution and larger s at coarser resolutions. The authors of VoxTell regard this design as still the strongest architecture for 3D medical image segmentation [Rok+26, Sec. 3].

A whole volume and its intermediate features often do not fit into the memory of a GPU, so volumetric segmentation networks are trained on patches cut from the volumes. At inference, a sliding window applies the network to overlapping patches, and their predictions are combined into one prediction per volume. The nnU-Net framework configures these steps automatically for a given dataset [Ise+21].

3.1.2 Attention

Attention lets each of a set of query vectors collect information from a set of source vectors, weighting each source by how well it matches the query [Vas+17]. Let the rows of $\boldsymbol{U} \in \mathbb{R}^{n_q \times d}$ be n_q query vectors and the rows of $\boldsymbol{S} \in \mathbb{R}^{n_s \times d}$ be n_s source vectors of dimension d. Multi-head attention projects the query vectors to queries with a learned matrix $\boldsymbol{W}_q \in \mathbb{R}^{d \times d}$ and the source vectors to keys and values with $\boldsymbol{W}_k, \boldsymbol{W}_v \in \mathbb{R}^{d \times d}$, and it splits each projection into n_h heads of dimension $d_h = d/n_h$. We write all projections as maps $\boldsymbol{x} \mapsto \boldsymbol{W}\boldsymbol{x}$ of column vectors, so they appear transposed when applied to the row-wise inputs $\boldsymbol{U}$ and $\boldsymbol{S}$. Let $\boldsymbol{W}_q^{(i)}, \boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)} \in \mathbb{R}^{d_h \times d}$ denote the rows of the projection matrices that belong to head i. Head i and the output of multi-head attention are then

$$\begin{aligned} \mathrm{head}_i &= \mathrm{softmax}\left(\frac{\boldsymbol{U}\boldsymbol{W}_q^{(i)\top}\left(\boldsymbol{S}\boldsymbol{W}_k^{(i)\top}\right)^\top}{\sqrt{d_h}}\right)\boldsymbol{S}\boldsymbol{W}_v^{(i)\top}, \\ \mathrm{MHA}(\boldsymbol{U}, \boldsymbol{S}) &= \left[\mathrm{head}_1, \ldots, \mathrm{head}_{n_h}\right]\boldsymbol{W}_o^\top, \end{aligned} \tag{3.1}$$

where the softmax is taken over the n_s sources for each query, $[\cdot]$ concatenates the heads along the feature dimension, and $\boldsymbol{W}_o \in \mathbb{R}^{d \times d}$ is the output projection. Each query vector thus receives a weighted mean of the values, and the weights are given by the similarity

of its query to every key. The result does not depend on the order of the sources, so a positional encoding is usually added to them.

In self-attention, the query vectors and the source vectors are the same ($\boldsymbol{S} = \boldsymbol{U}$). In cross-attention, they come from different inputs, for example a text embedding and image features. Keys and values are computed from each source vector separately, so further rows appended to $\boldsymbol{S}$ give every query additional sources to attend to without any change to the projections. The prompt-tuning strategy (Section 5.3.4) relies on this property.

A transformer decoder layer combines attention blocks with a feed-forward network, two linear layers with a nonlinearity between them that are applied to every query separately [Vas+17]. Each block is wrapped in a residual connection and a layer normalisation. In the original transformer, each decoder layer applies self-attention to its queries and then cross-attention to the output of the transformer encoder.

3.1.3 VoxTell

VoxTell is a vision–language model that maps a 3D volume and a free-text prompt to a segmentation mask of the structure that the prompt describes. The text prompt can range from a single word to a clinical sentence. The model has four parts (Figure 3.1): a frozen text encoder, a convolutional image encoder, a transformer prompt decoder that relates the text to the image, and a convolutional decoder into which the text is fused at every resolution [Rok+26, Secs. 1, 3, and 5]. Section 5.2 describes where the version used in this thesis differs from the paper.

VoxTell is built on the nnU-Net framework [Rok+26, Suppl. Sec. A], and its image encoder is the residual encoder of the nnU-Net configuration ResEnc L [Ise+24, Sec. 3.1]. The encoder has six stages ($s = 0, \ldots, 5$) with 32, 64, 128, 256, 320, and 320 channels [Rok+26, Suppl. Sec. A.2]. For a patch of 192^3 voxels, the coarsest stage has 6^3 voxels. The text encoder is the embedding model Qwen3-Embedding-4B [Zha+25b], which maps the text prompt p to an embedding $\boldsymbol{q} = f_{\text{text}}(p)$ [Rok+26, Eq. 2]. Before encoding, a fixed instruction is placed in front of the text prompt. The instruction asks the encoder to retrieve the anatomical entity and location that the text prompt describes [Rok+26, Suppl. Sec. A.2]. The text encoder is frozen, so the parameters θ of VoxTell comprise all parameters except those of f_{text}.

The prompt decoder consists of $L_{\text{dec}} = 6$ transformer decoder layers of the kind described in Section 3.1.2, with eight heads and dimension $d = 2048$ [Rok+26, Sec. 3, Eq. 3, Suppl. Sec. A.2]. Its query is the text embedding $\boldsymbol{q}$, and its keys and values are the

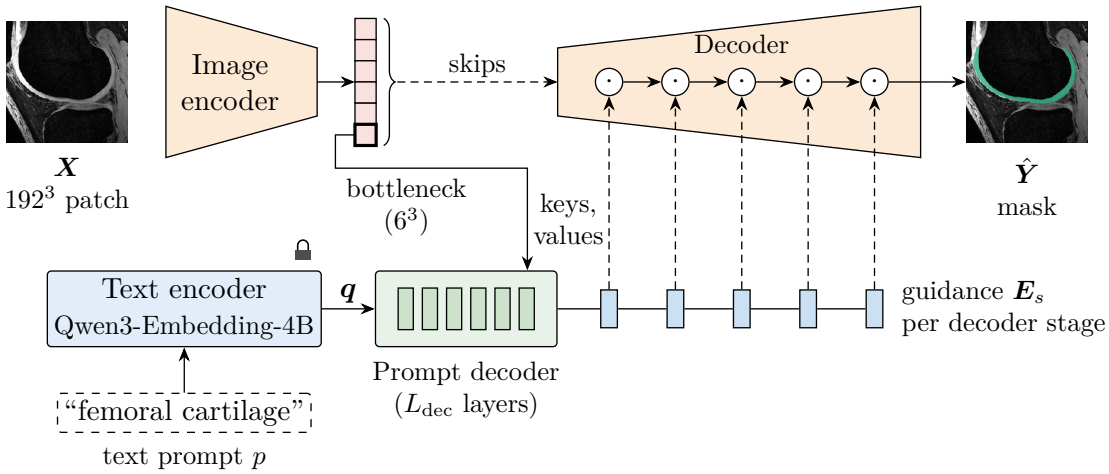


Figure 3.1: Architecture of VoxTell (adapted from [Rok+26, Fig. 2]). Each of the six stages of the image encoder passes its features to the decoder at the same resolution (skips). The text encoder embeds the text prompt into the query $\boldsymbol{q}$ and is frozen (lock). In the prompt decoder, $\boldsymbol{q}$ attends to the bottleneck features. The output of the prompt decoder is mapped to a text-guidance matrix $\boldsymbol{E}_s$ for each of the five decoder stages and fused into that stage (circles, Equation (3.2)). The input is a knee MRI patch from SKM-TEA with the text prompt "femoral cartilage". For illustration, the output shows the reference mask, not a prediction.

features $\boldsymbol{F}_5$ of the coarsest stage, the bottleneck. For each decoder stage s, a two-layer perceptron maps the output of the prompt decoder to a text-guidance matrix $\boldsymbol{E}_s \in \mathbb{R}^{G \times C_s}$. Its $G = 32$ rows are guidance vectors of dimension C_s. The guidance depends on both the text prompt and the image, because the query attends to the image features before it is mapped to the guidance.

The decoder upsamples the features stage by stage and fuses the guidance into every stage [Rok+26, Eqs. 4–6]. Starting from $\boldsymbol{H}_5 = \boldsymbol{F}_5$, it computes for $s = 4, \ldots, 1$

$$\boldsymbol{H}'_s = \text{ConvBlock}\left(\left[\,\text{Up}(\boldsymbol{H}_{s+1});\, \boldsymbol{F}_s\right]\right), \qquad \boldsymbol{H}_s = \left[\boldsymbol{H}'_s;\, \boldsymbol{E}_s \odot \boldsymbol{H}'_s\right], \tag{3.2}$$

where Up upsamples to the resolution of stage s, ConvBlock denotes the convolutional layers of a decoder stage, $[\cdot\,;\cdot]$ concatenates along the channel dimension, and $\boldsymbol{E}_s \odot \boldsymbol{H}'_s$ denotes the G maps of dot products between each guidance vector and the C_s-dimensional feature vector at every voxel. Each map thus shows how well the features at every voxel match one guidance vector, and the G maps enter all following convolutions as additional channels. At the full resolution, $s = 0$, the decoder features are combined with the guidance once more to give the output logits $\boldsymbol{Z}$.

VoxTell was trained with a combination of the binary cross-entropy and the Dice loss [Rok+26, Sec. 3, Eq. 7, Suppl. Sec. A.3]. With deep supervision, the loss was applied to the outputs of all five decoder stages, with weights from 1 at the full resolution down to 1/16 as in nnU-Net. Training used patches of 192^3 voxels, of which 85 % were sampled to contain foreground. It used the data augmentation of nnU-Net without left–right mirroring. The optimiser was stochastic gradient descent (SGD) with an initial learning rate of 10^{-4} and polynomial decay. Each training image was paired with three text prompts, two naming structures present in the image and one naming an absent structure, so that the model also learns to return an empty mask. Each text prompt was the default name of its concept with probability 0.25 and a rephrased variant otherwise.

3.2 Continual Learning

This section states the continual-learning problem in the notation of this thesis and defines three common continual-learning scenarios. It then describes the published methods on which our strategies are based, as they were originally proposed. These are low-rank adaptation, knowledge distillation with ZSCL, and prompt learning with CoCoOp and CPE-CLIP. Section 5.3 presents the corresponding strategies and explains how we adapted these methods to VoxTell and where our versions differ from the originals.

3.2.1 Problem Setting and Scenarios

In continual learning, a model is trained on a sequence of tasks $t = 1, \ldots, T$, one after the other. Training on task t starts from the parameters θ_{t-1} reached after the previous task, uses only the training data $\mathcal{D}_t^{\mathrm{tr}}$ of task t, and yields the parameters θ_t. In our setting, the sequence starts from the pretrained parameters θ_0, and each task has its own text prompt p_t. We index the resulting models by the stage j of the sequence: stage $j = 0$ is the pretrained model, and stage $j = t$ is the model after training on task t.

Van de Ven et al. distinguish three continual-learning scenarios by whether the task identity of an input is given at inference and, if it is not, whether the model has to determine it [vdVen+25]. In task-incremental learning, the task identity is given at inference. In domain-incremental learning, the possible outputs stay the same while the input data changes, and the model is not told the domain of an input. In class-incremental learning, the model has to distinguish all classes seen so far, including classes it never saw together in training, and thereby also has to infer the task.

3.2.2 Low-Rank Adaptation

LoRA keeps a pretrained weight matrix $\boldsymbol{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ frozen and learns an additive update of low rank [Hu+22, Sec. 4.1],

$$\boldsymbol{W} = \boldsymbol{W}_0 + \gamma_r \boldsymbol{B}\boldsymbol{A}, \qquad \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad \boldsymbol{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}), \tag{3.3}$$

where only $\boldsymbol{A}$ and $\boldsymbol{B}$ are trained and γ_r is a scaling factor. $\boldsymbol{A}$ is initialised randomly and $\boldsymbol{B}$ with zeros, so the update is zero at the start and training begins from the pretrained model. After training, $\gamma_r \boldsymbol{B}\boldsymbol{A}$ can be added to $\boldsymbol{W}_0$, so the adapted model has no additional inference cost, and switching to another task only requires exchanging the update. Hu et al. applied LoRA to the projection matrices of self-attention (Equation (3.1)). Comparing choices under a fixed number of trainable parameters, they found that adapting the query and value projections together gave the best performance overall [Hu+22, Secs. 4.2 and 7.1, Table 5].

In the original formulation, the scaling factor is α/r with a constant hyperparameter α [Hu+22, Sec. 4.1]. Kalajdzievski showed that this factor makes the gradients of the adapter collapse as the rank grows, so that large ranks learn no better than small ones. The paper also proves that, in the limit of large rank, only a factor of order $1/\sqrt{r}$ keeps the outputs and gradients of the adapter stable [Kal23, Secs. 2.1 and 3]. The resulting rank-stabilised LoRA (rsLoRA) sets

$$\gamma_r = \frac{\alpha}{\sqrt{r}}. \tag{3.4}$$

In fine-tuning experiments with a large language model and ranks up to 2048, rsLoRA improved with increasing rank while standard LoRA did not, and the gradient norms showed the same pattern with SGD as with AdamW [Kal23, Sec. 4].

3.2.3 Knowledge Distillation and ZSCL

In knowledge distillation, a student model is trained to reproduce the output distribution of a teacher model, where a temperature in the softmax softens the distribution [Hin+15, Sec. 2]. In continual learning, distillation serves as a regulariser. Learning without Forgetting (LwF) trains on the data of the new task only and uses the outputs of the model before this task as soft labels for the earlier tasks [Li+18]. The teacher is thus the previous model, and the distillation data are those of the current task.

ZSCL changes two choices of LwF to preserve CLIP's zero-shot transfer during continual learning [Zhe+23, Sec. 3.2, Eqs. 2–4]. The teacher is the initial pretrained model instead of the previous one, and the distillation data are a separate reference dataset of unlabelled public images and arbitrary texts instead of the current task's data. For each image, a softmax turns CLIP's similarities to a set of texts into a distribution, and the loss compares the distributions of student and teacher by a cross-entropy. A second term does the same for each text, with a distribution over the images of a batch, and both terms are added to the task loss with a weight λ. ZSCL also limits the change of the parameters themselves. The full method adds a weight-consolidation loss, a penalty on the squared distance between the parameters and a reference set of parameters [Zhe+23, Sec. 3.3, Eq. 5, Table 2]. Every I_{we} iterations, it also updates a weight ensemble, the running mean of the parameters along the training trajectory, starting from θ_0,

$$\hat{\theta}_\nu = \frac{1}{\nu+1}\,\theta + \frac{\nu}{\nu+1}\,\hat{\theta}_{\nu-1}, \qquad \hat{\theta}_0 = \theta_0, \tag{3.5}$$

where θ are the current parameters and ν counts the ensemble updates [Zhe+23, Sec. 3.3, Eq. 7].

3.2.4 Prompt Learning and CPE-CLIP

Prompt learning adapts a frozen vision–language model by training a few input vectors instead of its weights. CoOp replaces the hand-written context words of a text prompt, such as "a photo of a", by learnable context vectors in front of the class name [Zho+22b]. CoCoOp adds a small network, the meta-net, which computes a token from the image features and adds it to every context vector, so that the context vectors depend on the input image [Zho+22a, Secs. 3.3 and 4.1]. Its authors report that these image-dependent context vectors generalise better to classes not seen in training.

CPE-CLIP applies prompt learning to few-shot class-incremental learning [DAl+23, Secs. 1 and 3]. In this setting, a base session with many classes and examples is followed by sessions that each add a few classes with few examples, and the task identity is not given at evaluation [DAl+23, Secs. 2.3 and 3]. CPE-CLIP keeps CLIP frozen and learns prompt tokens in the layers of the language encoder up to a given depth, where each layer replaces the prompt tokens it receives by its own learned ones [DAl+23, Sec. 4, Eqs. 2–4]. The prompt tokens of the vision encoder are derived from those of the language encoder by a learned linear projection [DAl+23, Sec. 4, Eqs. 6–8]. To limit forgetting, CPE-CLIP

scales the gradients of the prompt tokens and of the projection in session t by

$$\beta_t = \frac{|\mathcal{C}_t|}{\sum_{\tau=0}^{t} |\mathcal{C}_\tau|}, \tag{3.6}$$

where $\mathcal{C}_t$ is the set of classes of session t, so that the prompt tokens change less as more classes have been learned [DAl+23, Sec. 4, Eqs. 11–13].

3.3 Evaluation Metrics

This section defines the segmentation metric and the continual-learning metrics computed from it.

The overlap between a binary prediction $\hat{\boldsymbol{Y}}$ and the label mask $\boldsymbol{Y}$ is measured with the DSC [Dic45],

$$\mathrm{DSC}(\hat{\boldsymbol{Y}}, \boldsymbol{Y}) = \frac{2\,|\hat{\boldsymbol{Y}} \cap \boldsymbol{Y}|}{|\hat{\boldsymbol{Y}}| + |\boldsymbol{Y}|}, \tag{3.7}$$

where $|\cdot|$ counts the foreground voxels of a mask and $\hat{\boldsymbol{Y}} \cap \boldsymbol{Y}$ denotes the voxels that are foreground in both. The DSC is 0 for disjoint masks and 1 for identical ones. The DSC is normalised by the sizes of the two masks and not by the size of the volume, so the many background voxels do not inflate it. This matters for structures as small as the knee cartilages and menisci (Section 4.1).

Evaluating the model on every task after every stage shows how training on later tasks affects earlier ones [Zhe+23, Fig. 4b]. We write $a_{i,j}$ for the mean DSC on the test set of task i after stage j, with $i = 1, \ldots, T$ and $j = 0, \ldots, T$. Together, the values $a_{i,j}$ form the evaluation matrix. The forgetting and intransigence measures of Chaudhry et al. were originally defined for classification accuracy, but the authors note that a task-specific measure such as the IoU of a segmentation can take the place of accuracy without changing the definitions [Cha+18, Sec. 3]. The diagonal entries $a_{i,i}$ show how well task i is learned when it is trained. The entries with $j > i$ show how much of this performance is kept after later tasks, and the entries with $j < i$ show the performance on task i before it is trained. Entries $a_{i,j} < a_{i,i}$ with $j > i$ therefore indicate catastrophic forgetting of task i. The matrix does not show whether the model loses pretrained abilities outside the task sequence, because its rows cover only the tasks of the sequence. We measure this loss of pretrained ability with a control task, a held-out task that is never trained on. We write c_j for its mean DSC after stage j.

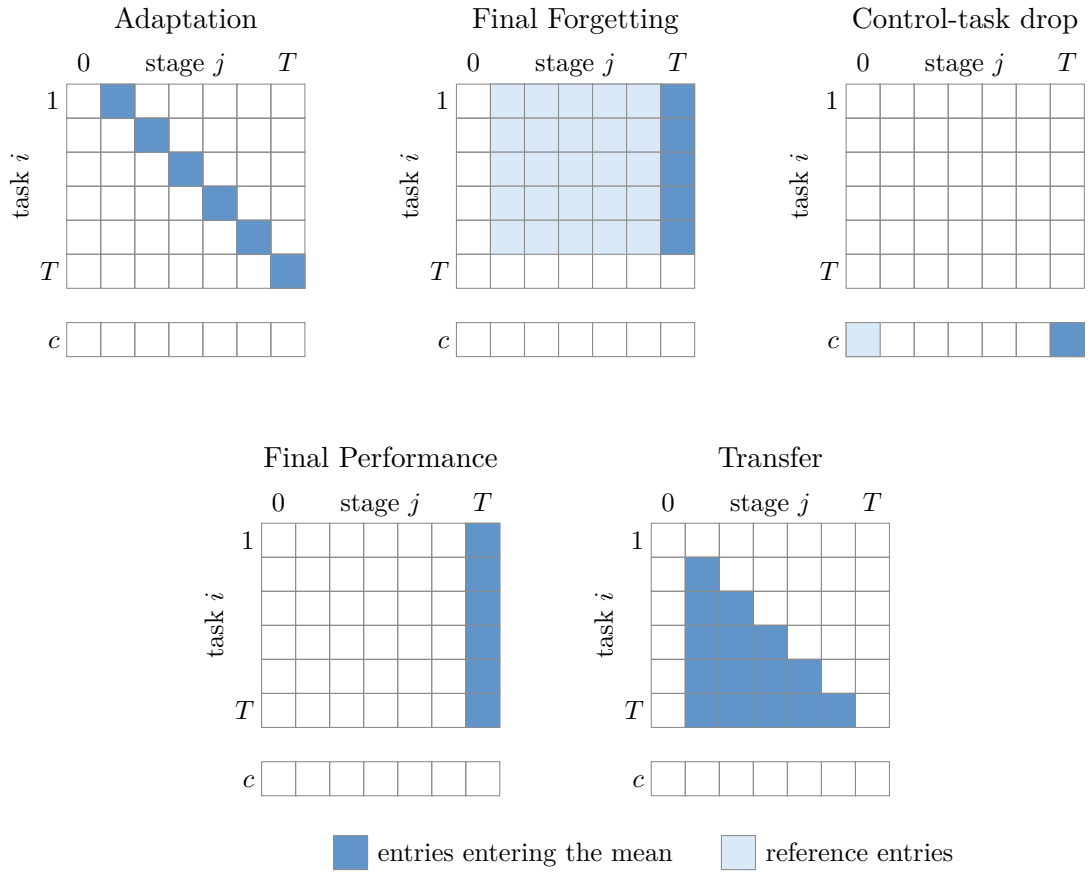


Figure 3.2: Entries of the evaluation matrix used by each metric. Rows are the tasks $i = 1, \ldots, T$, and columns are the stages $j = 0, \ldots, T$, with stage 0 the pretrained model. The separate row c is the control task. Dark cells enter the mean of a metric. Light cells are the reference from which the dark cell of the same row is subtracted, namely their maximum for Final Forgetting and the pretrained value c_0 for the control-task drop.

We summarise the evaluation matrix with four metrics, three taken from the literature and one derived from it, and the control task with a fifth. Figure 3.2 shows which entries each metric uses.

Adaptation. Adaptation measures how well each task is learned when it is trained,

$$\text{Adaptation} = \frac{1}{T} \sum_{i=1}^{T} a_{i,i}. \tag{3.8}$$

Each diagonal entry $a_{i,i}$ also appears in the intransigence measure of Chaudhry et al. This measure subtracts it from the performance on task i of a reference model trained on the data of tasks $1, \ldots, i$ together [Cha+18, Sec. 3, Eq. 4].

Final Forgetting. Final Forgetting is the average forgetting of Chaudhry et al. after the last task [Cha+18, Sec. 3, Eq. 3]. For each task before the last, it subtracts the performance at the end from the best performance at any stage from 1 to $T-1$,

$$\text{Final Forgetting} = \frac{1}{T-1} \sum_{i=1}^{T-1} \Big(\max_{j \in \{1,\ldots,T-1\}} a_{i,j} - a_{i,T} \Big). \tag{3.9}$$

Taking the best earlier performance instead of the value right after training also counts gains that a task made while later tasks were trained and lost again by the end. Lower values are better, and a negative value means that training on later tasks improved earlier ones, which Chaudhry et al. call positive backward transfer.

Control-task drop. The control-task drop follows SPU, which uses a control set to measure how much of a pretrained model's general ability is lost during continual learning [Zha+24, Sec. 5.1, Eq. 6]. SPU computes this loss as the accuracy of the frozen pretrained model on the control set minus that of the continually trained model. We compute the drop with the DSC, between the pretrained model and the model after the last stage of a task sequence,

$$\text{Control-task drop} = c_0 - c_T, \tag{3.10}$$

so a positive value is a loss and lower values are better.

Final Performance. Final Performance is the metric Last of ZSCL, the mean performance on all tasks after the last stage [Zhe+23, Sec. 4, Fig. 4b],

$$\text{Final Performance} = \frac{1}{T} \sum_{i=1}^{T} a_{i,T}. \tag{3.11}$$

A common reference for Final Performance is joint training, in which one model is trained on the data of all tasks at once. De Lange et al. describe joint training as a soft upper bound for continual learning [DeL+22, Sec. 6.2.1].

Transfer. Transfer is the metric of the same name in ZSCL [Zhe+23, Sec. 4, Fig. 4b]. For each task i, it averages the performance at the stages $1, \ldots, i-1$ before the task is trained, and it then averages these values over the tasks,

$$\text{Transfer} = \frac{1}{T-1} \sum_{i=2}^{T} \frac{1}{i-1} \sum_{j=1}^{i-1} a_{i,j}. \tag{3.12}$$

Stage $j = 0$ is not included, as in ZSCL, whose evaluation matrix has no entries for the model before the first training step. The first task therefore does not enter Transfer. When the task sequence starts from a pretrained model, as here, an entry $a_{i,j}$ with $j < i$ reflects both the ability of the pretrained model on structure i and the effect of training on tasks $1, \ldots, j$. Transfer does not separate the two. The pretrained-model reference row (Section 5.1) shows the first part alone.

Chapter 4

Data Description

This chapter describes the two public datasets used in this thesis. SKM-TEA [Des+21] provides the knee MRI volumes from which the six continual-learning tasks are built, and AeroPath [Stø+24] provides the chest CT volumes of the control task. Table 4.1 summarises both datasets. The chapter then describes how the volumes are split, how the few-shot training subsets are drawn, how the data are preprocessed, and which properties of the data limit the conclusions that can be drawn from them.

4.1 SKM-TEA

The Stanford Knee MRI with Multi-Task Evaluation (SKM-TEA) dataset contains knee MRI scans of 155 patients acquired at Stanford Healthcare [Des+21]. We chose SKM-TEA because VoxTell performs poorly on it (Section 1.1). SKM-TEA is distributed under the terms of the Stanford University School of Medicine and of the Lower Extremity Radiographs dataset [Des+21, App. A.5]. We signed the Stanford AIMI research-use agreement before downloading the data.

All scans were acquired with the 3D quantitative double-echo steady-state (qDESS) sequence on one of two 3 T GE MR750 scanners, with patient consent and approval of the Stanford University Institutional Review Board [Des+21]. In qDESS, two 3D images with different contrasts, called echoes, are acquired in one scan and are therefore registered to each other [Des+21, Sec. 2]. SKM-TEA defines two benchmark tracks, one based on raw k-space data and one based on the scanner-generated Digital Imaging and Communications in Medicine (DICOM) images [Stab]. We use the DICOM track, which stores the images and segmentations of each scan in one Hierarchical Data Format version 5 (HDF5) file [Stab].

Table 4.1: Datasets used in this thesis. Values computed from the data and the official SKM-TEA annotation files (version 1.0.0), except the rows on imaging, source, and access, which are taken from the dataset publications [Des+21; Stø+24], and the choice of echo, which follows the SKM-TEA tutorial [Staa].

	SKM-TEA	AeroPath
Role in this thesis	six continual-learning tasks	control task
Imaging	MRI, 3D qDESS, echo 1	CT angiography
Source	Stanford Healthcare	St. Olavs Hospital
Volumes (patients)	155 (155)	27 (27)
With pathology label[a]	130 of 155	–
Split (train/val/test)	108/23/24	19/4/4
Volume size (voxels)	512 × 512 × 144–168	487–512 × 407–512 × 241–843
Voxel size (mm)	0.3125 × 0.3125 × 0.8[b]	0.56–0.82 × 0.56–0.82 × 0.50–1.25
Access	github.com/StanfordMIMI/skm-tea	github.com/raidionics/AeroPath

[a] At least one pathology bounding box in the official SKM-TEA annotation files (cartilage lesion, effusion, meniscal tear, ligament tear).
[b] From the official annotation files; the NIfTI files used in this thesis store an identity affine (1 mm).

We use only the first of the two echoes, as the dataset authors do in the segmentation part of their tutorial [Staa]. The dataset also provides versioned annotation files that list the scans of each split defined by the dataset authors, with metadata such as the voxel size [Stab]. All 155 volumes have a voxel size of $0.3125 \times 0.3125 \times 0.8$ mm.

Four tissues were segmented manually on all DICOM images, labelled as six structures: patellar cartilage, femoral cartilage, medial and lateral tibial cartilage, and medial and lateral meniscus [Des+21]. The annotators drew the masks in sagittal slices and then corrected them for consistency in the axial and coronal planes [Des+21, App. A.4]. Each scan was labelled by one of two researchers with three to four years of experience in knee MRI interpretation, supervised by two board-certified musculoskeletal radiologists [Des+21, App. A.4]. The dataset therefore contains no information on the agreement between annotators. The dataset also provides bounding boxes for sixteen pathology categories [Des+21, Sec. 4.1]. We do not use them. According to the annotation files, 130 of the 155 scans contain at least one of these pathologies.

Each of the six segmented structures defines one task. The text prompt of a task is the name of its structure in lower case: “patellar cartilage”, “femoral cartilage”, “medial tibial cartilage”, “lateral tibial cartilage”, “medial meniscus”, and “lateral meniscus”. The same text prompt is used for training and evaluation, and its wording is not varied. Of the six

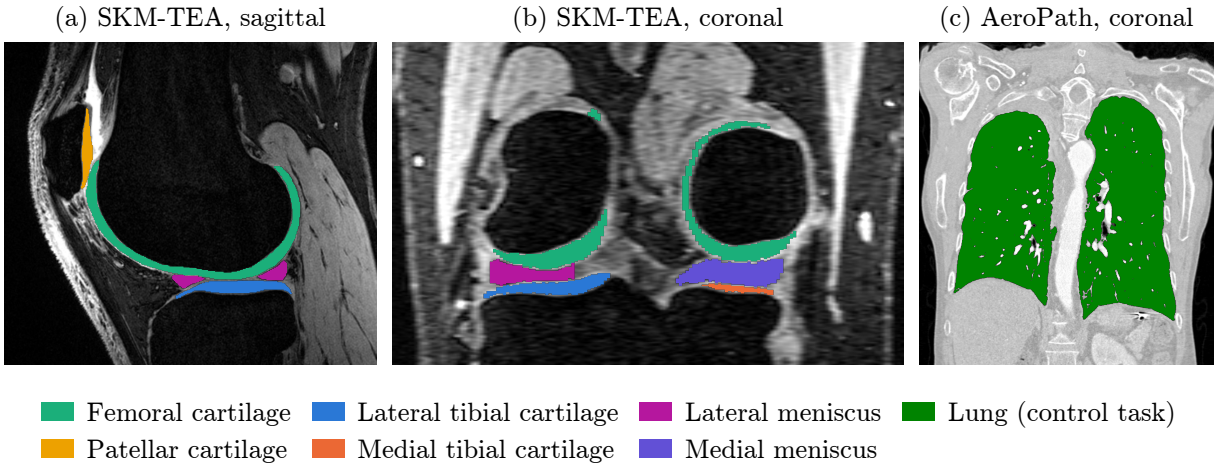


Figure 4.1: Example volumes with their labels. (a) Sagittal and (b) coronal slice of SKM-TEA volume MTR_005, cropped to the knee joint; each slice shows only the structures it intersects. In (a), anterior is on the left. In (b), the lateral side is on the left, and the image is coarser along the left–right axis, the slice direction. (c) Coronal slice of AeroPath volume 1 with the lung label of the control task. Both example volumes belong to the test split.

structures, the four cartilages are among VoxTell's training labels, whereas the two menisci are not [Rok+26, Suppl. Table 10]. Since all six structures are labelled in every volume, the six tasks share the same 155 volumes and differ only in the target structure. The structures are small, with median volumes from 1.90 cm^3 (lateral meniscus) to 10.99 cm^3 (femoral cartilage), and lie next to each other in the knee joint (Figure 4.1). The order in which the tasks are trained is part of the experimental setup and is given in Section 6.1.1.

4.2 AeroPath

AeroPath contains 27 contrast-enhanced chest CT scans (computed tomography angiography (CTA)) of patients undergoing diagnostic tests for lung cancer, acquired with a thorax lung protocol at St. Olavs Hospital in Trondheim, Norway [Stø+24]. Data collection was approved by the regional ethics committee, and the patients gave written informed consent [Stø+24]. The dataset is distributed under the Creative Commons Attribution 4.0 licence [Hof+23]. AeroPath provides the control task, for which we use only the lung label of each volume, with the text prompt "lung". Neither the publication, the dataset repository, nor the Zenodo record documents how the lung labels were created, so it is unknown whether they are manual or automatic references. AeroPath was a held-out test set in the VoxTell

paper [Rok+26, Suppl. Table 7]. However, it is among the sources added to the training data of checkpoint v1.1, which we use [Rok26]. VoxTell's training labels include "lung", in 3,427 training cases [Rok+26, Suppl. Table 10].

4.3 Data Splits and Few-Shot Subsets

We split each dataset once, at random. After shuffling the volumes with a fixed random seed (42), we assign the first 70 % of the shuffled list to the training split, the next 15 % to the validation split, and the remaining volumes to the test split. This gives 108 training, 23 validation, and 24 test volumes for SKM-TEA and 19, 4, and 4 volumes for AeroPath. Of the AeroPath volumes, we use only the 4 test volumes, to evaluate the control task. Because the six tasks share the same volumes, all tasks and all experiments in this thesis use the same split. Each SKM-TEA patient contributes exactly one scan [Des+21, Sec. 4.1], so no patient appears in more than one split. The validation volumes are used to monitor training and, in an additional evaluation, to select the checkpoint with the lowest validation loss (Section 5.2). All reported results are computed on the test volumes.

SKM-TEA also defines an official split of 86 training, 33 validation, and 36 test scans, whose test set consists of the 36 patients who received additional arthroscopic surgery [Des+21]. We use our own random split instead. Unlike the official split, it does not allow a comparison with published SKM-TEA results. For comparing the strategies with each other, it is sufficient that all strategies use the same split.

For the few-shot settings, we reduce the training split to its first k volumes in the order of the shuffled list, where $k \in \{5, 10, 20\}$ is the number of labelled training volumes per task. The subsets are therefore nested, with the 5 volumes contained in the 10 and the 10 contained in the 20. For each k, all six tasks use the same k volumes. Only one subset is drawn for each k, so we cannot measure how much the results depend on which volumes are chosen. Runs with the full training split use all 108 training volumes. The validation and test volumes are the same for every k and for the full training split.

4.4 Preprocessing

We convert each SKM-TEA HDF5 file into two Neuroimaging Informatics Technology Initiative (NIfTI) files, one for the first echo and one for the labels. In the HDF5 file, the segmentation is stored as six binary channels, one per tissue. We merge them into a single

label map, in which unlabelled voxels take the value 0 and labelled voxels the number of their channel in the order of the dataset documentation [Stab]: 1 for patellar cartilage, 2 for femoral cartilage, 3 for medial tibial cartilage, 4 for lateral tibial cartilage, 5 for medial meniscus, and 6 for lateral meniscus. The binary target mask of a task consists of the voxels with that task's label value. The NIfTI files are written with an identity affine, so they store neither the voxel size nor the orientation of the scan. Images and labels come from the same voxel grid, so they stay aligned. The AeroPath volumes and labels are used in their original NIfTI format.

When a volume is loaded, it is reoriented according to its affine and cropped to the bounding box of its non-zero voxels. The reorientation leaves the SKM-TEA volumes unchanged because their affine is the identity. According to the annotation files, 18 of the 155 SKM-TEA scans store the left–right axis in the opposite direction to the other 137. The intensities of each volume are then normalised to zero mean and unit standard deviation over the cropped volume, with the z-score normalisation of nnU-Net [Ise+21]. The same normalisation is applied to the MRI and the CT volumes. The volumes are not resampled, so the network receives each volume on its stored voxel grid, and the voxel size in the affine has no effect on the input. This preprocessing follows the configuration distributed with the VoxTell checkpoint, which uses z-score normalisation for all inputs, including CT, and a preprocessing plan without resampling.[1] Section 5.2 describes how patches are sampled during training and how predictions are assembled during evaluation.

[1] File `plans.json` of the checkpoint `voxtell_v1.1` in the Hugging Face repository `mrokuss/VoxTell`: normalisation scheme `ZScoreNormalization`, no target spacing, image reader `NibabelIOWithReorient` [Hug26].

Chapter 5

Method

This chapter describes how we address the research question and the reasons for the main design choices. Section 5.1 defines the continual-learning setting and the protocol that measures adaptation and forgetting, and Section 5.2 describes VoxTell and the procedure that adapts it to one task. Section 5.3 describes the six strategies that we evaluate, each defined as a variant of this procedure: sequential fine-tuning, which serves as the baseline, and five continual-learning strategies. Section 5.4 compares their computational cost, and Chapter 6 gives the values of all parameters and describes the experiments.

5.1 Evaluation Protocol

We consider a sequence of $T = 6$ tasks built from SKM-TEA. Each task $t \in \{1, \dots, T\}$ is defined by one knee structure, its text prompt p_t, and the binary masks of that structure (Section 4.1). We write $\mathcal{D}_t^{\text{tr}}$, $\mathcal{D}_t^{\text{val}}$, and $\mathcal{D}_t^{\text{te}}$ for the training, validation, and test sets of task t, each consisting of the volumes of the corresponding split (Section 4.3) paired with the masks of structure t. Because the six tasks share the same volumes, catastrophic forgetting shows up here as a loss of segmentation performance on earlier structures in the same images.

Each task has one fixed text prompt, which names its target structure during training and at inference, so the text prompt also identifies the task. The setting is therefore not class-incremental, because the model never has to infer which of the learned structures is meant. Nor is it domain-incremental, because the text prompt always makes clear to which task an input belongs. Of the three scenarios (Section 3.2.1), it is closest to task-incremental learning, in which the task identity is given at inference.

The control task (Section 3.3) is the segmentation of the lung in the AeroPath test volumes $\mathcal{D}_c^{\text{te}}$ with the text prompt $p_c =$ "lung". We chose it because lung is a concept in the training data of VoxTell that the pretrained model segments well (Section 6.2). AeroPath itself is among the training sources of checkpoint v1.1 (Section 4.2), so the control task measures the loss of pretrained ability on data the model was trained on. In the same way, the control set of SPU is sampled from the pretraining distribution [Zha+24, Sec. 3]. On all 27 AeroPath volumes, VoxTell segmented the lung with a DSC of 96.99 % [Rok+26, Suppl. Table 7], so a loss of this ability would be clearly visible. AeroPath is also openly available, which keeps the control task reproducible.

The protocol trains a strategy on the T tasks in sequence and, after every stage j, evaluates it on the test sets of all tasks and of the control task. This yields the evaluation matrix of the values $a_{i,j}$ and the control row of the values c_j (Section 3.3), from which all metrics of this thesis are computed. We evaluate all tasks at every stage, including tasks that have not been trained yet ($i > j$), because these entries show how training on one structure affects the segmentation of structures that come later in the sequence. Section 6.1.3 explains which of the metrics we report and why.

Each entry $a_{i,j}$ or c_j is the per-volume DSC between the binary prediction and the label mask, averaged over the volumes of one test set. For the binary prediction, each test volume is cropped and normalised as described in Section 4.4 and segmented with sliding-window inference. Patches of 192^3 voxels are placed with a step of half the patch size, and their logits are averaged with a Gaussian weighting, as in nnU-Net [Ise+21]. The Gaussian has a standard deviation of one eighth of the patch size. The averaged logits are passed through the logistic function and thresholded at 0.5, and the DSC is computed on the cropped volume. No test-time augmentation is used.

All entries of all runs use the same test volumes: the 24 SKM-TEA test volumes for the tasks and the 4 AeroPath test volumes for the control task. Differences between entries, and between runs, therefore never come from different test data. We also keep the per-volume values, from which Section 6.1.3 estimates the variability of the metrics over test volumes.

Algorithm 1 summarises the protocol. ADAPT adapts the model to one task with the procedure of Section 5.2, which each strategy changes as described in Section 5.3. EVALUATE computes one entry $a_{i,j}$ or c_j, the mean per-volume DSC on one test set. For a strategy $\mathcal{S}$, SELECT$_{\mathcal{S}}$ determines which parameters are used to evaluate task i after stage j. For all strategies except task-specific LoRA, these are the parameters after stage j.

Algorithm 1 Continual adaptation and evaluation protocol for strategy $\mathcal{S}$.

Input: pretrained parameters θ_0; strategy $\mathcal{S}$; tasks $t = 1, \ldots, T$ in the chosen task order, each with text prompt p_t and sets $\mathcal{D}_t^{\text{tr}}, \mathcal{D}_t^{\text{val}}, \mathcal{D}_t^{\text{te}}$; control task with p_c and $\mathcal{D}_c^{\text{te}}$
Output: evaluation matrix $(a_{i,j})$ with $i = 1, \ldots, T$, $j = 0, \ldots, T$; control row (c_j)

for $j = 0$ **to** T **do**
  if $j > 0$ **then**
    $\theta_j \leftarrow \textsc{Adapt}(\theta_{j-1}, j, \mathcal{S})$ {Algorithm 2}
  end if
  for $i = 1$ **to** T **do**
    $a_{i,j} \leftarrow \textsc{Evaluate}(\textsc{Select}_{\mathcal{S}}(\theta_{0:j}, i), p_i, \mathcal{D}_i^{\text{te}})$
  end for
  $c_j \leftarrow \textsc{Evaluate}(\textsc{Select}_{\mathcal{S}}(\theta_{0:j}, c), p_c, \mathcal{D}_c^{\text{te}})$
end for

Task-specific LoRA instead returns the pretrained parameters with the adapter chosen by the task identity, and without an adapter for the control task (Section 5.3.2).

The protocol has two variables that are set per run. The first is the task order, the fixed order in which a run trains all tasks. The second is the amount of labelled data per task, either the full training split or a few-shot subset of k volumes (Section 4.3), with the same k for all tasks of a run. With a fixed number of epochs, fewer volumes also mean fewer updates per task. A few-shot run can therefore also be trained with as many updates per task as on the full training split, which separates the effect of fewer volumes from that of fewer updates (equal-updates runs). Section 6.1.1 gives the task orders, the values of k, and the equal-updates runs that we used, together with the reasons for them.

As a reference, we apply the same metrics to the pretrained model itself, treating it as if it had been evaluated unchanged after every stage, that is, with $a_{i,j} = a_{i,0}$ and $c_j = c_0$ for all j. This pretrained-model reference shows whether a strategy moved away from the pretrained model at all and in which direction. Because the task order determines which task is task i, and some metrics depend on this, we compute this reference once for each task order.

As an upper reference, we use joint fine-tuning, which trains one model on the training sets of all six tasks at once, in a single stage. Each training sample keeps the text prompt of its own structure. Joint fine-tuning trains all parameters except the text encoder with the procedure and settings of Section 5.2, so the joint model receives as many updates as sequential fine-tuning over the whole sequence. After its single stage, it is evaluated on the test set of every task and on the control task. Without a task sequence, Adaptation, Final Forgetting, and Transfer are not defined for it. We therefore compare it with the strategies

on Final Performance, on the final DSC of each structure, and on the control-task drop. We do not compute the intransigence measure of Chaudhry et al. [Cha+18], because its reference for task i is a model trained jointly on tasks $1, \ldots, i$. Joint fine-tuning is trained on all six tasks and therefore provides this reference only for $i = T$.

5.2 Adapting VoxTell to One Task

This section describes VoxTell as we use it and the procedure ADAPT, which adapts it to task t in stage t of the protocol (Section 5.1).

We start every run from version 1.1 of the released VoxTell checkpoint [Rok+26; Hug26], whose architecture is described in Section 3.1.3. We use VoxTell because it accepts free-text prompts and reached the highest mean zero-shot DSC in its authors' comparison, ahead of SAT (Section 2.1). Figure 5.1 shows the model as used here and the parts that the strategies change.

Two properties of the released implementation determine where the strategies of Section 5.3 act. First, the prompt decoder takes its keys and values from the output of encoder stage 4, not from the bottleneck as in Eq. 3 of the paper [Rok+26]. For an input of 192^3 voxels, this output has 12^3 voxels. We use the released model unchanged and therefore keep this configuration. The distillation-based strategy and the prompt-tuning strategy both use these stage-4 features. Second, each of the L_{dec} prompt-decoder layers applies only a cross-attention block and a feed-forward block, each preceded by layer normalisation and followed by a residual connection. Each layer also contains a self-attention module, but the released configuration never executes it, so its parameters are never updated. The LoRA strategies and the learned prompt tokens of the prompt-tuning strategy act on the cross-attention. Apart from the text encoder, which stays frozen in every strategy, VoxTell has 440,029,541 parameters, of which 339,292,517 can be updated. The parameters that cannot be updated belong to the unused self-attention modules and the layer normalisations before them.

ADAPT (Algorithm 2) starts from the parameters θ_{t-1} of the previous stage and trains on the training volumes of task t, each paired with the task's text prompt. It returns the parameters θ_t, which are evaluated and from which the next task starts. As in the released VoxTell pipeline, each text prompt is placed in a fixed instruction before it is encoded: "Instruct: Given an anatomical term query, retrieve the precise anatomical entity and location it represents", followed by "Query:" and the text prompt. All strategies use this

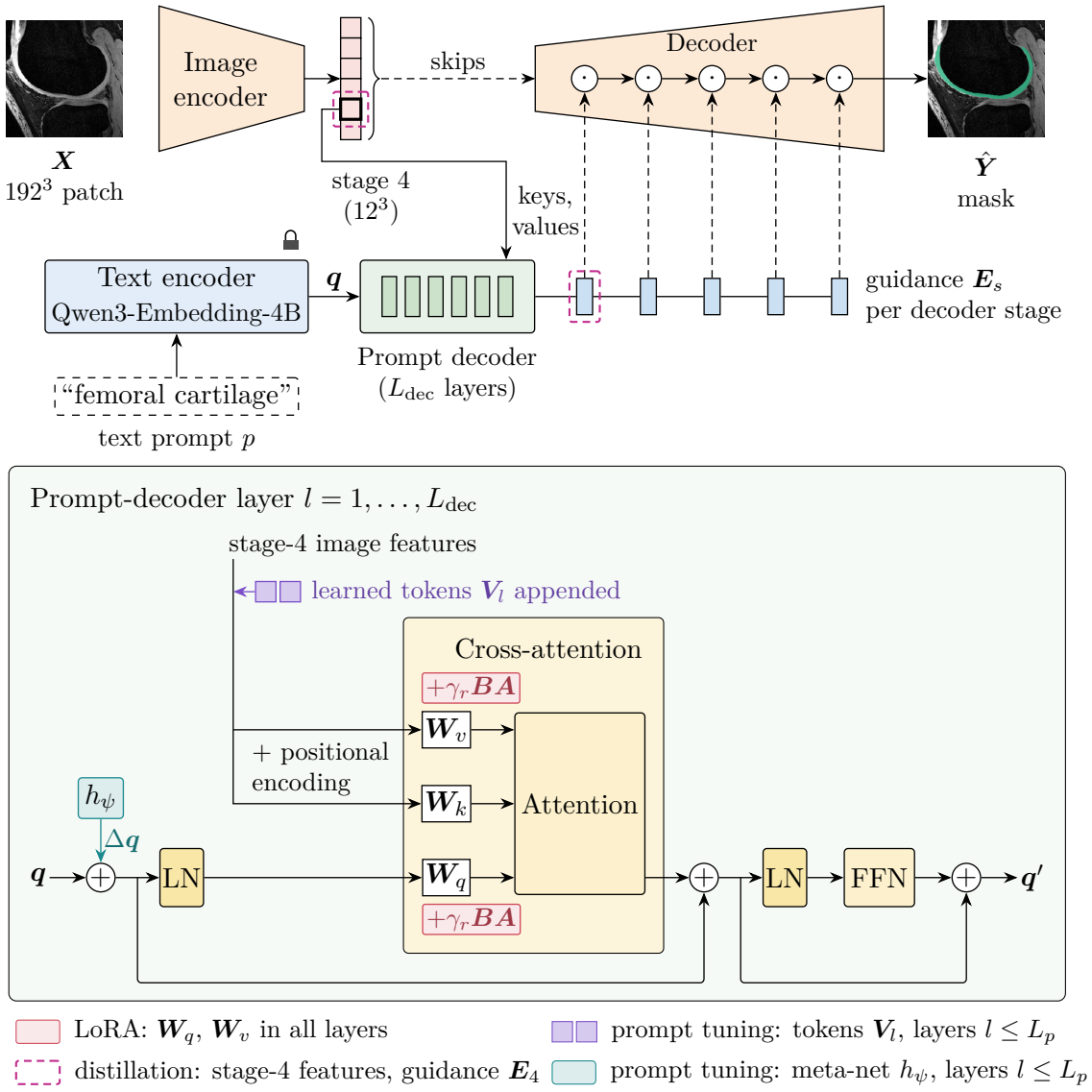


Figure 5.1: VoxTell as used in this thesis and the parts that the continual-learning strategies change (adapted from [Rok+26, Fig. 2]). Top: the prompt decoder attends to the stage-4 features, and its output is mapped to the guidance $\boldsymbol{E}_s$ of the five fusion points. The lock marks the frozen text encoder; the output panel shows the reference mask of femoral cartilage in an SKM-TEA volume, not a prediction. Bottom: one prompt-decoder layer as executed; the attention block includes the output projection. Legend terms refer to the strategies of Section 5.3; sequential fine-tuning and the distillation-based strategy update all parameters except the text encoder, and replay changes only the training data.

Algorithm 2 ADAPT: adapting VoxTell to task t. The steps with subscript $\mathcal{S}$ are where the strategies differ (Section 5.3).

Input: parameters θ_{t-1} carried forward from the previous task; task t with text prompt p_t, $\mathcal{D}_t^{\text{tr}}$, and $\mathcal{D}_t^{\text{val}}$; strategy $\mathcal{S}$; E, η_0, μ, ω, π_{fg}
Output: parameters θ_t after the last epoch

$(\theta, \phi) \leftarrow \textsc{Init}_{\mathcal{S}}(\theta_{t-1}, t)$ {model and trainable parameters ϕ}
$\mathcal{D} \leftarrow \mathcal{D}_t^{\text{tr}} \cup \textsc{ExtraData}_{\mathcal{S}}(t)$
$N_t \leftarrow |\mathcal{D}|$; $\quad U \leftarrow E \cdot N_t$; $\quad u \leftarrow 0$; $\quad \ell^* \leftarrow \infty$
for $e = 1$ **to** E **do**
  for each volume $(\boldsymbol{X}, \boldsymbol{Y}, p)$ of $\mathcal{D}$ in random order **do**
    $(\boldsymbol{X}', \boldsymbol{Y}') \leftarrow \textsc{SamplePatch}(\boldsymbol{X}, \boldsymbol{Y}, \pi_{\text{fg}})$
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{seg}}(f_\theta(\boldsymbol{X}', p), \boldsymbol{Y}') + \textsc{ExtraLoss}_{\mathcal{S}}(\boldsymbol{X}', p, \theta)$
    update ϕ by one SGD step on $\xi_{\mathcal{S},t} \nabla_\phi \mathcal{L}$ with learning rate $\eta_0(1 - u/U)$, momentum μ, weight decay ω; $\quad u \leftarrow u + 1$
  end for
  $\ell \leftarrow$ mean of $\mathcal{L}_{\text{seg}}$ over one sampled patch per volume of $\mathcal{D}_t^{\text{val}}$
  if $\ell < \ell^*$ **then**
    $\ell^* \leftarrow \ell$; $\quad \theta^{\text{best}} \leftarrow \theta$
  end if
end for
$\theta_t \leftarrow \theta$ {θ^{best} is stored for the additional evaluation}
return θ_t

procedure with the same settings, so that their differences do not stem from the training settings. They differ in three of its steps and in a factor on the gradient. $\textsc{Init}_{\mathcal{S}}$ sets the model with which a task starts and the parameters that are trained, $\textsc{ExtraData}_{\mathcal{S}}$ adds data to the training set of the task, and $\textsc{ExtraLoss}_{\mathcal{S}}$ adds a term to the segmentation loss. Each update multiplies the gradient by the factor $\xi_{\mathcal{S},t}$, which is one for every strategy except the prompt-tuning strategy. Section 5.3 describes these steps for each strategy. Three further differences come from the implementation. The prompt-tuning strategy keeps VoxTell in evaluation mode, so the dropout of the prompt-decoder layers (rate 0.1) is inactive, whereas it is active in all other strategies, and the LoRA strategies set the dropout on the attention weights of the cross-attention to zero. The distillation-based strategy also updates its weight ensemble during training (Section 5.3.5). The shared steps follow the pretraining of VoxTell in the patch size, the share of patches centred on the target structure, the loss, the optimiser, and the initial learning rate. We did not tune these settings, because we gave the time of the thesis to the comparison of all strategies on the full training split.

In every epoch, each training volume contributes one patch of 192^3 voxels. With probability π_{fg}, the patch is centred on a voxel drawn at random from the target structure; otherwise it is placed uniformly at random in the volume. Volumes smaller than the patch are padded with zeros. Unlike in the pretraining of VoxTell, we add no text prompts of absent structures, because a task is defined by one text prompt and its stage trains on that prompt alone. No strategy uses data augmentation or deep supervision.

The loss $\mathcal{L}_{\text{seg}}$ is the unweighted sum of the binary cross-entropy and the soft Dice loss, both computed on the foreground probabilities $\hat{y}_v$ of one patch, as in the pretraining of VoxTell (Section 3.1.3).

The trainable parameters are optimised by SGD with Nesterov momentum μ, weight decay ω, and initial learning rate η_0. The batch size is one, so that a training step fits into the memory of one GPU. VoxTell was pretrained with batches of 128 patches [Rok+26, Sec. 5]. Each of our updates therefore rests on one patch instead of 128, so at the same learning rate its gradient estimate is noisier. A task is trained for $U = E \cdot N_t$ iterations, each of which is one update of the parameters, where E is the number of epochs per task and $N_t = |\mathcal{D}|$ the number of training samples per epoch, including those added by ExtraData$_{\mathcal{S}}$. The learning rate decays linearly from η_0 to zero over these updates and is changed after every update. The optimiser state and the schedule are reset for every task. The values of all parameters are listed in Section A.2. Neither the learning rate nor the number of epochs was tuned per strategy, so our results compare the strategies under one shared training configuration, not each at its best setting.

All strategies are evaluated with the parameters after the last epoch. After every epoch, we compute $\mathcal{L}_{\text{seg}}$ on one patch per validation volume, drawn in the same way as the training patches. These patches are drawn anew in every epoch by a random-number generator that is created with the same seed at the start of every task. For all strategies except the three LoRA strategies, an additional evaluation uses the checkpoint with the lowest validation loss (Section 6.2). The LoRA strategies were evaluated only after the last epoch.

5.3 Strategies

We compare five continual-learning strategies with sequential fine-tuning, the baseline, which has no mechanism to reduce forgetting. The five were selected to cover the three families of continual learning for vision–language models that Liu et al. distinguish [Liu+26b, Sec. 4]. Parameter-efficient adaptation is represented by task-specific and shared LoRA,

Table 5.1: Steps of Algorithm 2 in which the six strategies differ. The column "Added in training" lists what a strategy adds to the training of sequential fine-tuning: extra training data ($\textsc{ExtraData}_{\mathcal{S}}$), an extra loss term ($\textsc{ExtraLoss}_{\mathcal{S}}$), or a gradient factor $\xi_{\mathcal{S},t} \neq 1$. The parentheses give the number of trained parameters. Section 5.2 gives the number of parameters that sequential fine-tuning and the distillation-based strategy can update.

Strategy	Trained parameters ϕ	Start of task t ($\textsc{Init}_{\mathcal{S}}$)	Added in training
Seq. fine-tuning	all parameters except the text encoder	parameters of task $t-1$	–
Task-specific LoRA	$\boldsymbol{B}, \boldsymbol{A}$ for $\boldsymbol{W}_q, \boldsymbol{W}_v$ of the cross-attention in all prompt-decoder layers ($4L_{\text{dec}}\, d\, r$)	pretrained model with a new adapter	–
Shared LoRA	as task-specific LoRA	adapter of task $t-1$	–
Shared LoRA + replay	as task-specific LoRA	adapter of task $t-1$	memory $\mathcal{M}_{t-1}$
Prompt tuning	tokens $\boldsymbol{V}_l$ and meta-net h_ψ in layers $l \leq L_p$ ($L_p\, m\, d + \lvert\psi\rvert$)	tokens and meta-net of task $t-1$	gradient factor $\xi_{\mathcal{S},t} = \max(0.25, \beta_t)$
Distillation-based	all parameters except the text encoder	parameters of task $t-1$	$\lambda\, \mathcal{L}_{\text{KD}}$

which use low-rank adaptation, and by the prompt-tuning strategy, whose template, CPE-CLIP, is listed by the survey among the prompt-based methods. Shared LoRA with replay represents multimodal replay. It combines replay with parameter-efficient adaptation, a kind of combination that the survey describes as common. The distillation-based strategy represents cross-modal regularisation. Its template, ZSCL, is one of the survey's examples of methods that preserve the structure of the pretrained image–text embedding space. We chose CPE-CLIP and ZSCL as templates because both were developed for continual learning of a pretrained vision–language model, and CPE-CLIP for the few-shot setting. We did not evaluate other methods, such as adapter-based or mixture-of-experts adaptation, constraints that restrict updates to orthogonal subspaces [Liu+26b, Sec. 4], or EWC [Kir+17].

Table 5.1 summarises how the six strategies differ in the steps of Algorithm 2.

5.3.1 Sequential Fine-Tuning

Sequential fine-tuning trains all parameters except those of the frozen text encoder on each task in turn, and starts each task from the parameters after the last epoch of the previous task. It adds no data and no loss term and freezes no parameters other than the text encoder, so it is standard fine-tuning of a pretrained model, repeated for every task. De Lange et al. likewise use fine-tuning that starts each task from the model of the previous task as the baseline of their comparison [DeL+22, Sec. 5]. Each of the other strategies changes at least one of the three steps of Algorithm 2, and their adaptation and forgetting are compared with those of sequential fine-tuning.

5.3.2 Task-Specific and Shared LoRA

Both LoRA strategies keep all pretrained weights frozen and train a low-rank update $\gamma_r \boldsymbol{B}\boldsymbol{A}$ (Equation (3.3)) of selected projection matrices $\boldsymbol{W}_0 \in \mathbb{R}^{d\times d}$ of the prompt decoder. We chose LoRA for both strategies because this update is separate from the frozen pretrained weights and can therefore be kept per task or shared across tasks [Hu+22, Sec. 4.1]. MedPEFT-CL likewise uses LoRA for continual learning of a 2D text-promptable medical segmentation model [Gao+26a].

The scaling factor is the rank-stabilised $\gamma_r = \alpha/\sqrt{r}$ of rsLoRA (Equation (3.4)) [Kal23], with the rank r and the constant α set per configuration. We use it instead of α/r because the shared adapter is trained with high ranks (Section 6.5), and rsLoRA showed that α/r slows learning at high ranks. The matrix $\boldsymbol{A}$ is initialised with the Kaiming-uniform scheme [He+15] instead of the Gaussian of the original LoRA [Hu+22, Sec. 4.1], and $\boldsymbol{B}$ with zeros as in the original. Dropout is applied only to the input of the low-rank path.

We apply LoRA to the query and value projections $\boldsymbol{W}_q$ and $\boldsymbol{W}_v$ of the cross-attention in all L_{dec} prompt-decoder layers (Figure 5.1). In the cross-attention, the text query attends to the image features, and through this step the text prompt steers the segmentation in VoxTell. The restriction to the query and value projections follows the finding of Hu et al. for the self-attention of GPT-3 [Hu+22, Sec. 7.1, Table 5]. Whether this finding also holds for the cross-attention of VoxTell was not tested. Our LoRA layers are adapted from the public code[1] of CLIP-LoRA [Zan+24], which applies LoRA to the attention projections of a vision–language model.

[1] https://github.com/MaxZanella/CLIP-LoRA/tree/main/loralib

The two strategies differ in how the adapters are used across tasks. Task-specific LoRA starts every task from the pretrained model with a new adapter, so each adapter is trained only on its own task and independently of all others. Each adapter is stored after its task, so T tasks give T adapters. At inference, the task identity t selects the adapter. Task-specific LoRA is thus the only strategy that uses the task identity beyond the text prompt. Shared LoRA creates one adapter before the first task and continues training it on every following task, so all tasks and the control task are evaluated with the same adapter. Their configurations also differ in rank, α, and dropout rate: task-specific LoRA uses $\alpha = 16$ and a dropout rate of 0.05, and shared LoRA uses $\alpha = 32$ and 0.1. The ranks are given with the experiments (Section 6.1.2).

At stage j of the protocol, task-specific LoRA evaluates a task $i \leq j$ with the adapter trained on task i, a task $i > j$ with the adapter of the current task j, and the control task with the pretrained model without any adapter ($\textsc{Select}_{\mathcal{S}}$ in Algorithm 1). Once a task has been trained, its entries in the evaluation matrix therefore stay at $a_{i,i}$, and the control task always gives the pretrained result. This requires knowing at inference that the text prompt of the control task belongs to none of the trained tasks. Final Forgetting takes the maximum over the stages $j = 1, \ldots, T-1$ (Equation (3.9)), including stages before task i was trained. For task-specific LoRA, it can therefore be positive only if, at such a stage, the adapter of another task gave a higher DSC on task i than the adapter of task i itself. For task-specific LoRA, a Final Forgetting near zero and a control-task drop of zero by construction are thus properties of this evaluation, not measured effects.

5.3.3 Shared LoRA with Replay

Shared LoRA with replay adds to shared LoRA a memory of training samples from earlier tasks, which are trained again together with each new task (Section 2.3). To compare shared LoRA with and without replay directly, we keep the rank $r = 1024$, α, and the dropout rate of shared LoRA. Because the six tasks share the same volumes, a memory entry consists of a volume together with the text prompt and the mask of the task it was drawn from. Keeping such entries can conflict with storage limits and privacy constraints [DeL+22, Sec. 1]. In medical imaging, privacy rules may prevent storing the data and annotations of earlier tasks [Zha+23].

Let $\mathcal{M}_t$ be the memory after task t. It starts empty, so it holds only samples of earlier tasks, and the control task stays held out. After each task t, $\max(1, \operatorname{round}(\rho\,|\mathcal{D}_t^{\mathrm{tr}}|))$ samples drawn at random from its training set are added, where $\rho = 0.1$ is the replay fraction.

The memory holds at most $M_{\max} = 200$ entries, a capacity that was not reached in our runs (Section 6.2). During task t, $\mathcal{M}_{t-1}$ is appended to the training set (ExtraData in Algorithm 2). Its entries are sampled and trained like the task's own volumes, each with the text prompt of its own task, so an epoch has $N_t = |\mathcal{D}_t^{\text{tr}}| + |\mathcal{M}_{t-1}|$ updates.

We draw the stored samples at random so that the comparison with shared LoRA shows the effect of replay itself, not that of a selection criterion. We therefore did not evaluate selection criteria such as those of Bera et al. and MedPEFT-CL (Sections 2.4.1 and 2.4.3). The replay fraction was set once and not tuned.

5.3.4 Prompt-Tuning Strategy

The prompt-tuning strategy is adapted from CPE-CLIP [DAl+23] (Section 3.2.4). It keeps every VoxTell parameter frozen and trains two components in the first L_p of the L_{dec} prompt-decoder layers (Figure 5.1). The first component is a separate set of m learned prompt tokens $\boldsymbol{V}_l \in \mathbb{R}^{m \times d}$ for each layer $l = 1, \ldots, L_p$. In layer l, these tokens are appended to the keys and values of the cross-attention with a positional encoding of zero. The tokens of layer L_p also remain in the keys and values of all later layers. We did not test removing them from the later layers. We do not place them in the encoders, because the image encoder is convolutional and has no token sequence to which they could be appended (Section 3.1.2), and the frozen text encoder is run without gradients. The tokens are initialised with the Xavier-uniform scheme [Glo+10]. We use $m = 2$ tokens per layer in $L_p = 3$ layers. These values were set once and not tuned.

The second component is a meta-net h_ψ, a two-layer perceptron with a hidden width of 2048, parameters ψ, and a Gaussian error linear unit (GELU) activation [Hen+16]. Before each of the L_p prompted layers, it adds an offset to the query,

$$\boldsymbol{q}_l \leftarrow \boldsymbol{q}_l + h_\psi\Big([\boldsymbol{q}_l; \bar{\boldsymbol{f}}]\Big), \tag{5.1}$$

where $\boldsymbol{q}_l$ is the query that enters layer l, derived from the text query $\boldsymbol{q}$ in the first layer (Section 3.1.3). The vector $\bar{\boldsymbol{f}}$ is the mean of the stage-4 image features (Figure 5.1) over all voxels after their projection to dimension d, and $[\cdot\,;\cdot]$ denotes concatenation. The meta-net is inspired by that of CoCoOp [Zho+22a], which makes the learned context vectors depend on the image (Section 3.2.4).

One set of tokens and one meta-net are trained through all tasks and used for every task at inference, so the strategy needs no task identity. The tokens change the keys and

values, and the meta-net changes the query, for every text prompt. Freezing the backbone therefore does not keep the outputs for other text prompts unchanged. The gradients of the tokens and of the meta-net in task t are multiplied by $\xi_{\mathcal{S},t} = \max(0.25, \beta_t)$ (Algorithm 2), where CPE-CLIP's scale β_t (Equation (3.6)) reduces to $1/t$ for one new structure per task and no base session. We keep this gradient scaling because, in CPE-CLIP's ablation, removing it lowered the accuracy more than removing any other component (Section 2.3). We scale the meta-net as well as the tokens, because the meta-net holds almost all of our trainable parameters. CPE-CLIP likewise scales its projection as well as its prompt tokens (Section 3.2.4). The lower bound of 0.25 is not part of CPE-CLIP and was set once and not tuned. Because only the learned prompt tokens and the gradient scaling are taken from CPE-CLIP, we name the strategy after its tokens rather than after CPE-CLIP.

5.3.5 Distillation-Based Strategy

The distillation-based strategy is adapted from ZSCL [Zhe+23] (Section 3.2.3). It trains all parameters except the text encoder, like sequential fine-tuning, and adds a distillation loss $\lambda \mathcal{L}_{\text{KD}}$ on each training patch (ExtraLoss in Algorithm 2). As in ZSCL, the teacher is a frozen copy of the pretrained model f_{θ_0}, which ZSCL found to work better than the model of the previous task [Zhe+23, Table 1d]. The teacher is not updated during the whole task sequence and receives the same patch and text prompt as the student.

The loss compares how the stage-4 guidance relates to the stage-4 image features (Figure 5.1). Let $\boldsymbol{e}_g$, $g = 1, \ldots, G$, be the G guidance vectors of $\boldsymbol{E}_4$ and $\boldsymbol{f}_v$ the encoder features of stage 4 at voxel v of the 12^3 voxels Ω_4 of that stage. For each guidance vector, the student defines a distribution over the voxels,

$$w_{g,v} = \frac{\exp\left(\langle \boldsymbol{e}_g/\|\boldsymbol{e}_g\|, \boldsymbol{f}_v \rangle\right)}{\sum_{v' \in \Omega_4} \exp\left(\langle \boldsymbol{e}_g/\|\boldsymbol{e}_g\|, \boldsymbol{f}_{v'} \rangle\right)}, \qquad \mathcal{L}_{\text{KD}} = \sum_{g=1}^{G} \sum_{v \in \Omega_4} \bar{w}_{g,v} \log \frac{\bar{w}_{g,v}}{w_{g,v}}, \tag{5.2}$$

where $\bar{w}_{g,v}$ is the same distribution computed by the teacher, so $\mathcal{L}_{\text{KD}}$ is the Kullback–Leibler divergence of the student's distributions from the teacher's. The gradient of $\mathcal{L}_{\text{KD}}$ reaches the image encoder up to stage 4, the prompt decoder, and the stage-4 guidance projection. Like ZSCL, the loss distils a similarity distribution between text-derived and image representations. The distribution runs over voxels for each guidance vector instead of over texts for each image, and ZSCL's second distribution, over images for each text, has no counterpart. ZSCL compares normalised image and text embeddings, whereas we

normalise only the guidance vectors and not the image features. We use stage 4 because the prompt decoder takes its keys and values from it, so text and image interact at this level (Figure 5.1). In the segmentation itself, $\boldsymbol{E}_4$ is fused with the decoder features of that resolution, whereas $\mathcal{L}_{\mathrm{KD}}$ pairs it with the encoder features.

The loss is computed on the patches of the current task instead of a reference dataset, so the strategy needs no data beyond the task data. In ZSCL's ablation, distilling on current-task data kept the transfer to unseen tasks at the level of a reference dataset but lowered the average and final accuracy [Zhe+23, Table 1b]. The weight λ is much smaller than ZSCL's $\lambda = 1$ [Zhe+23, App. B] and was not tuned. The weight-consolidation loss of the full ZSCL method is not used.

The strategy also maintains a weight ensemble following Equation (3.5), from which the next task is not trained. The main comparison evaluates the student, and the ensemble is evaluated after every task as an additional evaluation, whose update rule and differences from ZSCL are given in Section A.4. Because the student is trained by fine-tuning with feature distillation, we name the strategy after its distillation loss rather than after ZSCL.

5.4 Computational Complexity

Algorithm 1 calls ADAPT T times and evaluates $T + 1$ stages, each on the T task test sets and the control test set. A run therefore performs sliding-window inference on $(T + 1)\,(T\,|\mathcal{D}^{\mathrm{te}}| + |\mathcal{D}_c^{\mathrm{te}}|)$ volumes, where $|\mathcal{D}^{\mathrm{te}}|$ is the size of a task test set. For our sets, these are $7 \cdot (6 \cdot 24 + 4) = 1036$ volume inferences per run, and as many again for each additional evaluation. The evaluation time thus grows quadratically with the number of tasks, while the training time grows linearly. Evaluation needs memory for one parameter set and the prediction of one volume at a time.

Adapting to task t takes $E\,N_t$ updates and, with one validation pass after every epoch, $E\,|\mathcal{D}_t^{\mathrm{val}}|$ validation passes for every strategy, so its time is linear in the number of epochs and of training samples. For a given N_t, the strategies therefore differ only in the cost of one update. The backward pass of sequential fine-tuning and of the distillation-based strategy runs through the whole model, whereas that of the LoRA strategies and of the prompt-tuning strategy ends in the prompt decoder and does not reach the image encoder. The distillation-based strategy adds one forward pass of the teacher per update and, in our implementation, moves the teacher between CPU and GPU memory for it. Replay leaves the cost of an update unchanged and adds $|\mathcal{M}_{t-1}|$ updates per epoch through N_t.

The LoRA strategies form the low-rank update $\gamma_r \boldsymbol{B}\boldsymbol{A}$ and back-propagate through it in every update. The prompt-tuning strategy adds m keys and values in every prompt-decoder layer.

In asymptotic terms, every strategy needs $O(E\,N_t)$ forward and backward passes per task, and the distillation-based strategy $O(E\,N_t)$ teacher forward passes in addition. Forming the low-rank updates of the $2L_{\text{dec}}$ adapted projections adds $O(L_{\text{dec}}\,d^2 r)$ operations per update for the LoRA strategies. The optimiser memory is $O(|\phi|)$ for $|\phi|$ trained parameters, where $|\phi| = 4L_{\text{dec}}\,d\,r$ for the LoRA strategies and $|\phi| = L_p\,m\,d + |\psi|$ for the prompt-tuning strategy, and $|\phi|$ is the number of all parameters outside the text encoder for sequential fine-tuning and the distillation-based strategy. After T tasks, task-specific LoRA stores $O(T\,L_{\text{dec}}\,d\,r)$ adapter parameters in addition to the pretrained model.

Memory depends mainly on whether the backward pass reaches the image encoder, because only then are the activations of the image encoder stored. The optimiser keeps one momentum buffer for each trained parameter that receives a gradient, so this part of the memory grows with the number of trained parameters (Table 5.1). The distillation-based strategy also holds the weight ensemble, a further copy of all parameters, on the GPU, and the teacher during each update, and therefore needs the most memory. After T tasks, task-specific LoRA stores one adapter per task, so its storage grows linearly with the number of tasks, whereas all other strategies store one parameter set. At inference, the LoRA update is merged into $\boldsymbol{W}_0$, so the sliding-window forward passes cost about the same for all strategies. Section 6.7 reports the measured cost of the strategies on the NVIDIA A100 used in this work.

Chapter 6

Experiments and Results

This chapter reports the experiments with which we answer the research question. Section 6.1 gives the setup: the experiments, the data settings, the values of all parameters, and the metrics. The following sections present the experiments, each with its results and its conclusion. Section 6.8 states the limitations that apply across the experiments and which results may generalise.

6.1 Experimental Setup

Table 6.1 lists the experiments and the objective of Section 1.3 that each addresses. Each consists of runs of the protocol of Algorithm 1 that differ in one factor: the strategy, the LoRA rank, the number of training volumes or of updates per task, the task order, or the use of replay. The pretrained model is the reference in every experiment, and joint fine-tuning is the upper reference for the comparison of the strategies and for the amount of labelled data. A diagnostic experiment with task-specific LoRA trained on patellar cartilage alone tests why this strategy did not learn that structure in the task sequence (Section 6.3). In total, we report 38 runs. Section A.5 gives the evaluation matrices of all of them.

6.1.1 Task Orders and Few-Shot Settings

In the original task order, the structures are trained in the sequence lateral tibial cartilage, medial tibial cartilage, femoral cartilage, patellar cartilage, medial meniscus, and lateral meniscus. We chose this order at random. The second order is its reverse. Sequential fine-tuning, task-specific LoRA, and shared LoRA were trained in both orders. We trained no further orders, because each would require a complete run of every strategy.

Table 6.1: Overview of the experiments. Training volumes per task: 108 is the full training split. Objective: number of the research objective in Section 1.3. All runs use the original task order unless stated otherwise.

Experiment	Strategies and configurations	Training volumes per task	Objective	Section
Comparison of the strategies	Seq. fine-tuning; Task-specific LoRA ($r = 4$); Shared LoRA ($r = 1024$) with and without replay; Prompt tuning; Distillation-based	108	2	6.2
Upper reference: joint fine-tuning	Joint fine-tuning	5, 10, 20, 108	2, 3	6.2
Diagnosis of the patellar failure	Task-specific LoRA ($r = 4, 16$) on patellar cartilage only; rank, learning rate, seed or epochs varied	108	2	6.3
Amount of labelled data	Seq. fine-tuning; Task-specific LoRA ($r = 4$); Shared LoRA ($r = 1024$); each also with $k = 5$ and 216 epochs per task	5, 10, 20, 108	3	6.4
LoRA rank	Task-specific LoRA ($r = 4, 8$); Shared LoRA ($r = 4, 32, 128, 256, 512, 1024, 2048$)	108	4	6.5
Task order	Seq. fine-tuning; Task-specific LoRA ($r = 4$); Shared LoRA ($r = 1024$); each in original and reverse order	108	4	6.6

With $k = 5$, 10, or 20 volumes per task (Sections A.1 and 4.3) and $E = 10$ epochs, a task receives 50, 100, or 200 updates instead of 1,080 with the full training split. The equal-updates runs trained sequential fine-tuning, task-specific LoRA, and shared LoRA with $k = 5$ and $E = 216$ epochs, which gives these 1,080 updates. We ran them only for $k = 5$, the setting with the largest difference in updates. Joint fine-tuning was trained on the same k volumes, each with all six text prompts, that is, on $6k$ samples in a single stage.

6.1.2 Training and Evaluation Configuration

Section A.2 lists the settings that all runs share and the parameters of each strategy. The optimiser, the initial learning rate $\eta_0 = 10^{-4}$, and the fraction of foreground-centred patches follow the pretraining of VoxTell [Rok+26, Suppl. Sec. A]. The paper states a polynomial decay of the learning rate [Rok+26, Suppl. Sec. A] but not its exponent, the momentum, or the weight decay. For momentum and weight decay, we use the defaults of nnU-Net, on

which VoxTell builds, and for the schedule a linear decay, close to nnU-Net's polynomial decay with exponent 0.9 [Ise+26].

Each run was trained on one NVIDIA A100 GPU with 40 GB of memory on a cluster where a job may run for at most 24 hours. We chose $E = 10$ epochs per task so that most runs with the full training split fitted into one such job (12 of 17). Section A.2 gives the software versions and describes how runs that reached this limit were continued.

For the comparison of the strategies, we used rank $r = 4$ for task-specific LoRA and $r = 1024$ for shared LoRA. We chose these ranks from the test results of the LoRA rank experiment, for the reasons given in Section 6.5. The choice therefore favours the two LoRA configurations in the comparison. To show an alternative rank, we also report the run with $r = 8$ for task-specific LoRA.

6.1.3 Metrics and Statistical Analysis

We report the five metrics of Section 3.3. Adaptation, Final Forgetting, and the control-task drop are primary, because the research question asks how the strategies trade adaptation against forgetting of earlier tasks and of pretrained ability. Final Performance and Transfer are secondary. We do not report ZSCL's third metric, Avg., because it combines adaptation, retention, and transfer, which we analyse separately. Avg. is the mean over all entries of stages 1 to T [Zhe+23, Sec. 4].

For task-specific LoRA, the control-task drop is zero and Final Forgetting is near zero by construction (Section 5.3.2), so its comparison rests on Adaptation and Final Performance. For joint fine-tuning, which has no task sequence, only Final Performance and the control-task drop are defined. All metrics are computed from mean DSC fractions and reported with three decimals. Single entries of the evaluation matrices are given with two decimals, as in the figures; Section A.5 gives them with three.

Each configuration was trained once, so we cannot estimate how results vary with the training randomness or with the choice of training volumes. The only variability we can estimate is that over the test volumes, and we do so with a bootstrap [Efr+93]. In each of $N_{\text{boot}} = 2000$ draws, the 24 SKM-TEA test volumes are resampled with replacement, the same draw is used for all tasks and all stages, and the 4 control-task volumes are resampled separately. Every metric is then recomputed from the resampled means. We report the 2.5 and 97.5 percentiles of the resampled values as a 95 % interval. The figures show these intervals, whereas the tables show only the values on the full test set. In the text, we give

single-run intervals only for the patellar diagnosis (Section 6.3). With only 4 control-task volumes, the intervals of the control-task drop are coarse.

All runs are evaluated on the same test volumes, so the results of two runs are correlated, and overlapping intervals do not show that two runs perform alike. For the difference between two runs, we therefore apply the same draws to both runs and compute the interval of the difference (paired bootstrap). In the text, we give these intervals where a statement depends on them, that is, for small differences and for differences we describe as absent. All other reported differences have intervals that exclude zero by a wide margin. We interpret these intervals only for the comparisons that the experiments were designed for, which set a strategy against sequential fine-tuning or joint fine-tuning, or a run against the run that differs from it only in the factor studied (Table 6.1). Other differences, such as those between two continual-learning strategies or between LoRA ranks, are reported without intervals. The intervals are not corrected for the number of comparisons, so we use them to describe differences, not to test them, and call no difference significant.

6.2 Comparison of the Strategies

In the comparison of the strategies, we ask how the six strategies trade adaptation against forgetting when each task is trained on the full training split in the original task order, with joint fine-tuning as the upper reference. Table 6.2 gives all metrics, Figure 6.1 shows them with their bootstrap intervals, and Figure 6.2 shows the evaluation matrices from which they are computed.

In the primary metrics, the strategies formed two groups (Table 6.2 and Figure 6.1). The first group contains the two strategies that update the pretrained weights. They adapted most and forgot most. Sequential fine-tuning reached the highest Adaptation (0.781) and the highest Final Forgetting (0.782), followed by the distillation-based strategy (Adaptation: 0.747; Final Forgetting: 0.734). Both lost the control task completely (control-task drop: 0.979). Joint fine-tuning lost it as well, although it was trained without a task sequence and with the same total number of updates as sequential fine-tuning. In contrast, the three LoRA strategies adapted less within the ten epochs per task and retained much more. Their Adaptation was 0.485 (task-specific LoRA), 0.471 (shared LoRA), and 0.472 (shared LoRA with replay). With replay, the Final Forgetting of shared LoRA fell from 0.289 to 0.046 and its control-task drop from 0.237 to 0.138, the lowest values of the strategies with one parameter set. At the same time, its Adaptation changed by only 0.001 $[-0.002, 0.004]$.

Table 6.2: Results of the comparison of the strategies (full training split, original task order, one run each). Mean DSC; Adapt.: Adaptation; Forg.: Final Forgetting; Ctrl. drop: control-task drop; Final: Final Performance; Transf.: Transfer. ↑/↓: higher/lower is better. Bold: best of the six strategies in each column. †: zero or near zero by construction (Section 5.3.2), not ranked. –: not defined for joint fine-tuning.

Strategy	Adapt. ↑	Forg. ↓	Ctrl. drop ↓	Final ↑	Transf. ↑
Seq. fine-tuning	**0.781**	0.782	0.979	0.130	0.001
Task-specific LoRA ($r = 4$)	0.485	0.000†	0.000†	**0.485**	0.129
Shared LoRA ($r = 1024$)	0.471	0.289	0.237	0.233	0.174
Shared LoRA + replay ($r = 1024$)	0.472	**0.046**	**0.138**	0.450	**0.175**
Prompt tuning	0.580	0.510	0.865	0.155	0.043
Distillation-based	0.747	0.734	0.979	0.135	0.002
Joint fine-tuning (upper reference)	–	–	0.979	0.852	–
Pretrained model	0.114	0.000	0.000	0.114	0.137

The prompt-tuning strategy fell between the two groups in Adaptation (0.580) and Final Forgetting (0.510). However, its control-task drop (0.865) is close to that of the first group, although the prompt-tuning strategy keeps all pretrained weights frozen.

Within the first group, the distillation-based strategy forgot slightly less than sequential fine-tuning at the last epoch. Its Final Forgetting was 0.048 lower $[0.039, 0.055]$ at an Adaptation lower by 0.035 $[0.027, 0.041]$, its Final Performance was 0.005 higher $[0.001, 0.009]$, and its control-task drop was the same. These differences are small compared with the gap to the LoRA strategies. They hold for the configuration evaluated here, with an untuned distillation weight $\lambda = 0.01$ (Section 5.3.5). The training logs of the distillation-based strategy record only its total loss, so the contribution of the distillation term cannot be determined.

The last row of each matrix in Figure 6.2 shows when the control task was lost. For sequential fine-tuning and the distillation-based strategy, the DSC on the lung fell to 0.00 after the first task and did not recover. For the prompt-tuning strategy, it fell to 0.04 after the first task and then stayed between 0.04 and 0.23. With and without replay, shared LoRA kept the control task at 0.93 or above up to the fifth task, and most of its control-task drop arose in the last task. In this task, the DSC on the lung fell from 0.96 to 0.74 without replay and from 0.93 to 0.84 with it. With replay, this loss was smaller, although the memory holds no control-task samples. The memory supplied 55 of the 163 samples of each epoch of the last task, so the adapter was trained on a mixture of structures. Whether this

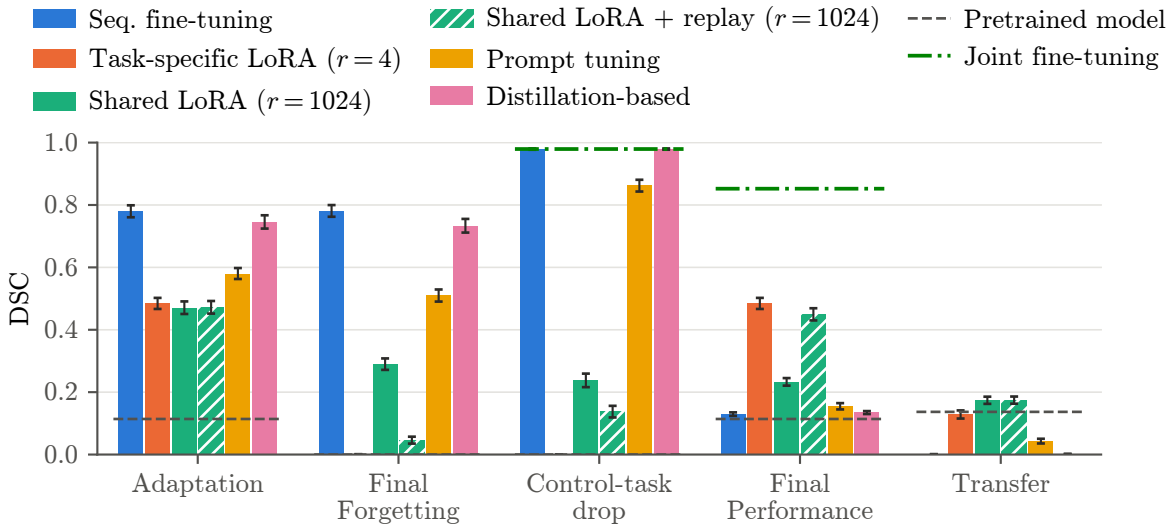


Figure 6.1: Metrics of the strategies of Table 6.2 (full training split, original task order, one run each; mean DSC), with 95 % bootstrap intervals over the test volumes (Section 6.1.3); joint fine-tuning is shown for the two metrics defined for it.

mixture explains the smaller loss on the control task was not tested. Because the effect arose in the last task, it may also depend on the task order (Section 6.6).

After the last task, sequential fine-tuning, the distillation-based strategy, and the prompt-tuning strategy segment little beyond the structure trained last. Their Final Performance (0.130, 0.135, and 0.155, respectively) is only slightly above that of the pretrained model (0.114). They also lost femoral cartilage, which the pretrained model segmented with a DSC of 0.68. Their DSC on it ended at 0.01 or less. For sequential fine-tuning and the distillation-based strategy, the loss of each task began with the training of the next task. In both, the DSC of a task at the stages after its training is close to zero. These are the entries right of the diagonal in Figure 6.2. The only one above 0.2 is the DSC on femoral cartilage after training on patellar cartilage in the distillation-based strategy (0.25). In contrast, the prompt-tuning strategy kept more of the earlier tasks, with six entries right of the diagonal above 0.2. The highest is the DSC of 0.49 on femoral cartilage after the model had been trained on patellar cartilage, the next task.

Of all strategies, task-specific LoRA reached the highest Final Performance (0.485), because each task keeps its own adapter. It reached this value although it did not learn patellar cartilage at all (Section 6.3). Shared LoRA with replay followed (0.450), and shared LoRA reached 0.233. Shared LoRA with replay still segmented every structure at the end, with a DSC between 0.38 for medial tibial cartilage and 0.70 for femoral cartilage. Joint

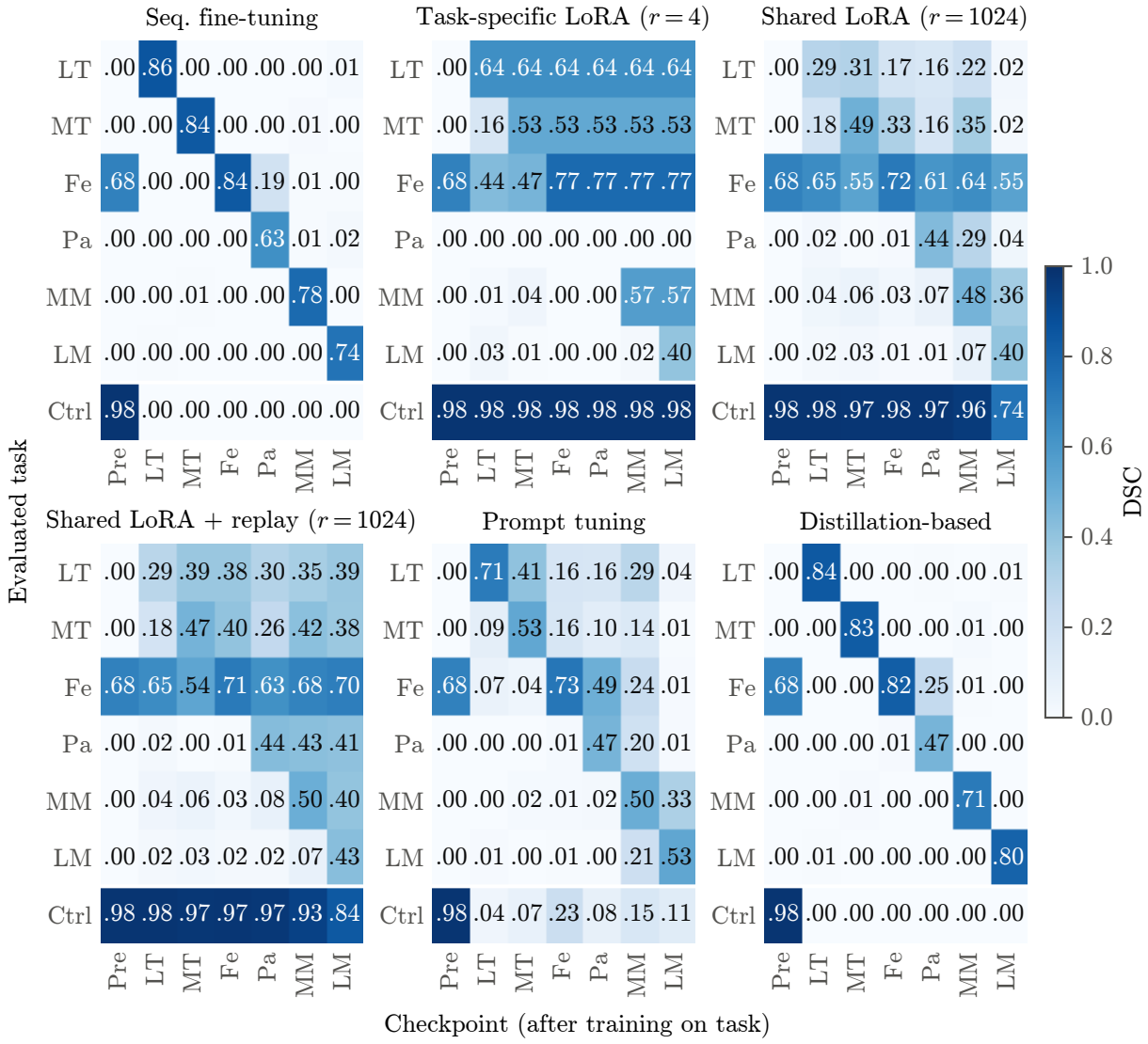


Figure 6.2: Evaluation matrices of the strategies (full training split, original task order, one run each; LoRA ranks as in Table 6.2): mean DSC of each task (rows) at each stage (columns), from the pretrained model (Pre) to the model after the last task, rounded to two decimals. LT, MT: lateral and medial tibial cartilage; Fe: femoral cartilage; Pa: patellar cartilage; MM, LM: medial and lateral meniscus; Ctrl: control task (lung, AeroPath).

fine-tuning reached a Final Performance of 0.852, with a DSC between 0.83 for patellar cartilage and 0.87 for femoral cartilage (Section A.5). This is 0.367 more than task-specific LoRA and 0.722 more than sequential fine-tuning.

In Transfer, the LoRA strategies stayed close to the pretrained model (task-specific LoRA: 0.129; shared LoRA: 0.174; shared LoRA with replay: 0.175; pretrained model: 0.137), whereas sequential fine-tuning, the distillation-based strategy, and the prompt-tuning strategy reached only 0.001, 0.002, and 0.043. Before a task was trained, these three strategies segmented almost none of it.

The distillation-based strategy was less effective in our setting than ZSCL in its own evaluation. In ZSCL, adding distillation to sequential fine-tuning raised Transfer from 44.6 to 58.9 % [Zhe+23, Table 2], whereas our distillation-based strategy reached 0.002, against 0.001 for sequential fine-tuning. Of the differences between the two methods (Section 5.3.5), distillation on current-task data is unlikely to explain the gap on its own, because in ZSCL's ablation it kept Transfer at the level of a reference dataset [Zhe+23, Table 1b]. The untuned weight $\lambda = 0.01$, the missing weight-consolidation loss, and the move from classifying images to segmenting 3D volumes remain as possible causes. We did not test which one explains the gap.

Evaluating the checkpoint with the lowest validation loss instead of the parameters after the last epoch changed the results noticeably only for sequential fine-tuning. Its Adaptation was 0.059 lower and its Final Forgetting 0.070 lower at the same Final Performance (Section A.3). At this checkpoint, sequential fine-tuning forgot less than the distillation-based strategy, by 0.021 $[0.012, 0.029]$, the reverse of their order at the last epoch. For the distillation-based strategy and the prompt-tuning strategy, the two evaluations differ by at most 0.005.

We also evaluated two components adopted from the published methods, the weight ensemble of the distillation-based strategy and the per-task gradient scaling of the prompt-tuning strategy. Neither kept the control task (Section A.4).

No strategy combined the Adaptation of sequential fine-tuning with the retention of the LoRA strategies. Freezing the pretrained weights did not keep the control task, because the learned prompt tokens and the meta-net of the prompt-tuning strategy change the output for every text prompt, including that of the control task (Section 5.3.4). Task-specific LoRA reached the highest Final Performance, but it is the only strategy that uses the task identity beyond the text prompt. Of the strategies with one parameter set, shared LoRA with replay came closest (0.450) and kept most of the control task. These results agree with

the Beyond Forgetting benchmark on 2D medical segmentation, in which replay gave the best balance between retaining earlier tasks and learning new ones, and parameter isolation avoided forgetting at the cost of a growing model [Wan+26a, Sec. 5]. In MedPEFT-CL, which adapts a 2D text-promptable model with LoRA, replay likewise reduced forgetting more than any other component [Gao+26a, Table 8]. In Lifelong nnU-Net, which trains a 3D nnU-Net on sequences of MRI datasets, the replay method forgot the least [Gon+23]. With a 3D brain-MRI model, two tasks, and 32 labelled samples per task, Chen et al. likewise found that sequential fine-tuning forgot most of the first task, whereas task-specific LoRA modules kept it [Che+26b, Table 1]. Because such modules keep earlier tasks by construction, this agreement concerns mainly the forgetting of sequential fine-tuning.

6.3 Diagnosis of the Patellar Failure

Task-specific LoRA did not learn patellar cartilage in the task sequence. Right after training on it, the DSC was 0.00 with $r = 4$ and with $r = 8$, although shared LoRA with $r = 1024$ reached 0.44 on the same task. The pretrained model does not segment patellar cartilage either (DSC 0.00). Task-specific LoRA starts every task from this model with a new adapter, whereas the shared adapter has already been trained on three tasks when it reaches patellar cartilage. To find the cause, we trained task-specific LoRA on patellar cartilage alone, first with the settings of the sequence and then with one setting changed at a time: the rank, the learning rate, the training seed, and the number of epochs. Each change tests one explanation: too little capacity, too small steps, an unlucky initialisation, or too little training. The diagnosis does not vary the starting point of the adapter. Section 6.8 discusses the possible role of this starting point. Figure 6.3 shows the training loss of all runs.

With 10 epochs, every run ended with a DSC of 0.000 on the 24 patellar test volumes (95 % bootstrap interval $[0.000, 0.000]$), whether with the settings of the sequence, a four times larger rank, a three times larger learning rate, or another seed. The run with the larger rank kept $\alpha = 16$, so its scaling factor γ_r was halved from 8 to 4. With 50 epochs, the same configuration reached 0.839 $[0.811, 0.864]$. The training loss shows why the runs differ. In every run, it stayed at about 1.0 during the first ten epochs, close to its value for an empty prediction, for which the Dice term alone is 1. Only in the 50-epoch run did it drop, from 0.97 at epoch 11 to 0.32 at epoch 12, after which it stayed at about 0.2. At epoch 22, it jumped back to about 1.0 for two epochs and then returned to about 0.2, well before epoch 50, after which the run was evaluated.

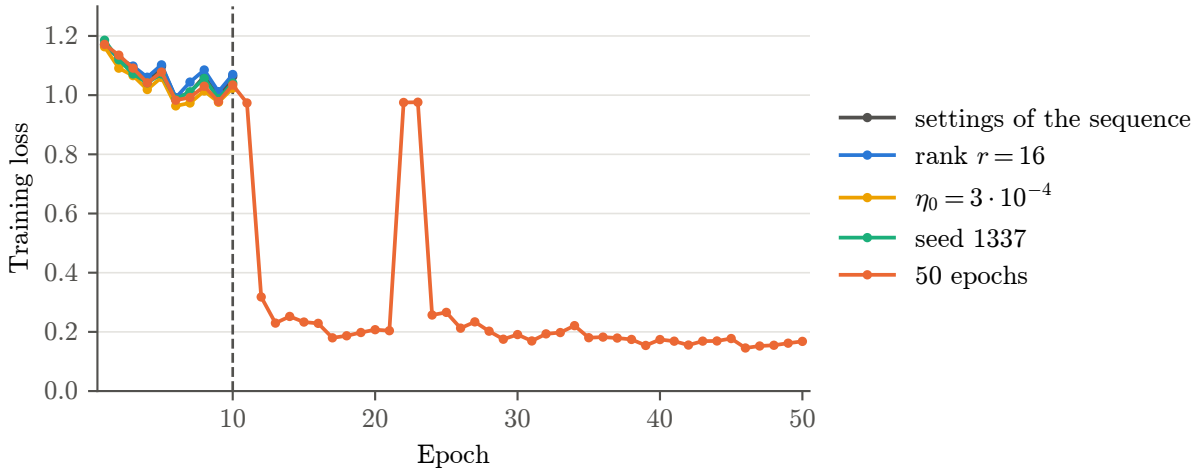


Figure 6.3: Mean training loss per epoch of task-specific LoRA trained on patellar cartilage alone, with the settings of the sequence and with one setting changed per run. Seed: training seed; the data split is the same in all runs. The four runs with 10 epochs lie almost on top of each other. Dashed line: end of training in the task sequence (epoch 10).

Ten epochs were therefore too few for task-specific LoRA to leave the empty prediction on patellar cartilage. In these runs, neither the rank, the learning rate, nor the seed was the cause, although the run with the larger rank also changed γ_r. However, the 50-epoch run differs from the others in two ways. It has five times as many updates, and its learning rate decays over these 5,400 updates. At epoch 10, its learning rate is therefore still at 80 % of η_0, while in the 10-epoch runs it has reached almost zero. The experiment does not separate these two effects. For the comparison of the strategies, the result means that the Adaptation of task-specific LoRA is limited by the training length or the learning-rate schedule of the shared configuration and not only by the strategy itself.

6.4 Amount of Labelled Data

In this experiment, we ask how the amount of labelled data affects adaptation and forgetting. We refer to it as the few-shot experiment. It uses the few-shot settings and the equal-updates runs of Section 6.1.1, with joint fine-tuning on the same k volumes as the reference. Table 6.3 gives all metrics, Figure 6.4 the main ones, and Section A.5 the evaluation matrices from which the per-task values below are taken.

Sequential fine-tuning learned every task in all three few-shot settings, and, with ten epochs per task, it forgot less with fewer volumes. With $k = 5$, its Adaptation was 0.545

Table 6.3: Results of the few-shot experiment (original task order, one run each). Vol. (updates): training volumes and updates per task, for joint fine-tuning the total number of updates of its single stage; 108 is the full training split, and 5 (1,080) are the equal-updates runs. Mean DSC; Adapt.: Adaptation; Forg.: Final Forgetting; Ctrl. drop: control-task drop; Final: Final Performance; Transf.: Transfer. ↑/↓: higher/lower is better. †: zero or near zero by construction (Section 5.3.2), not ranked. –: not defined for joint fine-tuning, which trains on all six tasks at once in a single stage.

Strategy	Vol. (updates)	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	5 (50)	0.545	0.382	0.010	0.227	0.092
Seq. fine-tuning	5 (1,080)	0.712	0.714	0.979	0.117	0.023
Seq. fine-tuning	10 (100)	0.616	0.522	0.305	0.180	0.035
Seq. fine-tuning	20 (200)	0.616	0.573	0.979	0.138	0.022
Seq. fine-tuning	108 (1,080)	0.781	0.782	0.979	0.130	0.001
Task-specific LoRA ($r = 4$)	5 (50)	0.114	0.000†	0.000†	0.114	0.137
Task-specific LoRA ($r = 4$)	5 (1,080)	0.429	0.000†	0.000†	0.429	0.059
Task-specific LoRA ($r = 4$)	10 (100)	0.115	0.000†	0.000†	0.115	0.137
Task-specific LoRA ($r = 4$)	20 (200)	0.116	0.000†	0.000†	0.116	0.138
Task-specific LoRA ($r = 4$)	108 (1,080)	0.485	0.000†	0.000†	0.485	0.129
Shared LoRA ($r = 1024$)	5 (50)	0.115	0.000	−0.001	0.115	0.137
Shared LoRA ($r = 1024$)	5 (1,080)	0.438	0.296	0.334	0.192	0.175
Shared LoRA ($r = 1024$)	10 (100)	0.116	0.000	−0.001	0.117	0.138
Shared LoRA ($r = 1024$)	20 (200)	0.140	−0.009	0.004	0.165	0.140
Shared LoRA ($r = 1024$)	108 (1,080)	0.471	0.289	0.237	0.233	0.174
Joint fine-tuning	5 (300 total)	–	–	0.733	0.600	–
Joint fine-tuning	10 (600 total)	–	–	0.894	0.585	–
Joint fine-tuning	20 (1,200 total)	–	–	0.979	0.753	–
Joint fine-tuning	108 (6,480 total)	–	–	0.979	0.852	–
Pretrained model		0.114	0.000	0.000	0.114	0.137

instead of 0.781. However, its Final Forgetting was 0.382 instead of 0.782, and it kept the control task (control-task drop: 0.010 instead of 0.979). Its Final Performance was therefore higher than with the full split (0.227 against 0.130). Sequential fine-tuning ended with a DSC of 0.42 on femoral cartilage, which the full-split run lost. With $k = 10$ and $k = 20$, Adaptation, Final Forgetting, and Final Performance lay between their values with $k = 5$ and with the full split (Table 6.3). The control task declined gradually with $k = 10$ (control-task drop: 0.305) and within two tasks with $k = 20$, almost as fast as with the full split, which lost it after the first task.

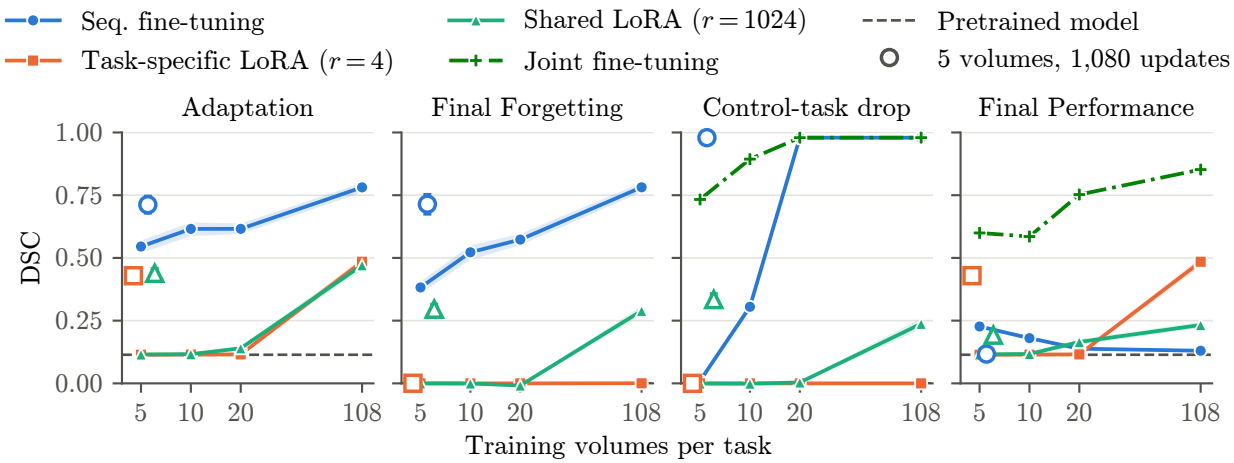


Figure 6.4: Metrics over the number of training volumes per task (logarithmic axis) for the strategies of Table 6.3 (original task order, one run each; mean DSC), with 95 % bootstrap intervals. Filled markers and solid lines: ten epochs per task; hollow markers at 5 volumes: equal-updates runs with 1,080 updates per task, shifted slightly to the side for legibility. Dash-dotted line: joint fine-tuning, shown for the two metrics defined for it. Dashed line: pretrained model.

With 5 and 10 volumes per task, neither LoRA strategy learned any task, whereas with the full training split they reached an Adaptation of 0.485 and 0.471 (Section 6.2). Their few-shot Adaptation of 0.114 to 0.116 is about that of the pretrained model (0.114). The evaluation matrices show why. Right after training, every structure except femoral cartilage had a DSC of 0.00, and femoral cartilage reached 0.69 to 0.70, close to the 0.68 of the pretrained model. Final Forgetting and the control-task drop were therefore about zero for both strategies. With 20 volumes, task-specific LoRA still did not learn any task, while shared LoRA began to, with an Adaptation of 0.140 and a Final Performance of 0.165. Its Final Forgetting was slightly negative (-0.009), because the structures it began to learn improved further while later tasks were trained.

The equal-updates runs separate the number of volumes from the number of updates at $k = 5$. For the LoRA strategies, the number of updates explains most of the failure. With 1,080 updates per task on 5 volumes, task-specific LoRA reached an Adaptation of 0.429, lower by only 0.056 $[0.040, 0.073]$ than with the full split, and shared LoRA reached 0.438, lower by only 0.032 $[0.025, 0.039]$. Task-specific LoRA again did not learn patellar cartilage (DSC 0.00 right after training), in agreement with Section 6.3, where 1,080 updates were not enough for this structure. With more updates, shared LoRA forgot as much as with the full split. Its Final Forgetting of 0.296 did not differ from that of the full split (paired

interval of the difference $[-0.007, 0.018]$), and its control-task drop of 0.334 was 0.097 higher. Sequential fine-tuning with 1,080 updates on 5 volumes reached an Adaptation of 0.712 instead of 0.545 and forgot almost as much as with the full split (Final Forgetting: 0.714; control-task drop: 0.979; Final Performance: 0.117).

Joint fine-tuning reached a Final Performance between 0.585 and 0.753 with 5 to 20 volumes per task, against 0.852 with the full split. With 5 volumes, it ended 0.373 above sequential fine-tuning with the same total number of updates. It also ended 0.170 above the strategy with the highest Final Performance in this setting, task-specific LoRA with equal updates. Its control-task drop rose with the total number of updates (Table 6.3), from 0.733 with $k = 5$ to 0.979 with $k = 20$ and with the full split.

With ten epochs per task, sequential fine-tuning was the only one of the three strategies that learned the tasks in the few-shot settings, with 50 to 200 updates per task. At $k = 5$, the fixed epoch budget explains most of the failure of the LoRA strategies, which learned the tasks with the update count of the full split. In the few-shot study of Silva-Rodríguez et al., which used a different model and different data, LoRA in the encoder learned new structures from 10 labelled volumes when trained for up to 200 epochs, although it stayed below full fine-tuning [Sil+25, Sec. 4.3.2, Table 4]. For sequential fine-tuning, the lower forgetting and the kept control task with few volumes are consistent with an effect of the number of updates rather than of the amount of data. We therefore read its higher Final Performance with 5 volumes as that of a less trained model, not as a benefit of less data. These conclusions rest on the equal-updates runs at $k = 5$. For $k = 10$ and $k = 20$, the number of volumes and the number of updates remain confounded. Each configuration was trained once, on one draw of volumes and with a learning rate that was not tuned.

6.5 LoRA Rank

In the LoRA rank experiment, we ask how the rank of the LoRA adapters affects adaptation and forgetting. Table 6.4 gives all metrics, and Figure 6.5 shows the trend for shared LoRA.

For shared LoRA, Adaptation, Final Forgetting, the control-task drop, and Final Performance changed monotonically with the rank. From $r = 4$ to $r = 2048$, Adaptation fell from 0.607 to 0.462, while Final Forgetting fell from 0.546 to 0.284 and the control-task drop from 0.978 to 0.091. Final Performance rose from 0.152 at $r = 4$ to 0.230 at $r = 512$ and hardly changed at higher ranks. Transfer rose from 0.045 at $r = 4$ to 0.170 at $r = 128$ and stayed between 0.170 and 0.175 at higher ranks. The control-task drop changed most.

Table 6.4: Results of the LoRA rank experiment (full training split, original task order, one run each). Mean DSC; Adapt.: Adaptation; Forg.: Final Forgetting; Ctrl. drop: control-task drop; Final: Final Performance; Transf.: Transfer. ↑/↓: higher/lower is better. Bold: best of the seven ranks of shared LoRA in each column. †: zero or near zero by construction (Section 5.3.2), not ranked.

Strategy	Rank r	Adapt. ↑	Forg. ↓	Ctrl. drop ↓	Final ↑	Transf. ↑
Task-specific LoRA	4	0.485	0.000†	0.000†	0.485	0.129
Task-specific LoRA	8	0.404	0.001†	0.000†	0.404	0.172
Shared LoRA	4	**0.607**	0.546	0.978	0.152	0.045
Shared LoRA	32	0.570	0.499	0.978	0.154	0.114
Shared LoRA	128	0.524	0.384	0.957	0.204	0.170
Shared LoRA	256	0.503	0.337	0.832	0.222	0.173
Shared LoRA	512	0.488	0.309	0.574	0.230	**0.175**
Shared LoRA	1024	0.471	0.289	0.237	**0.233**	0.174
Shared LoRA	2048	0.462	**0.284**	**0.091**	**0.233**	**0.175**
Pretrained model		0.114	0.000	0.000	0.114	0.137

Up to $r = 32$, the lung was no longer segmented at all after the last task, and only the two highest ranks kept most of it.

The evaluation matrices show which tasks drive these trends (Section A.5). The gain in Final Performance comes from femoral cartilage, which the pretrained model already segments with a DSC of 0.68. Femoral cartilage was lost by the end of the sequence at $r = 4$ and $r = 32$ but kept at 0.54 to 0.55 from $r = 512$ on. In contrast, lateral tibial, medial tibial, and patellar cartilage ended below 0.06 at every rank, and the two menisci between 0.35 and 0.55. Lateral tibial cartilage, the first task, accounted for the largest share of the fall in Adaptation. Its DSC right after training fell from 0.69 at $r = 4$ to 0.28 at $r = 2048$, whereas that of each other task fell by at most 0.17.

In shared LoRA, $\alpha = 32$ is the same for all ranks, so the scaling factor $\gamma_r = \alpha/\sqrt{r}$ (Section 5.3.2) falls from 16 at $r = 4$ to 0.71 at $r = 2048$ (Table A.3). The rank and the scaling factor therefore change together, so this experiment does not separate the effect of the adapter's capacity from that of its scale.

For task-specific LoRA, $r = 8$ reached a lower Adaptation than $r = 4$ (0.404 against 0.485). The difference comes from lateral tibial cartilage (0.52 against 0.64) and lateral meniscus (0.12 against 0.40), while the other tasks differ by at most 0.05. Each rank was trained once, so we cannot determine whether such differences between single tasks come from the rank or from the training randomness.

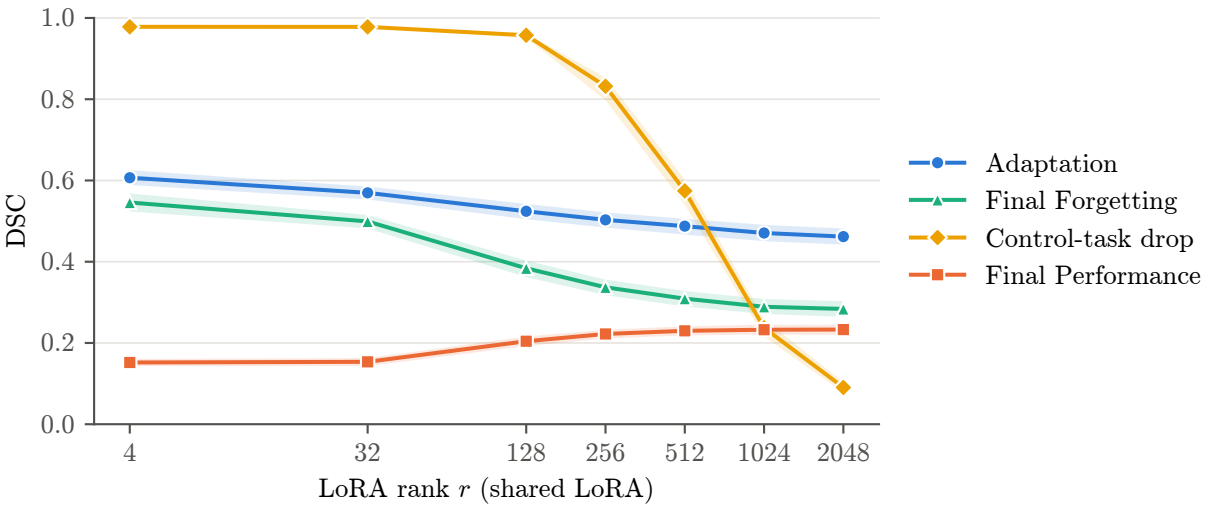


Figure 6.5: Metrics of shared LoRA over the rank r (logarithmic axis; full training split, original task order, one run per rank). Transfer is given in Table 6.4. Shaded bands: 95 % bootstrap intervals over the test volumes.

For shared LoRA with fixed α, the rank sets a trade-off between adaptation and retention. Higher ranks forget less, both of the trained tasks and of the control task, at a cost in Adaptation. The fall in Adaptation contrasts with rsLoRA, whose scaling factor we use and which improved with increasing rank when fine-tuning a large language model [Kal23, Sec. 4]. We did not test which of the differences in model, task, and training configuration explains the opposite trend. Why higher ranks retain more remains open. For the comparison of the strategies, we use $r = 1024$ for shared LoRA. It has half the trainable parameters of $r = 2048$ and the same Final Performance. Its Adaptation and Final Forgetting differ by less than 0.01. However, the control-task drop still falls from 0.237 to 0.091 between the two ranks, so the comparison of the strategies understates how much of the control task shared LoRA can keep. For task-specific LoRA, doubling the rank brought no gain in this run, so we use $r = 4$.

6.6 Task Order

In the task-order experiment, we ask whether the results depend on the order in which the structures are learned, for the three strategies trained in both task orders (Section 6.1.1). Table 6.5 gives all metrics, Figure 6.6 the main ones, and Section A.5 the evaluation matrices from which the per-task values below are taken.

Table 6.5: Results of the task-order experiment (full training split, one run each). The pretrained-model reference is computed for each task order. Mean DSC; Adapt.: Adaptation; Forg.: Final Forgetting; Ctrl. drop: control-task drop; Final: Final Performance; Transf.: Transfer. ↑/↓: higher/lower is better. †: zero or near zero by construction (Section 5.3.2), not ranked.

Strategy	Task order	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	Original	0.781	0.782	0.979	0.130	0.001
Seq. fine-tuning	Reverse	0.784	0.780	0.979	0.134	0.014
Task-specific LoRA ($r = 4$)	Original	0.485	$0.000^{\dagger}$	$0.000^{\dagger}$	0.485	0.129
Task-specific LoRA ($r = 4$)	Reverse	0.478	$0.002^{\dagger}$	$0.000^{\dagger}$	0.478	0.132
Shared LoRA ($r = 1024$)	Original	0.471	0.289	0.237	0.233	0.174
Shared LoRA ($r = 1024$)	Reverse	0.439	0.124	0.067	0.345	0.169
Pretrained model	Original	0.114	0.000	0.000	0.114	0.137
Pretrained model	Reverse	0.114	0.000	0.000	0.114	0.137

For sequential fine-tuning, no effect of the task order can be detected. In the reverse order, Adaptation was 0.003 higher $[-0.005, 0.010]$, Final Forgetting 0.002 lower $[-0.005, 0.009]$, and Final Performance 0.004 higher $[-0.002, 0.010]$. In both orders, sequential fine-tuning forgot almost all earlier tasks (Final Forgetting 0.782 and 0.780) and lost the control task after the first task (DSC 0.00 in both). Only Transfer differed, 0.014 instead of 0.001 ($+0.013$, $[0.012, 0.014]$). This difference is small and comes from the order itself: in the reverse order, femoral cartilage is trained after patellar cartilage and reached a DSC of 0.19 before its own task.

For task-specific LoRA, the task order changes neither the data nor the settings of a task. Every task starts from the pretrained model with a new adapter (Section 5.3.2), and the random seed is reset at the start of every task. Only the random initialisation of each adapter differs between the two orders, because the adapter is created before the seed is reset. The two runs therefore give one example of how a single run varies with its initialisation. Four of the six tasks reached the same DSC within 0.001. Lateral tibial cartilage reached 0.64 in the original and 0.65 in the reverse order, and lateral meniscus 0.40 and 0.34, so that Adaptation differs by 0.006 (0.485 against 0.478). The paired bootstrap interval of this difference, $[0.004, 0.008]$, does not contain zero, although the difference comes only from the initialisation. This illustrates the limit stated in Section 6.1.3: an interval that excludes zero shows that two trained models differ, not that the factor varied

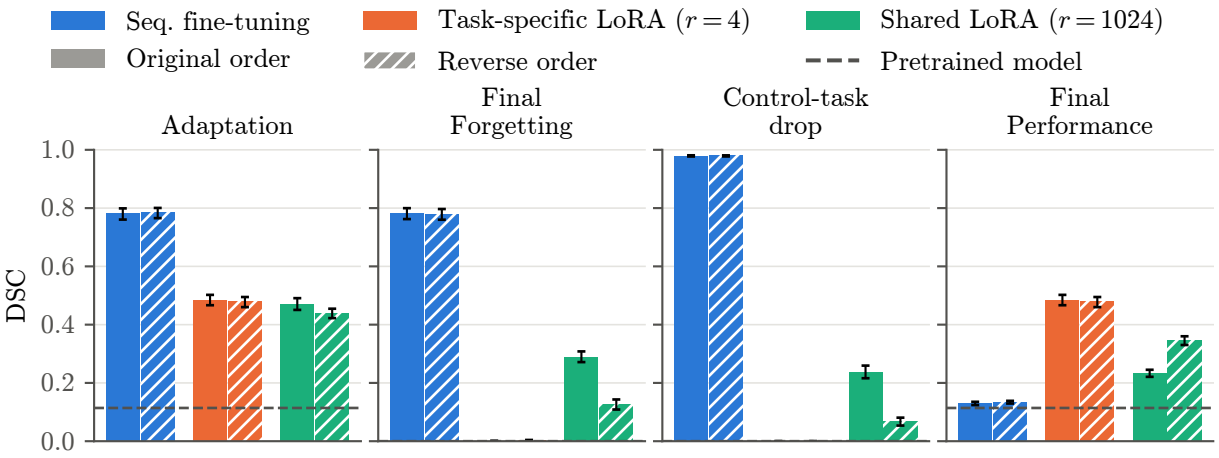


Figure 6.6: Metrics in the original (plain bars) and the reverse task order (hatched bars), full training split, one run each, with 95 % bootstrap intervals. Dashed line: pretrained model.

between them caused the difference. Differences of this size between single runs can arise from training randomness alone.

For shared LoRA, the reverse order led to better retention. Final Forgetting was 0.124 instead of 0.289, and the control-task drop 0.067 instead of 0.237. Final Performance rose from 0.233 to 0.345, while Adaptation was 0.032 lower $[0.022, 0.042]$ (0.439 against 0.471). At the end of the reverse sequence, every structure was still segmented to some degree, from 0.13 for lateral meniscus to 0.51 for lateral tibial cartilage, whereas in the original order three of the six structures ended below 0.05. Transfer was about the same in both orders (0.174 in the original and 0.169 in the reverse order).

For shared LoRA in the original order, most of the control-task drop and of the forgetting arose in the last step. Training on lateral meniscus lowered the DSC on the control task from 0.96 to 0.74 (Section 6.2). In the same step, lateral tibial, medial tibial, and patellar cartilage fell from between 0.22 and 0.35 to 0.04 or less. In the reverse order, the last task is lateral tibial cartilage. Training on it lowered the DSC on the control task only from 0.97 to 0.91, and that on femoral cartilage, which lost most in this step, from 0.65 to 0.50. Whether the loss in the original order comes from lateral meniscus itself or from the tasks trained before it cannot be separated with two orders.

In both orders, shared LoRA learned the first task poorly. Lateral tibial cartilage, the first task of the original order, reached 0.29 right after training, but 0.51 when it was trained last in the reverse order. Lateral meniscus, the first task of the reverse order,

reached 0.01, but 0.40 when it was trained last in the original order. In contrast, sequential fine-tuning learned the first task well in both orders, lateral tibial cartilage with 0.86 and lateral meniscus with 0.83.

The training loss of shared LoRA with $r = 1024$ (Section A.6) fell only slowly from its value for an empty prediction on the first task in both orders, but started much lower on the second task, whereas with $r = 4$ it fell quickly on the first task.

For sequential fine-tuning, the task order changed only Transfer, by 0.013, and the strategy forgot almost everything in both orders. For task-specific LoRA, the task order has no effect beyond the variation between two runs, as its design implies. For shared LoRA, the task order has a strong effect. In the reverse order, the adapter retained more of the earlier tasks and of the control task, at a lower Adaptation. In the original order, no run of a strategy with one parameter set trained only on current-task data exceeded a Final Performance of 0.233, at any rank or number of training volumes (Sections 6.2, 6.4 and 6.5). In the reverse order, shared LoRA reached 0.345, above this value. With two orders, we cannot determine which property of the order causes the difference in retention. The ranking of the three strategies was the same in both orders for every metric. Over ten orders of 2D prostate segmentation tasks, the Beyond Forgetting benchmark likewise found that the task order affects some strategies more than others [Wan+26a, Sec. 4.4]. A ranking obtained from one or two orders may therefore not hold in others. Section 6.8 discusses two possible explanations for the poorly learned first task and connects them with the LoRA rank, patellar, and few-shot experiments.

6.7 Computational Cost

Table 6.6 reports the measured cost of the strategies in the comparison. Task-specific LoRA, shared LoRA, and the prompt-tuning strategy needed the least time per epoch and the least memory, sequential fine-tuning more, and the distillation-based strategy the most, in the order expected from Section 5.4. Shared LoRA with replay needed as little memory as shared LoRA, but its replayed samples add updates to every epoch, so its time per epoch was close to that of sequential fine-tuning. All strategies fit on a single 40 GB GPU and took about the same time per volume at inference.

Table 6.6: Measured cost of the strategies in the comparison (full training split, original task order, one run per strategy, one NVIDIA A100 40 GB). Trainable param.: parameters set as trainable (Section 5.2). Epoch: mean time of one training epoch, including validation. Evaluation: median time per test volume, including data loading and text encoding. Peak GPU mem.: maximum over training and evaluation.

Strategy	Trainable param. (M)	Epoch (min)	Evaluation (s/volume)	Peak GPU mem. (GiB)
Seq. fine-tuning	440.03	26.8	8.8	31.4
Task-specific LoRA ($r = 4$)	0.20	18.1	7.9	20.3
Shared LoRA ($r = 1024$)	50.33	19.8	8.5	20.6
Shared LoRA + replay ($r = 1024$)	50.33	26.4	8.6	20.5
Prompt tuning	12.60	18.9	8.0	20.3
Distillation-based	440.03	37.1	8.7	37.9

6.8 Limitations and Generalisation

Three limitations affect the conclusions most: the hyperparameters shared by all strategies, the single run of each configuration, and the narrow scope of the study. The remaining limitations concern the strategies we did not evaluate, the evaluation protocol, the setup of the runs, and the data. The section ends with the parts of this work that may hold beyond the setting studied here.

First, all strategies were trained with one set of hyperparameters, and neither the learning rate nor the number of epochs was tuned per strategy. Our comparison therefore measures the strategies under one budget, not each at its best setting (Section 5.2). Several experiments also change a second factor together with the one they study. The few-shot settings reduce the number of updates together with the number of volumes (Section 6.4). In the LoRA rank experiment and in the patellar run with the larger rank, the scaling factor γ_r changes together with the rank (Sections 6.3 and 6.5). The 50-epoch patellar run decays the learning rate more slowly, together with the larger number of updates. In these experiments, a difference cannot be attributed to the studied factor alone, except for volumes and updates at $k = 5$, which the equal-updates runs separate for three strategies.

Within the ten epochs per task used for all strategies, shared LoRA with $r = 1024$ learned little on its first task in both task orders (Section 6.6). Task-specific LoRA learned nothing on patellar cartilage at any rank tested (Section 6.3), and neither LoRA strategy learned any task with 5 or 10 volumes (Section 6.4). We see two possible explanations, but none of our runs tests either of them. The first is the zero initialisation of $\boldsymbol{B}$, which

leaves the prediction of the model unchanged at the start of training (Section 5.3.2). The second is a training setup that was not tuned for LoRA. In this setup, the adapters were trained with SGD and the learning rate of full fine-tuning. Hu et al. state only for Adam that "tuning α is roughly the same as tuning the learning rate if we scale the initialization appropriately" [Hu+22, Sec. 4.1]. In our shared LoRA, $\gamma_r = \alpha/\sqrt{r}$ falls as the rank rises (Table A.3), so with SGD the change of the adapter's output per update becomes smaller at high ranks. The low Adaptation of the LoRA strategies may therefore reflect our training setup as much as the strategies themselves. Initialising the adapters from an adapter trained on another structure would test the first explanation, and training them with Adam or varying α at a fixed rank would test the second.

Second, each configuration was trained once, on one random split of each dataset and, in the few-shot settings, on one draw of training volumes. Our bootstrap intervals therefore cover only the choice of test volumes (Section 6.1.3). As the two task-specific LoRA runs of the task-order experiment show, small differences between single runs can come from training randomness alone, even when their interval excludes zero (Section 6.6). The effects of the rank and the task order on shared LoRA are many times larger than the difference between the two task-specific LoRA runs, but we did not repeat any configuration of shared LoRA, so its variation between runs is unknown. Our bootstrap intervals are also not adjusted for multiple comparisons.

Third, the scope of the study is narrow. We adapted one model, VoxTell, to six structures of one knee MRI dataset in two task orders, without data augmentation and with one fixed text prompt per structure for both training and evaluation (Section 4.1). Our evaluation therefore cannot show whether the adapted models remain as robust to rephrased text prompts as the pretrained VoxTell (Section 2.1). Each stage also trained on a single text prompt, whereas VoxTell was pretrained with several text prompts per image, including text prompts of absent structures (Section 5.2). Whether this contributed to the forgetting was not tested. We gave priority to the comparison of all six strategies on the full training split and ran the distillation-based and the prompt-tuning strategies only in this setting and in the original task order. The few-shot part of the research question is therefore answered only for sequential fine-tuning, task-specific LoRA, and shared LoRA (Table 6.1). Shared LoRA with replay was likewise run only with the full training split, in the original task order, and at $r = 1024$. The effect of replay in other configurations is unknown, because the task order and the rank strongly changed how shared LoRA balanced adaptation and

retention (Sections 6.5 and 6.6). The same uncertainty applies to the smaller control-task drop with replay, which arose in the last task and whose cause we did not test (Section 6.2).

Our results also cover only the strategies we evaluated, in the form we implemented them. The distillation-based and prompt-tuning strategies are adaptations of ZSCL and CPE-CLIP (Sections 5.3.4 and 5.3.5), so their results do not describe the original methods. Our distillation teacher is the pretrained model, as in ZSCL, and we did not test other teachers, such as the model of the previous task used by PRIMED (Section 2.3). We also did not evaluate sharing adapters between tasks with similar text prompts, as MedCRP-CL does by clustering the tasks by the embeddings of their text prompts [Gao26], or the other methods named in Section 5.3.

The evaluation protocol has two further limits. The control task measures the loss of pretrained ability with a single large, high-contrast structure, the lung in CT, so it cannot show how the strategies affect the many other structures and modalities that VoxTell was trained on. With only four test volumes, its DSC is also less certain than that of the SKM-TEA tasks, which are evaluated on 24 volumes. The origin of its reference labels is not documented. Because AeroPath is among the training sources of checkpoint v1.1, the control task also measures the loss on data the model was trained on, not on unseen data (Section 4.2). The second limit is the joint reference. It covers all six tasks only, so the intransigence measure of Chaudhry et al. cannot be computed (Section 5.1).

Three further threats to internal validity come from how the runs were set up and evaluated. The ranks of the two LoRA strategies in the comparison were chosen from the test results of the LoRA rank experiment (Section 6.1.2), so the results favour these two configurations. The two LoRA strategies were also run with different values of α and of the dropout rate (Section 5.3.2), so a difference between them cannot be attributed to the sharing of the adapter alone. The choice of the evaluated checkpoint can also reverse close comparisons, as it did for the Final Forgetting of the distillation-based strategy and sequential fine-tuning (Section 6.2).

Finally, the data and the checkpoint limit how far our numbers carry over to other evaluations. We used our own random split of SKM-TEA instead of the official one [Des+21, Sec. 4.2], so our numbers cannot be compared with published SKM-TEA results. Our NIfTI files of SKM-TEA store no orientation, so the volumes reach the model in their stored axis order, which may differ from that of the knee volumes in VoxTell's training data and may lower the results of the pretrained model (Section 4.4). The model also sees only the first qDESS echo, although the annotators relied on both echoes at the boundary

between cartilage and meniscus [Des+21, App. A.4]. On our test volumes, the pretrained model segmented femoral cartilage with a DSC of 0.68 but none of the other five structures (Section 6.2), whereas the VoxTell authors report 46.45 % for femoral cartilage and at most 14.78 % for the others with checkpoint v1.0 [Rok+26, Suppl. Table 9; Hug26]. We used checkpoint v1.1, which differs from v1.0 in its training data. Its added sources include "Stanford Knee" [Rok26], a name the VoxTell paper uses for SKM-TEA [Rok+26, Sec. 6], so our test volumes may have been part of its training data, although the pretrained model did not segment five of the six structures. In that case, the pretrained DSC on femoral cartilage, which enters the pretrained-model reference and the Transfer of every strategy, would partly reflect training on these volumes.

The numerical results and the ranking of the strategies apply only to the setting studied here. The evaluation protocol (Section 5.1) depends less on this setting. It requires only a task sequence, a held-out control task that the pretrained model segments well, and the pretrained model and joint fine-tuning as references. It can therefore be applied to other free-text promptable models, where it would show which of our observations depend on VoxTell, and to other task sequences.

Chapter 7

Summary

Universal segmentation models such as VoxTell [Rok+26] segment many anatomical structures from a free-text prompt. On knee MRI, which its training data covered only rarely, VoxTell still segments most structures poorly [Rok+26, Suppl. Table 9]. This thesis studied what happens when such a model is adapted to new structures one after another, with few labelled volumes per structure. We asked how well it learns each new structure, how much it forgets of the earlier ones, and how much of its pretrained ability it keeps. We adapted VoxTell to six structures of the SKM-TEA knee MRI dataset [Des+21], four cartilages and two menisci. On these structures, the pretrained model reached a mean DSC of 0.114. The lung in the chest CT volumes of AeroPath [Stø+24] served as a control task, which was not trained on during the adaptation and measured the loss of pretrained ability.

We compared sequential fine-tuning, the baseline, with five continual-learning strategies that build on existing methods: task-specific and shared LoRA [Hu+22], shared LoRA with replay [DeL+22, Sec. 3.1], a distillation-based strategy adapted from ZSCL [Zhe+23], and a prompt-tuning strategy adapted from CPE-CLIP [DAl+23]. Our own work is the evaluation protocol, the adaptation of these methods to VoxTell, and all experiments: the comparison of the strategies with the diagnosis of the failure of task-specific LoRA on patellar cartilage, and the analyses of the amount of labelled data, the task order, and the LoRA rank (Section 1.5). The protocol combines existing metrics with a held-out control task, following SPU's control set [Zha+24], and a pretrained-model reference.

In answer to the research question, no strategy adapted as well as sequential fine-tuning while retaining as much as the LoRA strategies. With the full training split of 108 volumes per task, sequential fine-tuning and the distillation-based and prompt-tuning strategies adapted more than the LoRA strategies, but forgot most of what they had learned on the

earlier tasks and lost most of the control task (Section 6.2). Sequential fine-tuning and the distillation-based strategy ended at a Final Performance of 0.130 and 0.135, respectively, only slightly above that of the pretrained model (0.114). This was because at the end they segmented almost only the structure trained last. In contrast, the LoRA strategies kept more of the earlier tasks and much more of the control task, but, within ten epochs per task, adapted much less than sequential fine-tuning (Adaptation: at most 0.485 against 0.781). In the few-shot settings, with 5 to 20 volumes per task, we ran only sequential fine-tuning, task-specific LoRA, and shared LoRA. Within ten epochs per task, only sequential fine-tuning learned the tasks (Section 6.4).

With the full training split and in the original task order, only two of the LoRA strategies kept most of what they had learned on the earlier tasks, each at a cost (Section 6.2). Shared LoRA with replay lowered the Final Forgetting of shared LoRA from 0.289 to 0.046 at the same Adaptation, but stores patient data from earlier tasks and was run in one configuration only. Task-specific LoRA ($r = 4$) kept its Final Forgetting near zero by construction, but needs the task identity at inference to select its adapter. As the upper reference, joint fine-tuning was trained on all six tasks at once. It reached a Final Performance of 0.852, against 0.485 for task-specific LoRA and 0.450 for shared LoRA with replay. However, it lost the control task as completely as sequential fine-tuning.

With 5 volumes and as many updates as on the full training split, task-specific and shared LoRA reached an Adaptation of 0.429 and 0.438, respectively, slightly below those with the full training split, while sequential fine-tuning again forgot most of what it had learned (Section 6.4). At $k = 5$, these results are consistent with the number of updates, rather than the number of volumes, being the main reason why the LoRA strategies hardly adapted with few volumes. Runs with as many updates as on the full training split exist only for 5 volumes, so for 10 and 20 volumes the two effects remain confounded. Even with the full training split, ten epochs were too few for task-specific LoRA to learn patellar cartilage, for every rank, learning rate, and seed we tested (Section 6.3); the run with the larger rank also halved the LoRA scaling factor γ_r. The same configuration reached a DSC of 0.839 with 50 epochs. In that run, the learning rate also decayed more slowly, so the two effects are not separated.

The task order and the rank mainly affected shared LoRA (Sections 6.5 and 6.6). In the reverse task order, shared LoRA forgot less of the earlier tasks and lost less of the control task (Final Forgetting: 0.124 instead of 0.289), but at a lower Adaptation. For the three strategies run in both orders, the order did not change their ranking in any metric. Higher

ranks of shared LoRA reduced forgetting but also Adaptation. However, the scaling factor γ_r changed with the rank, so the experiment does not show whether the rank or γ_r caused this.

The control task showed an effect that Final Forgetting alone would have missed (Section 6.5). For shared LoRA, raising the rank from 256 to 2048 lowered the control-task drop from 0.832 to 0.091, whereas Final Forgetting fell only from 0.337 to 0.284. The loss of pretrained ability and the forgetting of earlier tasks therefore need not change together. Our control task follows the control set of SPU [Zha+24], and comparable measures exist for segmentation models prompted with points or boxes [Wan+26b; Wan+24], but none of the works closest to ours uses one (Section 2.5).

Several limitations should be considered (Section 6.8). All strategies were trained with one configuration of ten epochs per task, and each configuration was trained once. Small differences between runs can therefore come from training randomness alone. The study covers one model, one dataset, one fixed text prompt per structure, and a control task with a single structure.

The main extension of this work is a strategy that adapts as well as sequential fine-tuning while retaining as much as the LoRA strategies. Our results do not show whether the strategies themselves or the shared training configuration prevented this, because the LoRA adapters learned slowly within ten epochs per task and we did not test why (Section 6.8). Tuning the learning rate, the number of epochs, and the LoRA ranks for each strategy on the validation split would show whether one of them achieves this at its best setting. If none does, candidates are MoE-Adapters, which selects LoRA experts from the input [Yu+24], and nested learning, which updates parts of one model at different frequencies [Beh+25]. In the works we found, MoE-Adapters was evaluated only on 2D images [Yu+24; Gao26], and nested learning only with language models [Beh+25, Sec. 4].

For shared LoRA with replay, runs in other task orders and at other ranks would show whether its benefit holds beyond the one configuration we ran. Selecting the stored samples instead of drawing them at random could make replay more effective at the same memory size, as it did for Bera et al. [Ber+23, Table 1]. C^{2}GR replays generated samples instead of stored ones, but trains its generative model on the data of each task [Li+26, Sec. 3.3]. Replaying such samples would avoid storing the volumes of earlier tasks.

Equal-updates runs with 10 and 20 volumes per task and few-shot runs of the distillation-based and prompt-tuning strategies would extend the few-shot part of the comparison. Repeated runs with several seeds and draws of training volumes would quantify the variation

between runs. Rephrased text prompts, more task orders, other datasets, and other free-text promptable models would show how far our results hold beyond the setting studied here. A control task with several structures would measure the loss of pretrained ability beyond the lung. With VoxTell checkpoint v1.0, which was not trained on AeroPath, the control task would measure this loss on unseen data (Section 4.2).

Appendix A

Additional Experimental Details and Results

This appendix complements Chapter 6. Section A.1 lists the volumes of the few-shot training sets and of the test sets, Section A.2 the settings and parameters of all runs, Section A.3 the additional evaluations, and Section A.4 analyses the effect of the weight ensemble and the gradient scaling. Section A.5 gives, for every reported run, the evaluation matrix from which its metrics after the last epoch are computed. Section A.6 shows the training loss of the first two tasks of shared LoRA.

A.1 Training and Test Volumes

This section belongs to Sections 4.3 and 6.1.1. Table A.1 lists the SKM-TEA volumes of the few-shot training sets and the test volumes of both datasets.

Table A.1: Few-shot training volumes and test volumes, named as in the dataset releases: SKM-TEA volumes by their scan identifier, AeroPath volumes by their case number. The full training split contains 108 SKM-TEA volumes.

Set	Volumes
SKM-TEA training, $k = 5$	MTR_038, MTR_074, MTR_097, MTR_201, MTR_225
SKM-TEA training, $k = 10$	the 5 volumes above and MTR_118, MTR_154, MTR_188, MTR_207, MTR_219
SKM-TEA training, $k = 20$	the 10 volumes above and MTR_001, MTR_037, MTR_056, MTR_120, MTR_156, MTR_167, MTR_179, MTR_209, MTR_224, MTR_243
SKM-TEA test (24 volumes)	MTR_005, MTR_015, MTR_016, MTR_018, MTR_045, MTR_046, MTR_049, MTR_052, MTR_066, MTR_079, MTR_098, MTR_104, MTR_105, MTR_107, MTR_110, MTR_115, MTR_129, MTR_130, MTR_190, MTR_191, MTR_199, MTR_215, MTR_227, MTR_248
AeroPath test (4 volumes)	1, 3, 6, 12

A.2 Training and Strategy Parameters

This section belongs to Section 6.1.2. Table A.2 lists the training and evaluation settings that all runs share, and Table A.3 the parameters of each strategy.

The runs used PyTorch 2.8.0 with CUDA 12.6 and cuDNN 9.10.2, MONAI 1.5.2, nnU-Net 2.7.0, Transformers 4.57.6, NumPy 2.4.3, and NiBabel 5.4.2. These versions were read from the environment after training, not logged per run.

Runs that reached the 24-hour limit of a job were continued from their last checkpoint. In five runs of the comparison of the strategies and the few-shot experiment, training was continued inside a task. From that point on, these runs drew training samples and patches in a different order, so they cannot be reproduced exactly from their configuration.

Table A.2: Training and evaluation settings shared by all runs (Sections 5.1 and 5.2).

Setting	Symbol	Value
Optimiser		SGD, Nesterov
Initial learning rate	η_0	10^{-4}
Momentum	μ	0.99
Weight decay	ω	$3 \cdot 10^{-5}$
Learning-rate schedule		linear (polynomial, exponent 1), per iteration, restarted for every task
Epochs per task	E	10; 216 in the equal-updates runs (validation every 24 epochs)
Batch size		1
Patch size (voxels)		$192 \times 192 \times 192$
Foreground-centred patches	π_{fg}	0.85
Loss	$\mathcal{L}_{\text{seg}}$	binary cross-entropy + soft Dice (smoothing 10^{-5}), weights 1:1
Deep supervision		off
Dropout in the prompt decoder		0.1; inactive in the prompt-tuning strategy, 0 on the attention weights in the LoRA strategies
Data augmentation		none
Random seed (training, split)		42, 42
Sliding-window step		0.5 of the patch size
Gaussian weighting		standard deviation 1/8 of the patch size
Probability threshold		0.5

Table A.3: Parameters of the strategies (Section 5.3). Sequential fine-tuning has no parameters of its own. Bold: rank used in the comparison of the strategies. The scaling factor $\gamma_r = \alpha/\sqrt{r}$ is listed in the order of the ranks. The ensemble interval applies only to the weight ensemble, which is reported as an additional evaluation (Section A.3). The prompt-tuning strategy without gradient scaling uses the same parameters with $\xi_{\mathcal{S},t} = 1$.

Strategy	Parameter	Symbol	Value
Task-specific LoRA	Rank	r	**4**, 8
	LoRA scaling hyperparameter	α	16
	Scaling factor	γ_r	8, 5.66
	Dropout (low-rank path)		0.05
	Adapted projections		$\boldsymbol{W}_q, \boldsymbol{W}_v$, cross-attention
Shared LoRA	Rank	r	4, 32, 128, 256, 512, **1024**, 2048
	LoRA scaling hyperparameter	α	32
	Scaling factor	γ_r	16, 5.66, 2.83, 2, 1.41, 1, 0.71
	Dropout (low-rank path)		0.1
	Adapted projections		$\boldsymbol{W}_q, \boldsymbol{W}_v$, cross-attention
Shared LoRA + replay	Rank (otherwise as shared LoRA)	r	1024
	Replay fraction	ρ	0.1
	Memory capacity	$M_{\max}$	200
Prompt tuning	Learned prompt tokens per layer	m	2
	Prompted decoder layers	L_p	3
	Gradient-scale lower bound		0.25
Distillation-based	Distillation weight	λ	0.01
	Ensemble interval (iterations)	I_{we}	10

A.3 Additional Evaluations

This section belongs to Section 6.2. Section A.4 uses the metrics of this section as well. Table A.4 gives the metrics of the additional evaluations, and Table 6.2 gives the metrics of the same runs with the parameters after the last epoch.

A.4 Weight Ensemble and Gradient Scaling

This section belongs to Section 6.2. It compares the distillation-based strategy with and without its weight ensemble, and the prompt-tuning strategy with and without its per-task gradient scaling. The weight ensemble is evaluated in the same run as the student, whereas gradient scaling is compared with a separate run without it. Their metrics are given in Section A.3, and Table A.5 gives the evaluation matrix of the weight ensemble.

The weight ensemble follows Equation (3.5) and is updated every I_{we} training updates, counted anew in each epoch. Our interval $I_{\text{we}} = 10$ is smaller than ZSCL's interval of 100. ZSCL reports only slight changes between intervals of 1, 10, and 100 [Zhe+23, App. B]. The ensemble is never reset, so it is the equally weighted mean of θ_0 and all snapshots taken so far. Each task starts training from the student, not from the ensemble. In the public code of ZSCL[1], by contrast, the ensemble restarts at every task and the next task is trained from it.

The weight ensemble of the distillation-based strategy reached half the Adaptation of the student, 0.376 against 0.747, a Final Performance lower by 0.011 $[0.007, 0.017]$, and the same control-task drop (0.979 for both). Its Final Forgetting was 0.354, against 0.734 for the student. It is lower mainly because the ensemble reached a lower DSC on each task right after training on it. On the diagonal of its evaluation matrix (Table A.5), the DSC fell from 0.83 on the first task to 0.22 on the fourth and to 0.00 on the last two.

[1] https://github.com/Thunderbeee/ZSCL, file `mtil/src/models/finetune.py`, commit 8cd0caf.

Table A.4: Additional evaluations (full training split, original task order, one run each). Best validation: checkpoint with the lowest validation loss (Section 5.2). Weight ensemble: the weight ensemble of the distillation-based strategy (Section 5.3.5). No scaling: without the per-task gradient scaling (Section 5.3.4). Mean DSC; Adapt.: Adaptation; Forg.: Final Forgetting; Ctrl. drop: control-task drop; Final: Final Performance; Transf.: Transfer. ↑/↓: higher/lower is better.

Strategy	Evaluation	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	best validation	0.723	0.712	0.979	0.130	0.001
Prompt tuning	best validation	0.576	0.505	0.863	0.155	0.039
Prompt tuning (no scaling)	best validation	0.608	0.568	0.979	0.135	0.043
Distillation-based	best validation	0.745	0.732	0.979	0.135	0.002
Distillation-based	weight ensemble	0.376	0.354	0.979	0.124	0.007
Pretrained model		0.114	0.000	0.000	0.114	0.137

Table A.5: Evaluation matrix of the weight ensemble of the distillation-based strategy (full training split, original task order); run `exp_20261022_171452_65011743`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.828**	0.002	0.003	0.001	0.001	0.000
MT	0.000	0.001	**0.690**	0.300	0.095	0.033	0.010
Fe	0.684	0.002	0.009	**0.514**	0.615	0.415	0.286
Pa	0.000	0.000	0.000	0.069	**0.219**	0.372	0.446
MM	0.000	0.000	0.003	0.006	0.005	**0.002**	0.000
LM	0.000	0.009	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.002	0.000	0.000	0.000	0.000	0.000

The per-task gradient scaling of the prompt-tuning strategy lowered its forgetting at a small cost in Adaptation. Both runs are compared at the checkpoint with the lowest validation loss, because the run without scaling was evaluated only there. With scaling, Final Forgetting was 0.063 lower, the control-task drop 0.117 lower, and Final Performance 0.020 higher $[0.014, 0.025]$, at an Adaptation lower by 0.032 $[0.025, 0.040]$. Without scaling, the DSC on the control task fell to 0.02 after the second task and to 0.00 after the last. With scaling, it stayed between 0.04 and 0.23 from the first task on, far below the 0.98 of the pretrained model.

Both components differ from their published form. Our weight ensemble differs from that of ZSCL in its update interval, in not being restarted at each task, and in being built from a student distilled without a reference dataset (Section 5.3.5 and above). Our gradient scaling differs from that of CPE-CLIP in its lower bound of 0.25 (Section 5.3.4). Each component was tested in one run and in the original task order, so the results of this section hold for our configuration only.

A.5 Evaluation Matrices of All Runs

This section belongs to Chapter 6. For every reported run, one table gives the mean DSC on the test volumes of each task (rows, in training order) after each stage (columns): *Pre* is the pretrained model, and the following columns are the stages after training on the task named in the header. The diagonal entries are set in bold. They are measured directly after training on the task of their row. The last row is the control task. Task abbreviations: LT lateral tibial cartilage, MT medial tibial cartilage, Fe femoral cartilage, Pa patellar cartilage, MM medial meniscus, LM lateral meniscus, Ctrl control task (lung in AeroPath). The runs are grouped by the experiment in which they first appear in Table 6.1, so a run that belongs to several experiments is listed once. The prompt-tuning run without gradient scaling (Section A.4) is not listed in Table 6.1 and is listed last here. Joint fine-tuning has a single stage, *Joint*, after which all six structures are evaluated. All matrices are computed with the parameters after the last epoch, except that of the prompt-tuning run without gradient scaling, which was evaluated only at the checkpoint with the lowest validation loss. The run identifier in each caption names the folder of the run in the archived results, which holds its configuration, logs, and evaluation matrix.

Comparison of the strategies

Table A.6: Evaluation matrix of sequential fine-tuning (full training split, original task order); run `exp_20261022_171452_a9f4ca56`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.856**	0.000	0.002	0.000	0.000	0.014
MT	0.000	0.000	**0.843**	0.002	0.000	0.010	0.000
Fe	0.684	0.001	0.001	**0.841**	0.192	0.006	0.004
Pa	0.000	0.000	0.000	0.003	**0.632**	0.006	0.019
MM	0.000	0.000	0.007	0.001	0.000	**0.775**	0.002
LM	0.000	0.005	0.000	0.001	0.000	0.001	**0.740**
Ctrl	0.979	0.001	0.000	0.000	0.000	0.000	0.000

Table A.7: Evaluation matrix of task-specific LoRA ($r = 4$, full training split, original task order); run `exp_20260816_155157_668e608d`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.635**	0.635	0.635	0.635	0.635	0.635
MT	0.000	0.163	**0.527**	0.527	0.527	0.527	0.527
Fe	0.684	0.440	0.470	**0.774**	0.774	0.774	0.774
Pa	0.000	0.002	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.012	0.042	0.000	0.000	**0.572**	0.572
LM	0.000	0.033	0.006	0.000	0.000	0.020	**0.401**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.8: Evaluation matrix of shared LoRA ($r = 1024$, full training split, original task order); run `exp_20260926_095539_90e8b4e6`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.294**	0.311	0.174	0.156	0.216	0.025
MT	0.000	0.185	**0.490**	0.326	0.165	0.355	0.018
Fe	0.684	0.648	0.550	**0.719**	0.609	0.644	0.555
Pa	0.000	0.024	0.004	0.008	**0.443**	0.291	0.039
MM	0.000	0.037	0.061	0.030	0.066	**0.477**	0.357
LM	0.000	0.020	0.026	0.008	0.011	0.071	**0.402**
Ctrl	0.979	0.980	0.975	0.976	0.974	0.955	0.742

Table A.9: Evaluation matrix of shared LoRA ($r = 1024$, full training split, original task order, replay $\rho = 0.1$); run `exp_20261023_014438_55e6abcf`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.287**	0.386	0.383	0.305	0.347	0.385
MT	0.000	0.183	**0.473**	0.405	0.261	0.421	0.379
Fe	0.684	0.647	0.539	**0.713**	0.628	0.678	0.699
Pa	0.000	0.023	0.004	0.006	**0.435**	0.425	0.412
MM	0.000	0.041	0.062	0.031	0.078	**0.497**	0.400
LM	0.000	0.020	0.030	0.020	0.021	0.075	**0.427**
Ctrl	0.979	0.980	0.970	0.975	0.968	0.931	0.842

Table A.10: Evaluation matrix of the prompt-tuning strategy (full training split, original task order); run `exp_20261022_180923_7bcbb6cc`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.711**	0.407	0.165	0.162	0.286	0.043
MT	0.000	0.095	**0.528**	0.160	0.104	0.137	0.005
Fe	0.684	0.065	0.043	**0.730**	0.492	0.235	0.010
Pa	0.000	0.003	0.000	0.012	**0.472**	0.203	0.010
MM	0.000	0.003	0.016	0.014	0.018	**0.503**	0.327
LM	0.000	0.010	0.002	0.006	0.004	0.212	**0.534**
Ctrl	0.979	0.041	0.071	0.226	0.078	0.155	0.115

Table A.11: Evaluation matrix of the distillation-based strategy (full training split, original task order); run `exp_20261022_171452_65011743`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.841**	0.000	0.001	0.005	0.000	0.007
MT	0.000	0.000	**0.831**	0.003	0.003	0.013	0.000
Fe	0.684	0.001	0.001	**0.823**	0.255	0.005	0.002
Pa	0.000	0.000	0.000	0.008	**0.472**	0.000	0.000
MM	0.000	0.000	0.011	0.001	0.000	**0.713**	0.000
LM	0.000	0.006	0.000	0.001	0.000	0.000	**0.801**
Ctrl	0.979	0.001	0.000	0.003	0.002	0.000	0.000

Upper reference: joint fine-tuning

Table A.12: Evaluation matrix of joint fine-tuning (5 volumes per structure); run `exp_20261024_002524_abc243ff`.

Task	Pre	Joint
LT	0.000	0.631
MT	0.000	0.546
Fe	0.684	0.698
Pa	0.000	0.461
MM	0.000	0.600
LM	0.000	0.661
Ctrl	0.979	0.246

Table A.13: Evaluation matrix of joint fine-tuning (10 volumes per structure); run `exp_20261024_014447_afb34098`.

Task	Pre	Joint
LT	0.000	0.550
MT	0.000	0.470
Fe	0.684	0.775
Pa	0.000	0.498
MM	0.000	0.582
LM	0.000	0.637
Ctrl	0.979	0.086

Table A.14: Evaluation matrix of joint fine-tuning (20 volumes per structure); run `exp_20261024_051245_f9067927`.

Task	Pre	Joint
LT	0.000	0.810
MT	0.000	0.771
Fe	0.684	0.782
Pa	0.000	0.601
MM	0.000	0.779
LM	0.000	0.773
Ctrl	0.979	0.000

Table A.15: Evaluation matrix of joint fine-tuning (full training split); run `exp_20261022_171655_6a6e9cb5`.

Task	Pre	Joint
LT	0.000	0.872
MT	0.000	0.847
Fe	0.684	0.873
Pa	0.000	0.834
MM	0.000	0.842
LM	0.000	0.845
Ctrl	0.979	0.000

Diagnosis of the patellar failure

Table A.16: Evaluation matrix of task-specific LoRA ($r = 4$, patellar cartilage only, settings of the sequence); run `exp_20260824_040750_012f67c8`.

Task	Pre	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.17: Evaluation matrix of task-specific LoRA ($r = 16$, patellar cartilage only); run `exp_20260828_041835_89e6de63`.

Task	Pre	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.18: Evaluation matrix of task-specific LoRA ($r = 4$, patellar cartilage only, learning rate $3 \cdot 10^{-4}$); run `exp_20260828_045913_2e306fe6`.

Task	Pre	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.19: Evaluation matrix of task-specific LoRA ($r = 4$, patellar cartilage only, seed 1337); run `exp_20260901_110801_d8b133c2`.

Task	Pre	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.20: Evaluation matrix of task-specific LoRA ($r = 4$, patellar cartilage only, 50 epochs); run `exp_20260905_220334_6697f1c2`.

Task	Pre	Pa
Pa	0.000	**0.839**
Ctrl	0.979	0.979

Amount of labelled data

Table A.21: Evaluation matrix of sequential fine-tuning (5 volumes per task, original task order); run `exp_20261023_212327_a67fb8c9`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.620**	0.513	0.197	0.371	0.129	0.013
MT	0.000	0.000	**0.487**	0.163	0.127	0.185	0.007
Fe	0.684	0.465	0.292	**0.713**	0.366	0.472	0.421
Pa	0.000	0.000	0.000	0.000	**0.338**	0.023	0.036
MM	0.000	0.000	0.013	0.022	0.003	**0.593**	0.362
LM	0.000	0.075	0.015	0.133	0.005	0.118	**0.521**
Ctrl	0.979	0.964	0.961	0.967	0.974	0.970	0.969

Table A.22: Evaluation matrix of sequential fine-tuning (5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_195739_6044ad8d`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.761**	0.014	0.001	0.000	0.000	0.004
MT	0.000	0.083	**0.673**	0.001	0.000	0.007	0.000
Fe	0.684	0.000	0.006	**0.797**	0.079	0.002	0.002
Pa	0.000	0.000	0.000	0.002	**0.756**	0.000	0.000
MM	0.000	0.000	0.007	0.002	0.000	**0.676**	0.086
LM	0.000	0.003	0.000	0.001	0.000	0.122	**0.610**
Ctrl	0.979	0.005	0.001	0.003	0.000	0.013	0.000

Table A.23: Evaluation matrix of sequential fine-tuning (10 volumes per task, original task order); run `exp_20261023_234659_f041f82f`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.777**	0.405	0.004	0.066	0.036	0.033
MT	0.000	0.002	**0.534**	0.010	0.013	0.017	0.003
Fe	0.684	0.111	0.091	**0.743**	0.241	0.014	0.009
Pa	0.000	0.000	0.000	0.008	**0.361**	0.012	0.003
MM	0.000	0.004	0.025	0.014	0.001	**0.596**	0.351
LM	0.000	0.018	0.001	0.022	0.001	0.243	**0.683**
Ctrl	0.979	0.908	0.772	0.691	0.655	0.435	0.674

Table A.24: Evaluation matrix of sequential fine-tuning (20 volumes per task, original task order); run `exp_20261024_001927_861cfa24`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.675**	0.190	0.002	0.025	0.002	0.008
MT	0.000	0.006	**0.689**	0.003	0.001	0.003	0.000
Fe	0.684	0.032	0.011	**0.793**	0.233	0.005	0.004
Pa	0.000	0.000	0.000	0.007	**0.410**	0.001	0.006
MM	0.000	0.000	0.017	0.001	0.000	**0.496**	0.180
LM	0.000	0.002	0.001	0.001	0.000	0.367	**0.632**
Ctrl	0.979	0.587	0.005	0.033	0.000	0.000	0.000

Table A.25: Evaluation matrix of task-specific LoRA ($r = 4$, 5 volumes per task, original task order); run `exp_20260926_120018_3ab58c90`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.685	0.685	**0.685**	0.685	0.685	0.685
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.26: Evaluation matrix of task-specific LoRA ($r = 4$, 5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_195940_ea9221f2`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.601**	0.601	0.601	0.601	0.601	0.601
MT	0.000	0.121	**0.528**	0.528	0.528	0.528	0.528
Fe	0.684	0.089	0.230	**0.751**	0.751	0.751	0.751
Pa	0.000	0.002	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.011	0.033	0.001	0.000	**0.466**	0.466
LM	0.000	0.001	0.000	0.000	0.000	0.017	**0.229**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.27: Evaluation matrix of task-specific LoRA ($r = 4$, 10 volumes per task, original task order); run `exp_20260926_184314_43a15eeb`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.686	0.685	**0.688**	0.688	0.688	0.688
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.28: Evaluation matrix of task-specific LoRA ($r = 4$, 20 volumes per task, original task order); run `exp_20260926_105629_ba181db6`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.689	0.688	**0.693**	0.693	0.693	0.693
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.29: Evaluation matrix of shared LoRA ($r = 1024$, 5 volumes per task, original task order); run `exp_20260930_164451_55aada20`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.686	0.688	**0.690**	0.690	0.691	0.692
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.980	0.980	0.980	0.980	0.980	0.980

Table A.30: Evaluation matrix of shared LoRA ($r = 1024$, 5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_200332_9840ab20`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.245**	0.172	0.060	0.130	0.036	0.010
MT	0.000	0.171	**0.428**	0.260	0.151	0.089	0.008
Fe	0.684	0.653	0.595	**0.721**	0.635	0.656	0.384
Pa	0.000	0.038	0.007	0.006	**0.453**	0.310	0.031
MM	0.000	0.035	0.055	0.020	0.087	**0.438**	0.374
LM	0.000	0.016	0.018	0.002	0.011	0.035	**0.346**
Ctrl	0.979	0.979	0.971	0.977	0.978	0.951	0.645

Table A.31: Evaluation matrix of shared LoRA ($r = 1024$, 10 volumes per task, original task order); run `exp_20260930_165220_b0c6c426`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.001	0.001	0.001	0.001	0.002
MT	0.000	0.000	**0.000**	0.000	0.000	0.001	0.002
Fe	0.684	0.690	0.692	**0.698**	0.698	0.699	0.699
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.001
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.980	0.980	0.980	0.980	0.981	0.981

Table A.32: Evaluation matrix of shared LoRA ($r = 1024$, 20 volumes per task, original task order); run `exp_20260930_163605_91573278`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.002**	0.005	0.005	0.012	0.020	0.028
MT	0.000	0.000	**0.016**	0.016	0.039	0.092	0.117
Fe	0.684	0.695	0.699	**0.705**	0.701	0.692	0.683
Pa	0.000	0.000	0.002	0.002	**0.067**	0.081	0.087
MM	0.000	0.000	0.001	0.001	0.003	**0.047**	0.073
LM	0.000	0.000	0.000	0.000	0.000	0.001	**0.003**
Ctrl	0.979	0.980	0.980	0.981	0.980	0.978	0.976

LoRA rank

Table A.33: Evaluation matrix of task-specific LoRA ($r = 8$, full training split, original task order); run `exp_20260822_110320_530ca92a`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.515**	0.515	0.515	0.515	0.515	0.515
MT	0.000	0.273	**0.516**	0.516	0.516	0.516	0.516
Fe	0.684	0.601	0.488	**0.748**	0.748	0.748	0.748
Pa	0.000	0.006	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.021	0.077	0.000	0.000	**0.526**	0.526
LM	0.000	0.040	0.008	0.000	0.000	0.033	**0.118**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.34: Evaluation matrix of shared LoRA ($r = 4$, full training split, original task order); run `exp_20260924_202757_88afb75d`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.693**	0.380	0.022	0.051	0.022	0.005
MT	0.000	0.104	**0.520**	0.065	0.004	0.053	0.001
Fe	0.684	0.053	0.096	**0.783**	0.547	0.069	0.004
Pa	0.000	0.000	0.000	0.031	**0.510**	0.005	0.000
MM	0.000	0.002	0.011	0.039	0.012	**0.581**	0.348
LM	0.000	0.011	0.001	0.021	0.001	0.062	**0.554**
Ctrl	0.979	0.004	0.001	0.330	0.014	0.226	0.001

Table A.35: Evaluation matrix of shared LoRA ($r = 32$, full training split, original task order); run `exp_20260926_095635_1dd1adb4`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.650**	0.427	0.140	0.100	0.195	0.010
MT	0.000	0.175	**0.515**	0.158	0.038	0.201	0.004
Fe	0.684	0.458	0.183	**0.751**	0.504	0.439	0.018
Pa	0.000	0.003	0.000	0.014	**0.492**	0.179	0.016
MM	0.000	0.024	0.025	0.082	0.029	**0.517**	0.381
LM	0.000	0.031	0.002	0.012	0.003	0.103	**0.493**
Ctrl	0.979	0.708	0.193	0.880	0.106	0.080	0.001

Table A.36: Evaluation matrix of shared LoRA ($r = 128$, full training split, original task order); run `exp_20260816_124851_bc388139`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.453**	0.316	0.102	0.111	0.242	0.015
MT	0.000	0.241	**0.531**	0.164	0.058	0.289	0.010
Fe	0.684	0.633	0.417	**0.732**	0.630	0.639	0.358
Pa	0.000	0.014	0.001	0.008	**0.484**	0.230	0.024
MM	0.000	0.034	0.044	0.071	0.034	**0.490**	0.363
LM	0.000	0.029	0.009	0.002	0.004	0.101	**0.455**
Ctrl	0.979	0.969	0.791	0.937	0.672	0.434	0.022

Table A.37: Evaluation matrix of shared LoRA ($r = 256$, full training split, original task order); run `exp_20260817_172554_da49178a`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.361**	0.309	0.142	0.121	0.247	0.014
MT	0.000	0.223	**0.522**	0.223	0.085	0.343	0.012
Fe	0.684	0.646	0.468	**0.726**	0.634	0.664	0.478
Pa	0.000	0.023	0.002	0.008	**0.478**	0.243	0.014
MM	0.000	0.042	0.056	0.053	0.045	**0.489**	0.373
LM	0.000	0.026	0.014	0.004	0.005	0.089	**0.443**
Ctrl	0.979	0.979	0.945	0.968	0.918	0.772	0.148

Table A.38: Evaluation matrix of shared LoRA ($r = 512$, full training split, original task order); run `exp_20260924_163047_f4aa9c98`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.310**	0.290	0.137	0.143	0.192	0.020
MT	0.000	0.201	**0.511**	0.264	0.145	0.321	0.014
Fe	0.684	0.647	0.516	**0.721**	0.619	0.640	0.537
Pa	0.000	0.029	0.003	0.010	**0.461**	0.251	0.020
MM	0.000	0.042	0.058	0.038	0.061	**0.494**	0.361
LM	0.000	0.022	0.023	0.006	0.010	0.074	**0.427**
Ctrl	0.979	0.980	0.965	0.973	0.965	0.874	0.405

Table A.39: Evaluation matrix of shared LoRA ($r = 2048$, full training split, original task order); run `exp_20260927_213503_efa7a48c`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.280**	0.326	0.208	0.185	0.230	0.038
MT	0.000	0.180	**0.473**	0.342	0.208	0.346	0.023
Fe	0.684	0.648	0.571	**0.720**	0.630	0.648	0.550
Pa	0.000	0.026	0.003	0.005	**0.447**	0.326	0.047
MM	0.000	0.040	0.047	0.023	0.081	**0.464**	0.352
LM	0.000	0.019	0.025	0.008	0.014	0.068	**0.388**
Ctrl	0.979	0.980	0.977	0.978	0.978	0.968	0.889

Task order

Table A.40: Evaluation matrix of sequential fine-tuning (full training split, reverse task order); run `exp_20261022_171520_2dba6ff1`.

Task	Pre	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.830**	0.000	0.000	0.001	0.000	0.004
MM	0.000	0.000	**0.826**	0.000	0.001	0.011	0.000
Pa	0.000	0.000	0.000	**0.650**	0.001	0.000	0.000
Fe	0.684	0.001	0.002	0.190	**0.822**	0.001	0.004
MT	0.000	0.000	0.006	0.000	0.002	**0.780**	0.001
LT	0.000	0.006	0.000	0.000	0.002	0.002	**0.796**
Ctrl	0.979	0.000	0.000	0.000	0.001	0.000	0.000

Table A.41: Evaluation matrix of task-specific LoRA ($r = 4$, full training split, reverse task order); run `exp_20260822_123645_5b18bc03`.

Task	Pre	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.343**	0.343	0.343	0.343	0.343	0.343
MM	0.000	0.185	**0.572**	0.572	0.572	0.572	0.572
Pa	0.000	0.008	0.002	**0.000**	0.000	0.000	0.000
Fe	0.684	0.695	0.539	0.000	**0.774**	0.774	0.774
MT	0.000	0.068	0.080	0.000	0.000	**0.527**	0.527
LT	0.000	0.041	0.001	0.000	0.000	0.078	**0.654**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.42: Evaluation matrix of shared LoRA ($r = 1024$, full training split, reverse task order); run `exp_20260930_101141_5d80eb69`.

Task	Pre	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.014**	0.073	0.072	0.065	0.073	0.127
MM	0.000	0.038	**0.469**	0.303	0.330	0.411	0.328
Pa	0.000	0.001	0.008	**0.440**	0.406	0.212	0.176
Fe	0.684	0.693	0.670	0.609	**0.722**	0.646	0.501
MT	0.000	0.018	0.188	0.106	0.061	**0.477**	0.427
LT	0.000	0.017	0.012	0.090	0.035	0.118	**0.510**
Ctrl	0.979	0.972	0.978	0.975	0.977	0.967	0.912

Gradient scaling of the prompt-tuning strategy

Table A.43: Evaluation matrix of the prompt-tuning strategy (full training split, original task order, gradient scaling inactive, best validation checkpoint); run `exp_20260816_161201_b39ea60e`.

Task	Pre	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.707**	0.364	0.109	0.055	0.164	0.017
MT	0.000	0.102	**0.549**	0.119	0.025	0.090	0.001
Fe	0.684	0.088	0.033	**0.777**	0.408	0.006	0.001
Pa	0.000	0.002	0.000	0.015	**0.434**	0.033	0.002
MM	0.000	0.008	0.019	0.012	0.007	**0.555**	0.162
LM	0.000	0.009	0.002	0.005	0.001	0.166	**0.629**
Ctrl	0.979	0.108	0.019	0.060	0.001	0.002	0.000

A.6 Training Loss of Shared LoRA

This section belongs to Section 6.6. Figure A.1 shows the training loss per epoch of the first two tasks of shared LoRA.

With $r = 1024$, the loss of the first task started at about 1.0, its value for an empty prediction, and fell slowly, to 0.69 after ten epochs in the original order and only to 0.96 in the reverse order. On the second task, it started much lower, at 0.81 in the original and 0.86 in the reverse order in the first epoch, and fell to 0.56 in both orders, although in the reverse order the first task was hardly learned at all. With $r = 4$ in the original order, by contrast, the loss of the first task fell to 0.74 within three epochs and to 0.38 after ten, in line with the higher DSC of the first task at low ranks (Section 6.5).

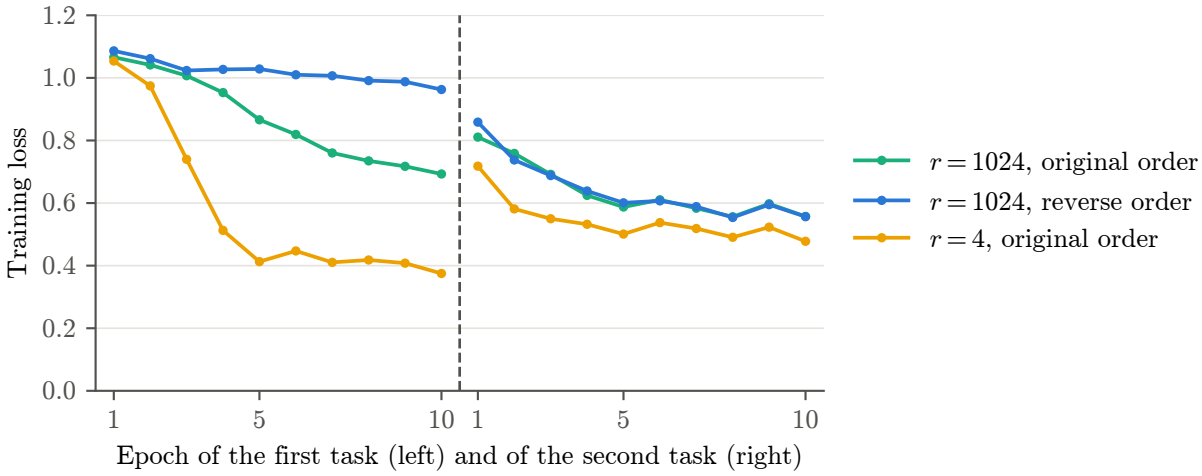


Figure A.1: Training loss $\mathcal{L}_{\text{seg}}$ of shared LoRA in the first two tasks, averaged over each epoch, with $r = 1024$ in both task orders and $r = 4$ in the original order (full training split, one run each). The first two tasks are lateral and medial tibial cartilage in the original order and lateral and medial meniscus in the reverse order. Dashed line: start of the second task.

List of Abbreviations

CT computed tomography

CTA computed tomography angiography

DICOM Digital Imaging and Communications in Medicine

DSC Dice similarity coefficient

EWC elastic weight consolidation

GELU Gaussian error linear unit

HDF5 Hierarchical Data Format version 5

IoU intersection over union

LoRA low-rank adaptation

MLP multilayer perceptron

MRI magnetic resonance imaging

NIfTI Neuroimaging Informatics Technology Initiative

PET positron emission tomography

qDESS quantitative double-echo steady-state

SAM Segment Anything Model

SGD stochastic gradient descent

List of Figures

1.1 Setting of this thesis 3

3.1 Architecture of VoxTell 26
3.2 Entries of the evaluation matrix used by each metric 31

4.1 Example volumes with their labels 37

5.1 VoxTell as used in this thesis and the parts changed by the strategies . . . 45

6.1 Metrics of the strategies 60
6.2 Evaluation matrices of the strategies 61
6.3 Training loss of the patellar diagnosis runs 64
6.4 Metrics over the number of training volumes 66
6.5 Metrics of shared LoRA over the rank 69
6.6 Metrics in the original and the reverse task order 71

A.1 Training loss of the first two tasks of shared LoRA 102

List of Tables

2.1 Setting of this thesis compared with the closest works 19
2.2 Summary of the reviewed works 20

4.1 Datasets used in this thesis 36

5.1 Overview of the strategies 48

6.1 Overview of the experiments 56
6.2 Results of the comparison of the strategies 59
6.3 Results of the few-shot experiment 65
6.4 Results of the LoRA rank experiment 68
6.5 Results of the task-order experiment 70
6.6 Measured cost of the strategies 73

A.1 Few-shot training volumes and test volumes 82
A.2 Training and evaluation settings shared by all runs 83
A.3 Parameters of the strategies 84
A.4 Additional evaluations 86

Bibliography

[Baw+18] C. Baweja, B. Glocker, and K. Kamnitsas. Towards continual learning in medical imaging, 2018. arXiv: 1811.02496 [cs.CV]. URL: https://arxiv.org/abs/1811.02496 (cited on p. 15).

[Beh+25] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni. Nested Learning: the illusion of deep learning architectures. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems* (November 30–December 7, 2025), volume 38, pages 46968–47002, San Diego, CA, USA and Mexico City, Mexico. Curran Associates, Inc., 2025. DOI: 10.52202/085713-1565. URL: https://proceedings.neurips.cc/paper_files/paper/2025/file/4309616aaed8e848009bc4a7ef73b493-Paper-Conference.pdf (cited on p. 79).

[Ber+23] S. Bera, V. Ummadi, D. Sen, S. Mandal, and P. K. Biswas. Memory replay for continual medical image segmentation through atypical sample selection. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part IV* (October 8–12, 2023), volume 14223 of *Lecture Notes in Computer Science*, pages 513–522, Cham. Springer Nature Switzerland, 2023. ISBN: 978-3-031-43901-8. DOI: 10.1007/978-3-031-43901-8_49 (cited on pp. 15, 21, 79).

[Bom+21] R. Bommasani et al. On the opportunities and risks of foundation models, version 3, 2021. arXiv: 2108.07258 [cs.LG]. URL: https://arxiv.org/abs/2108.07258. Center for Research on Foundation Models (CRFM), Stanford University (cited on p. 14).

[But+23] V. I. Butoi, J. J. Gonzalez Ortiz, T. Ma, M. R. Sabuncu, J. Guttag, and A. V. Dalca. UniverSeg: universal medical image segmentation. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 21381–21394, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCV51070.2023.01960 (cited on p. 11).

[Cer+20] F. Cermelli, M. Mancini, S. Rota Bulò, E. Ricci, and B. Caputo. Modeling the background for incremental learning in semantic segmentation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 13–19, 2020), pages 9230–9239, Seattle, WA, USA. IEEE, June 2020. DOI: 10.1109/CVPR42600.2020.00925 (cited on pp. 14, 15).

[Cha+18] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr. Riemannian walk for incremental learning: understanding forgetting and intransigence. In V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, editors, *Computer Vision – ECCV 2018. 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part XI* (September 8–14, 2018), volume 11215 of *Lecture Notes in Computer Science*, pages 556–572, Cham. Springer International Publishing, 2018. ISBN: 9783030012526. DOI: 10.1007/978-3-030-01252-6_33. URL: http://dx.doi.org/10.1007/978-3-030-01252-6_33 (cited on pp. 5, 30, 32, 44).

[Che+23] J. Cheng, J. Ye, Z. Deng, J. Chen, T. Li, H. Wang, Y. Su, Z. Huang, J. Chen, L. Jiang, H. Sun, J. He, S. Zhang, M. Zhu, and Y. Qiao. SAM-Med2D, 2023. arXiv: 2308.16184 [cs.CV]. URL: https://arxiv.org/abs/2308.16184 (cited on pp. 1, 9).

[Che+26a] C. Che, Z. Wang, P. Yang, C. Wang, H. Ma, and Z. Shi. LoRA in LoRA: towards parameter-efficient architecture expansion for continual visual instruction tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence* (January 20–27, 2026), volume 40, pages 19978–19986, Singapore. Association for the Advancement of Artificial Intelligence, March 2026. DOI: 10.1609/aaai.v40i24.39082 (cited on pp. 13, 20).

[Che+26b] C.-S. Chen, X. Zhang, G.-Y. Chen, Q. Xie, F. Zhang, and E.-J. Kuo. Few-shot continual learning for 3D brain MRI with frozen foundation models, 2026. arXiv: 2602.23533 [eess.IV]. URL: https://arxiv.org/abs/2602.23533 (cited on pp. 4, 12, 17, 19, 21, 63).

[Che+26c] Z. Chen, Y. Gao, M. Han, Y. Liu, Z. Chen, D. Yang, and L. Zhang. Forging a dynamic memory: retrieval-guided continual learning for generalist medical foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 3–7, 2026), pages 32309–32321, Denver, CO, USA, June 2026. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Chen_Forging_a_Dynamic_Memory_Retrieval-Guided_Continual_Learning_for_Generalist_Medical_CVPR_2026_paper.html (cited on pp. 13, 14, 20).

[Çiç+16] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3D U-Net: learning dense volumetric segmentation from sparse annotation. In S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, and W. Wells, editors, *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016. 19th International Conference, Athens, Greece, October 17–21, 2016, Proceedings, Part II* (October 17–21, 2016), volume 9901 of *Lecture Notes in Computer Science*, pages 424–432, Cham. Springer International Publishing, 2016. ISBN: 978-3-319-46723-8. DOI: 10.1007/978-3-319-46723-8_49 (cited on p. 24).

[DAl+23] M. D'Alessandro, A. Alonso, E. Calabrés, and M. Galar. Multimodal parameter-efficient few-shot class incremental learning. In *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)* (October 2–6, 2023), pages 3385–3395, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCVW60793.2023.00364. URL: http://dx.doi.org/10.1109/ICCVW60793.2023.00364 (cited on pp. 5, 14, 21, 29, 30, 51, 77).

[DeL+22] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2022. DOI: 10.1109/TPAMI.2021.3057446 (cited on pp. 2, 3, 6, 12, 32, 49, 50, 77).

[Des+21] A. D. Desai, A. M. Schmidt, E. B. Rubin, C. M. Sandino, M. S. Black, V. Mazzoli, K. J. Stevens, R. Boutin, C. Ré, G. E. Gold, B. Hargreaves, and A. Chaudhari. SKM-TEA: a dataset for accelerated MRI reconstruction with dense image labels for quantitative clinical evaluation. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks* (December 6–14, 2021), volume 1, Virtual,

2021. URL: https://openreview.net/forum?id=YDMFgD_qJuA (cited on pp. 1, 2, 5, 10, 35, 36, 38, 75–77).

[Dic45] L. R. Dice. Measures of the amount of ecologic association between species. *Ecology*, 26(3):297–302, July 1945. DOI: 10.2307/1932409 (cited on p. 30).

[Dis+25] T. Dissanayake, Y. George, D. Mahapatra, S. Sridharan, C. Fookes, and Z. Ge. Few-shot learning for medical image segmentation: a review and comparative study. *ACM Computing Surveys*, 58(1):11, September 2025. ISSN: 0360-0300. DOI: 10.1145/3746224. URL: https://doi.org/10.1145/3746224 (cited on pp. 10, 11).

[Du+24] Y. Du, F. Bai, T. Huang, and B. Zhao. SegVol: universal and interactive volumetric medical image segmentation. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems* (December 10–15, 2024), volume 37, pages 110746–110783, Vancouver, BC, Canada. Curran Associates, Inc., 2024. DOI: 10.52202/079017-3516. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/c7c7cf10082e454b9662a686ce6f1b6f-Paper-Conference.pdf (cited on pp. 9, 20).

[Efr+93] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*, number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, New York, 1993. ISBN: 978-0-412-04231-7. DOI: 10.1007/978-1-4899-4541-9 (cited on p. 57).

[Gao+26a] Z. Gao and P. Morel. MedPEFT-CL: dual-phase parameter-efficient continual learning with medical semantic adapter and bidirectional memory consolidation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (March 6–10, 2026), pages 7699–7708, Tucson, AZ, USA. IEEE, March 2026. DOI: 10.1109/WACV61042.2026.00743 (cited on pp. 4, 17–19, 22, 49, 63).

[Gao+26b] Z. Gao and P. Morel. Prompt-aware adaptive elastic weight consolidation for continual learning in medical vision-language models. In J. Lokoč, L. Peška, J. Zahálka, S. Rudinac, M. Kastner, J. Chen, M.-C. Hu, J. Wu, and U. Sharma, editors, *MultiMedia Modeling. 32nd International Conference on Multimedia Modeling, MMM 2026, Prague, Czech Republic, January 29–31, 2026, Proceedings, Part I* (January 29–31, 2026), volume 16412 of *Lecture Notes*

in Computer Science, pages 349–363, Singapore. Springer Nature Singapore, 2026. DOI: 10.1007/978-981-95-6950-2_25 (cited on pp. 4, 17–19, 22).

[Gao26] Z. Gao. MedCRP-CL: continual medical image segmentation via Bayesian nonparametric semantic modality discovery. In T. Zhang, M. Dudik, M. Jaggi, A. Agarwal, S. Li, D. Schuurmans, J. Zhu, F. Berkenkamp, H. Dong, and A. Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning* (July 6–11, 2026), volume 306 of *Proceedings of Machine Learning Research*, pages 33385–33403, Seoul, South Korea. PMLR, 2026. URL: https://proceedings.mlr.press/v306/gao26s.html (cited on pp. 4, 17, 19, 21, 75, 79).

[Glo+10] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh and M. Titterington, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy. PMLR, 2010. URL: https://proceedings.mlr.press/v9/glorot10a.html (cited on p. 51).

[Gon+23] C. González, A. Ranem, D. Pinto dos Santos, A. Othman, and A. Mukhopadhyay. Lifelong nnU-Net: a framework for standardized medical continual learning. *Scientific Reports*, 13(1):9381, June 2023. DOI: 10.1038/s41598-023-34484-2 (cited on pp. 15, 16, 21, 63).

[Guo+25] D. Guo, Z. Ji, Y. Su, D. Zheng, H. Guo, P. Wang, K. Yan, Y. Wang, Q. Yu, Z. Li, M. Xu, J. Zhang, H. Li, J. Ge, T.-Y. Ho, B.-S. Huang, T. Ai, K. Zhao, N. Shen, Q. Wang, Y. Bian, T. Wu, P. Du, H. Zhang, F.-M. Kong, A. L. Yuille, C. H. Tan, C. Miao, P. J. Pickhardt, S. Yan, R. M. Summers, L. Lu, D. Jin, and X. Ye. A continual learning-driven model for accurate and generalizable segmentation of clinically comprehensive and fine-grained whole-body anatomies in CT, 2025. arXiv: 2503.12698 [eess.IV]. URL: https://arxiv.org/abs/2503.12698 (cited on pp. 3, 15).

[He+15] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In *2015 IEEE International Conference on Computer Vision (ICCV)* (December 7–13, 2015), pages 1026–1034, Santiago, Chile. IEEE, December 2015. DOI: 10.1109/ICCV.2015.123 (cited on p. 49).

[Hen+16] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs), 2016. arXiv: 1606.08415 [cs.LG]. URL: https://arxiv.org/abs/1606.08415 (cited on p. 51).

[Hin+15] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015. arXiv: 1503.02531 [stat.ML]. URL: https://arxiv.org/abs/1503.02531 (cited on p. 28).

[Hof+23] E. Hofstad, D. Bouget, A. Pedersen, K.-H. Støverud, T. Langø, and H. O. Leira. AeroPath: an airway segmentation benchmark dataset with challenging pathology. Version 1. Licence: Creative Commons Attribution 4.0 International. Zenodo, November 3, 2023. DOI: 10.5281/zenodo.10069289. URL: https://zenodo.org/records/10069289 (visited on 10/19/2026) (cited on p. 37).

[Hu+22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9 (cited on pp. 5, 11, 28, 49, 74, 77).

[Hug26] VoxTell: free-text promptable universal 3D medical image segmentation. Model card of the mrokuss/VoxTell model repository. Hugging Face. URL: https://huggingface.co/mrokuss/VoxTell (visited on 11/07/2026) (cited on pp. 39, 44, 76).

[Ise+21] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, February 2021. ISSN: 1548-7105. DOI: 10.1038/s41592-020-01008-z. URL: https://doi.org/10.1038/s41592-020-01008-z (cited on pp. 1, 24, 39, 42).

[Ise+24] F. Isensee, T. Wald, C. Ulrich, M. Baumgartner, S. Roy, K. Maier-Hein, and P. F. Jäger. nnU-Net revisited: a call for rigorous validation in 3D medical image segmentation. In M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir, and J. A. Schnabel, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024. 27th International Conference, Marrakesh, Morocco, October 6–10, 2024, Proceedings, Part IX* (October 6–10, 2024), volume 15009 of *Lecture Notes in Computer Science*, pages 488–498, Cham. Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-72114-4_47 (cited on p. 25).

[Ise+26] F. Isensee and Helmholtz Imaging Applied Computer Vision Lab. nnU-Net. Source code of the nnunetv2 package in the MIC-DKFZ/nnUNet repository. Version 2.7.0. GitHub. April 9, 2026. URL: https://github.com/MIC-DKFZ/nnUNet/tree/v2.7.0 (visited on 11/12/2026) (cited on p. 57).

[Ji+23] Z. Ji, D. Guo, P. Wang, K. Yan, L. Lu, M. Xu, Q. Wang, J. Ge, M. Gao, X. Ye, and D. Jin. Continual segment: towards a single, unified and non-forgetting continual segmentation model of 143 whole-body organs in CT scans. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 21083–21094, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCV51070.2023.01933 (cited on pp. 3, 15).

[Kal23] D. Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA, 2023. arXiv: 2312.03732 [cs.CL]. URL: https://arxiv.org/abs/2312.03732 (cited on pp. 28, 49, 69).

[Kir+17] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN: 1091-6490. DOI: 10.1073/pnas.1611835114. URL: http://dx.doi.org/10.1073/pnas.1611835114 (cited on pp. 15, 48).

[Kir+23] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar, and R. Girshick. Segment Anything. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 3992–4003, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCV51070.2023.00371 (cited on p. 9).

[Li+18] Z. Li and D. Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, December 2018. DOI: 10.1109/TPAMI.2017.2773081 (cited on pp. 17, 28).

[Li+24] Z. Li, Y. Li, Q. Li, P. Wang, D. Guo, L. Lu, D. Jin, Y. Zhang, and Q. Hong. LViT: language meets vision transformer in medical image segmentation. *IEEE Transactions on Medical Imaging*, 43(1):96–107, January 2024. DOI: 10.1109/TMI.2023.3291719 (cited on p. 9).

[Li+25] Y. Li, G. Pang, W. Suo, C. Jing, Y. Xi, L. Liu, H. Chen, G. Liang, and P. Wang. CoLeCLIP: open-domain continual learning via joint task prompt and vocabulary learning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(8):15137–15151, August 2025. DOI: 10.1109/TNNLS.2025.3547882 (cited on p. 14).

[Li+26] W. Li, J. Zhang, G. Wang, J. Ning, Y. Chen, G. Yang, and L. Gu. C^{2}GR: coupled comprehensive generative replay for a continually learnable universal segmentation model, 2026. arXiv: 2606.23473 [cs.CV]. URL: https://arxiv.org/abs/2606.23473 (cited on pp. 15, 21, 79).

[Liu+22] P. Liu, X. Wang, M. Fan, H. Pan, M. Yin, X. Zhu, D. Du, X. Zhao, L. Xiao, L. Ding, X. Wu, and S. K. Zhou. Learning incrementally to segment multiple organs in a CT image. In L. Wang, Q. Dou, P. T. Fletcher, S. Speidel, and S. Li, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2022. 25th International Conference, Singapore, September 18–22, 2022, Proceedings, Part IV* (September 18–22, 2022), volume 13434 of *Lecture Notes in Computer Science*, pages 714–724, Cham. Springer Nature Switzerland, 2022. DOI: 10.1007/978-3-031-16440-8_68 (cited on pp. 14, 15).

[Liu+23a] J. Liu, Y. Zhang, J.-N. Chen, J. Xiao, Y. Lu, B. A. Landman, Y. Yuan, A. Yuille, Y. Tang, and Z. Zhou. CLIP-driven universal model for organ segmentation and tumor detection. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 21095–21107, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCV51070.2023.01934. URL: http://dx.doi.org/10.1109/ICCV51070.2023.01934 (cited on pp. 10, 20).

[Liu+23b] X. Liu, H. A. Shih, F. Xing, E. Santarnecchi, G. El Fakhri, and J. Woo. Incremental learning for heterogeneous structure segmentation in brain tumor MRI. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part II* (October 8–12, 2023), volume 14221 of *Lecture Notes in Computer Science*, pages 46–56, Cham. Springer Nature Switzerland, 2023. DOI: 10.1007/978-3-031-43895-0_5 (cited on p. 15).

[Liu+24] J. Liu, Y. Zhang, K. Wang, M. C. Yavuz, X. Chen, Y. Yuan, H. Li, Y. Yang, A. Yuille, Y. Tang, and Z. Zhou. Universal and extensible language-vision models for organ segmentation and tumor detection from abdominal computed tomography. *Medical Image Analysis*, 97:103226, October 2024. DOI: 10.1016/j.media.2024.103226 (cited on pp. 15, 16, 19, 21).

[Liu+26a] A. Liu, X. R. Cao, Y. Shen, Y. Lu, X. Li, Q. Chen, and J. Chen. MedSAM3: delving into segment anything with medical concepts, 2026. arXiv: 2511.19046 [cs.CV]. URL: https://arxiv.org/abs/2511.19046 (cited on p. 10).

[Liu+26b] Y. Liu, Q. Hong, L. Huang, A. Gomez-Villa, D. Goswami, T. Peng, X. Liu, J. van de Weijer, and Y. Tian. Continual learning for VLMs: a survey and taxonomy beyond forgetting, 2026. arXiv: 2508.04227 [cs.CV]. URL: https://arxiv.org/abs/2508.04227 (cited on pp. 5, 12, 13, 47, 48).

[Ly+26] S. T. Ly and H. V. Nguyen. Self-auditing parameter-efficient fine-tuning for few-shot 3D medical image segmentation. In G. Yang, E. Adeli, M. de Bruijne, B. W. Papież, S. Speidel, P. Tiwari, G. Zheng, M. Yaqub, Q. Dou, and I. Rekik, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2026. 29th International Conference, Strasbourg, France, September 27 – October 1, 2026, Proceedings, Part III* (September 27–October 1, 2026), volume 16880 of *Lecture Notes in Computer Science*, pages 559–569, Cham. Springer Nature Switzerland, 2026. DOI: 10.1007/978-3-032-38069-2_53 (cited on p. 11).

[Ma+24] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang. Segment anything in medical images. *Nature Communications*, 15(1):654, January 2024. ISSN: 2041-1723. DOI: 10.1038/s41467-024-44824-z. URL: http://dx.doi.org/10.1038/s41467-024-44824-z (cited on pp. 1, 9).

[Mic+19] U. Michieli and P. Zanuttigh. Incremental learning techniques for semantic segmentation. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)* (October 27–28, 2019), pages 3205–3212, Seoul, Korea (South). IEEE, October 2019. DOI: 10.1109/ICCVW.2019.00400 (cited on p. 15).

[Ozd+18] F. Ozdemir, P. Fuernstahl, and O. Goksel. Learn the new, keep the old: extending pretrained models with new anatomy and images. In A. F. Frangi, J. A. Schnabel, C. Davatzikos, C. Alberola-López, and G. Fichtinger, editors,

Medical Image Computing and Computer Assisted Intervention – MICCAI 2018. 21st International Conference, Granada, Spain, September 16–20, 2018, Proceedings, Part IV (September 16–20, 2018), volume 11073 of *Lecture Notes in Computer Science*, pages 361–369, Cham. Springer International Publishing, 2018. ISBN: 9783030009373. DOI: 10.1007/978-3-030-00937-3_42. URL: http://dx.doi.org/10.1007/978-3-030-00937-3_42 (cited on p. 15).

[Rad+21] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning* (July 18–24, 2021), volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, Virtual. PMLR, 2021. URL: https://proceedings.mlr.press/v139/radford21a.html (cited on pp. 9, 12).

[Ran+22] A. Ranem, C. Gonzalez, and A. Mukhopadhyay. Continual hippocampus segmentation with transformers. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (June 19–20, 2022), pages 3710–3719, New Orleans, LA, USA. IEEE, June 2022. DOI: 10.1109/CVPRW56347.2022.00415 (cited on p. 16).

[Rok+26] M. Rokuss, M. Langenberg, Y. Kirchhoff, F. Isensee, B. Hamm, C. Ulrich, S. Regnery, L. Bauer, E. Katsigiannopulos, T. Norajitra, and K. Maier-Hein. VoxTell: free-text promptable universal 3D medical image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 3–7, 2026), pages 37538–37557, Denver, CO, USA, June 2026. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Rokuss_VoxTell_Free-Text_Promptable_Universal_3D_Medical_Image_Segmentation_CVPR_2026_paper.html (cited on pp. 1, 2, 5, 10, 12, 14, 20, 24–27, 37, 38, 42, 44, 45, 47, 56, 76, 77).

[Rok26] M. Rokuss. Question on VoxTell v1.1 training data (PanTrack). Comment by the maintainer in issue #15 of the MIC-DKFZ/VoxTell repository. GitHub. October 2, 2026. URL: https://github.com/MIC-DKFZ/VoxTell/issues/15#issuecomment-5949825606 (visited on 11/07/2026) (cited on pp. 38, 76).

[Sil+25] J. Silva-Rodríguez, J. Dolz, and I. Ben Ayed. Towards foundation models and few-shot parameter-efficient fine-tuning for volumetric organ segmentation.

Medical Image Analysis, 103:103596, July 2025. DOI: 10.1016/j.media.2025.103596 (cited on pp. 11, 20, 67).

[Staa] Stanford MIMI Lab. SKM-TEA tutorial. Google Colab notebook, linked as Tutorial from the SKM-TEA code repository (https://github.com/StanfordMIMI/skm-tea). URL: https://colab.research.google.com/drive/1PluqK77pobD5dXE7zzBLEAeBgaaeGKXa (visited on 10/19/2026) (cited on p. 36).

[Stab] Stanford MIMI Lab. Stanford knee MRI multi-task evaluation (SKM-TEA) dataset. Dataset documentation (DATASET.md) in the SKM-TEA code repository (https://github.com/StanfordMIMI/skm-tea). URL: https://github.com/StanfordMIMI/skm-tea/blob/main/DATASET.md (visited on 10/19/2026) (cited on pp. 35, 36, 39).

[Stø+24] K.-H. Støverud, D. Bouget, A. Pedersen, H. O. Leira, T. Amundsen, T. Langø, and E. F. Hofstad. AeroPath: an airway segmentation benchmark dataset with challenging pathology and baseline method. *PLOS ONE*, 19(10):e0311416, 2024. DOI: 10.1371/journal.pone.0311416 (cited on pp. 5, 35–37, 77).

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems* (December 4–9, 2017), volume 30, pages 5998–6008, Long Beach, CA, USA. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf (cited on pp. 24, 25).

[vdVen+25] G. M. van de Ven, N. Soures, and D. Kudithipudi. Continual learning and catastrophic forgetting. In J. T. Wixted, editor, *Learning and Memory: A Comprehensive Reference*, pages 153–168. Elsevier, 3rd edition, 2025. ISBN: 9780443157554. DOI: 10.1016/B978-0-443-15754-7.00073-0. URL: http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0 (cited on pp. 12, 27).

[Wan+22] Y. Wang, Z. Huang, and X. Hong. S-Prompts learning with pre-trained transformers: an Occam's razor for domain incremental learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh,

editors, *Advances in Neural Information Processing Systems* (November 28–December 9, 2022), volume 35, pages 5682–5695, New Orleans, LA, USA. Curran Associates, Inc., 2022. DOI: 10.52202/068431-0411. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/25886d7a7cf4e33fd44072a0cd81bf30-Paper-Conference.pdf (cited on p. 14).

[Wan+24] G. Wang, J. Ye, J. Cheng, T. Li, Z. Chen, J. Cai, J. He, and B. Zhuang. SAM-Med3D-MoE: towards a non-forgetting Segment Anything Model via mixture of experts for 3D medical image segmentation. In M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir, and J. A. Schnabel, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024. 27th International Conference, Marrakesh, Morocco, October 6–10, 2024, Proceedings, Part IX* (October 6–10, 2024), volume 15009 of *Lecture Notes in Computer Science*, pages 552–561, Cham. Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-72114-4_53 (cited on pp. 4, 16, 21, 79).

[Wan+26a] B. Wang, H. Zhou, Y. Gao, and X. Zhuang. Beyond forgetting in continual medical image segmentation: a comprehensive benchmark study, 2026. arXiv: 2605.06160 [cs.CV]. URL: https://arxiv.org/abs/2605.06160 (cited on pp. 16, 21, 63, 72).

[Wan+26b] J. Wang, W. Dai, H. Wang, S. Yang, H. Bi, and J. Sun. Continual alignment for SAM: rethinking foundation models for medical image segmentation in continual learning. In *2026 IEEE/CVF Conference on Computer Vision and Pattern Recognition – Findings Track (CVPRF)* (June 3–7, 2026), pages 7523–7532, Denver, CO, USA. IEEE, June 2026. DOI: 10.1109/CVPRF71909.2026.00722 (cited on pp. 4, 16, 17, 21, 79).

[Wu+25] B. Wu, W. Shi, J. Wang, and M. Ye. Synthetic data is an elegant GIFT for continual vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 10–17, 2025), pages 2813–2823, Nashville, TN, USA. IEEE, June 2025. DOI: 10.1109/CVPR52734.2025.00268 (cited on pp. 12, 14, 20).

[Ye+25] Y. Ye, Y. Wu, X. Luo, H. Zhang, Z. Chen, T. Dang, Y. Zhang, and Y. Xia. MedSeqFT: sequential fine-tuning foundation models for 3D medical image

segmentation, 2025. arXiv: 2509.06096 [cs.CV]. URL: https://arxiv.org/abs/2509.06096 (cited on pp. 3, 17, 19, 21).

[Yu+24] J. Yu, Y. Zhuge, L. Zhang, P. Hu, D. Wang, H. Lu, and Y. He. Boosting continual learning of vision-language models via mixture-of-experts adapters. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 16–22, 2024), pages 23219–23230, Seattle, WA, USA. IEEE, June 2024. DOI: 10.1109/CVPR52733.2024.02191 (cited on pp. 14, 79).

[Zan+24] M. Zanella and I. Ben Ayed. Low-rank few-shot adaptation of vision-language models. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (June 17–18, 2024), pages 1593–1603, Seattle, WA, USA. IEEE, June 2024. DOI: 10.1109/CVPRW63382.2024.00166 (cited on pp. 11, 20, 49).

[Zha+23] Y. Zhang, X. Li, H. Chen, A. L. Yuille, Y. Liu, and Z. Zhou. Continual learning for abdominal multi-organ and tumor segmentation. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part II* (October 8–12, 2023), volume 14221 of *Lecture Notes in Computer Science*, pages 35–45, Cham. Springer Nature Switzerland, 2023. DOI: 10.1007/978-3-031-43895-0_4 (cited on pp. 3, 16, 19, 21, 50).

[Zha+24] W. Zhang, P. Janson, R. Aljundi, and M. Elhoseiny. Overcoming generic knowledge loss with selective parameter update. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 16–22, 2024), pages 24046–24056, Seattle, WA, USA. IEEE, June 2024. DOI: 10.1109/CVPR52733.2024.02270 (cited on pp. 4, 13, 32, 42, 77, 79).

[Zha+25a] H. Zhang, Y. Liang, H. Kuang, L. Cen, Z. Qu, Y. Cen, M. Zeng, and S. Kan. Contrastive regularization over LoRA for multimodal biomedical image incremental learning. In *Proceedings of the 33rd ACM International Conference on Multimedia*, MM '25, pages 7500–7509, Dublin, Ireland. ACM, October 2025. DOI: 10.1145/3746027.3754917. URL: http://dx.doi.org/10.1145/3746027.3754917 (cited on pp. 13, 21).

[Zha+25b] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 Embedding: advancing text embedding and reranking through foundation models, 2025. arXiv: 2506.05176 [cs.CL]. URL: https://arxiv.org/abs/2506.05176 (cited on p. 25).

[Zha+25c] T. Zhao, Y. Gu, J. Yang, N. Usuyama, H. H. Lee, S. Kiblawi, T. Naumann, J. Gao, A. Crabtree, J. Abel, C. Moung-Wen, B. Piening, C. Bifulco, M. Wei, H. Poon, and S. Wang. A foundation model for joint segmentation, detection and recognition of biomedical objects across nine modalities. *Nature Methods*, 22(1):166–176, 2025. ISSN: 1548-7091. DOI: 10.1038/s41592-024-02499-w (cited on p. 10).

[Zha+25d] Z. Zhao, Y. Zhang, C. Wu, X. Zhang, X. Zhou, Y. Zhang, Y. Wang, and W. Xie. Large-vocabulary segmentation for medical images with text prompts. *npj Digital Medicine*, 8(1):566, September 2025. DOI: 10.1038/s41746-025-01964-w (cited on pp. 1, 10, 20).

[Zha+26] T. Zhang, C. Wang, Y. Li, Z. Chen, X. Zhan, and H. Shan. Skill-evolving grounded reasoning for free-text promptable 3D medical image segmentation. In G. Yang, E. Adeli, M. de Bruijne, B. W. Papież, S. Speidel, P. Tiwari, G. Zheng, M. Yaqub, Q. Dou, and I. Rekik, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2026. 29th International Conference, Strasbourg, France, September 27 – October 1, 2026, Proceedings, Part VII* (September 27–October 1, 2026), volume 16884 of *Lecture Notes in Computer Science*, pages 536–545, Cham. Springer Nature Switzerland, 2026. DOI: 10.1007/978-3-032-38085-2_50 (cited on p. 10).

[Zhe+23] Z. Zheng, M. Ma, K. Wang, Z. Qin, X. Yue, and Y. You. Preventing zero-shot transfer degradation in continual learning of vision-language models. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 19068–19079, Paris, France. IEEE, 2023. DOI: 10.1109/ICCV51070.2023.01752 (cited on pp. 2, 5, 6, 12–14, 20, 29, 30, 32, 33, 52, 53, 57, 62, 77, 85).

[Zho+22a] K. Zhou, J. Yang, C. C. Loy, and Z. Liu. Conditional prompt learning for vision-language models. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 18–24, 2022), pages 16795–16804, New

Orleans, LA, USA. IEEE, June 2022. DOI: 10.1109/CVPR52688.2022.01631 (cited on pp. 5, 11, 29, 51).

[Zho+22b] K. Zhou, J. Yang, C. C. Loy, and Z. Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 130(9):2337–2348, 2022. DOI: 10.1007/s11263-022-01653-1 (cited on pp. 11, 29).